

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École Nationale d'Électronique
et des Télécommunications de Sfax



Ingénieur en :
Ingénierie des Données et Systèmes
Décisionnels

Option :
Informatique Décisionnelle

Projet de fin d'études
N° d'ordre : IDSD-ID-3-2025-016

OIP -FR-01-00
05/05/2025

MEMOIRE

Présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

En vue de l'obtention du
Diplôme National d'Ingénieur en :
Ingénierie des Données et Systèmes Décisionnels

Option :
Informatique Décisionnelle

Par :

JMAL Amir

**Développement d'une application intelligente pour la
génération de cas de test et de scripts d'automatisation
intitulée « Smart QA »**

Soutenu le 20-06-2025, devant la commission d'examen :

M.	OUNI Tarak	Président
Mme.	REKIK Molka	Examinatrice
M.	BEN HAMED Bassem	Encadrant Académique
M.	HADDAR Boukthir	Encadrant Académique
Mme.	MEZGHANNI Rim	Encadrante Professionnelle
Mme.	AMICH Randa	Encadrante Professionnelle



DEDICATION

Je dédie ce modeste travail, ainsi que ma profonde gratitude, à ma mère et mon père, pour l'éducation qu'ils m'ont prodiguée, au prix de nombreux sacrifices, et pour les valeurs de devoir et de respect qu'ils m'ont transmises depuis mon enfance.

À ma chère sœur Amal et à mon cher frère Ahmed vous m'avez toujours soutenu et encouragé tout au long de mon parcours académique. Votre présence, vos mots, et votre réconfort ont été pour moi une force inestimable.

À tous mes amis, pour votre amitié sincère, votre bonne humeur, et votre soutien constant dans les moments de joie comme dans les moments de doute.

À tous mes professeurs, en témoignage de ma reconnaissance et de mon profond respect, pour l'inspiration, les savoirs et les valeurs que vous m'avez transmis.

Enfin, à toutes celles et ceux qui ont contribué, de près ou de loin, à la réalisation de ce travail, je vous remercie du fond du cœur.

À vous tous,

Je dédie ce travail.



REMERCIEMENT

Avec un immense plaisir, j'exprime mes vifs remerciements et ma profonde gratitude à tous ceux qui ont contribué à l'élaboration de ce projet de fin d'études.

Je tiens tout d'abord à remercier l'ensemble de l'équipe pédagogique de l'École Nationale d'Électronique et des Télécommunications de Sfax, ainsi que les intervenants professionnels, pour la qualité de la formation théorique dispensée tout au long de notre cursus.

Je remercie en particulier Monsieur **Bassem BEN HAMED** et Monsieur **Boukthir HAD-DAR**, mes encadrants académiques, pour leur disponibilité, leur patience et la qualité de leur accompagnement tout au long de ce travail.

Je tiens à remercier Monsieur **Hatem JARRAYA**, directeur de Freedom Of Dev Services, ainsi qu'à toute l'équipe de FOD, pour leur accueil chaleureux, leur bienveillance et l'esprit d'équipe dont ils ont fait preuve, qui ont grandement facilité mon intégration dans un environnement professionnel aussi stimulant qu'enrichissant.

J'adresse mes vifs remerciements à Madame **Rim MEZGHANNI**, mon encadrante industrielle, pour son encadrement rigoureux, sa disponibilité constante et ses conseils judicieux. Je remercie chaleureusement également Madame **Randa AMICH** pour son rôle en tant que Product Owner tout au long de mon stage. Ses orientations, ses remarques pertinentes et ses recommandations m'ont été d'une grande utilité pour la bonne conduite de ce projet.

Enfin, j'adresse mes remerciements les plus sincères aux membres du jury pour le temps qu'ils m'ont accordé, l'intérêt porté à ce travail, ainsi que pour l'honneur qu'ils me font en acceptant d'évaluer ce projet.



TABLE DES MATIÈRES

LISTE DES FIGURES	viii
LISTE DES TABLES	ix
LISTE DES ABRÉVIATIONS	x
INTRODUCTION GÉNÉRALE	1
1 Contexte Général	3
1.1 INTRODUCTION	4
1.2 Présentation de l'organisme d'accueil	4
1.2.1 Freedom Of Dev Services	4
1.2.2 Secteurs d'activité	5
1.3 Présentation du projet	6
1.3.1 Concepts Clés	6
1.3.2 Étude de l'existant	8
1.3.3 Critique de l'existant	12
1.3.4 Solution proposée	14
1.3.5 Objectifs de la solution	15
1.4 Méthodologie de travail	16
1.4.1 Choix de la méthodologie	16
1.4.2 Méthodologie Scrum	17
1.5 Langage de modélisation	18
1.6 Planification du projet par sprints	19
1.7 CONCLUSION	19
2 Spécification et Analyse des exigences	20
2.1 INTRODUCTION	21
2.2 Spécification des exigences	21
2.2.1 Identification des acteurs	21

2.2.2	Exigences fonctionnelles	22
2.2.3	Exigences non fonctionnelles	24
2.3	Exigences techniques	25
2.3.1	Environnement matériel	25
2.3.2	Environnement logiciel	25
2.4	Pilotage du projet avec Scrum	28
2.4.1	Backlog du produit	29
2.4.2	Utilisation d’Azure DevOps pour le suivi du projet	33
2.4.3	Préparation de l’application à Tester <i>QA Task Manager</i>	34
2.5	Diagramme des cas d’utilisation	34
2.5.1	Description textuelle de « S’authentifier »	36
2.5.2	Description textuelle de « Configurer les projets »	37
2.5.3	Description textuelle de « Générer les cas de test »	39
2.5.4	Description textuelle de « Configurer et générer le rapport de test »	40
2.5.5	Description textuelle de « Consulter le tableau de bord »	40
2.6	Diagramme d’activité	41
2.7	CONCLUSION	43
3	Étude Conceptuelle	44
3.1	INTRODUCTION	45
3.2	Conception générale	45
3.2.1	Architecture globale et choix technologiques	45
3.2.2	Architecture logique	47
3.3	Conception détaillée	49
3.3.1	Diagramme de classes	49
3.3.2	Diagrammes de séquence	51
3.3.2.1	Diagramme de séquence de « S’authentifier »	51
3.3.2.2	Diagramme de séquence de « Générer un rapport de test »	52
3.3.2.3	Diagramme de séquence de « Lancer un test automatisé »	52
3.4	Étude théorique des modules intelligents de génération des cas de test et des scripts de test	53
3.4.1	Approches théoriques communes pour les deux modules	53
3.4.2	Comparaison des approches	55
3.4.3	Prompt Engineering dans le contexte des LLM	56
3.4.3.1	Importance du prompt engineering	56
3.4.3.2	Techniques de prompt engineering	57

3.4.4	Choix du LLM : Évaluation des modèles et résultats	57
3.4.4.1	Protocole de benchmarking commun	57
3.4.4.2	Modèles évalués pour la génération des cas de test	58
3.4.4.3	Modèles évalués pour la génération des scripts de test	59
3.4.5	Modèle adopté : LLM DeepSeek-R1-Distill-Llama-8B	59
3.5	Mise en Œuvre des modules intelligents de génération des cas de test et des scripts de test	60
3.5.1	Outil utilisé	60
3.5.2	Mise en Œuvre du module de génération des cas de test	61
3.5.2.1	Conception du prompt	61
3.5.2.2	Résultats et validation des cas de test générés	62
3.5.3	Mise en œuvre du module de génération des scripts de test	64
3.5.3.1	Conception de prompt	65
3.5.3.2	Génération à base d'enregistrement	66
3.5.3.3	Génération basée sur la similarité	67
3.5.3.4	Génération simple (basée uniquement sur le cas de test, sans script de référence)	69
3.5.3.5	Résultat obtenu	70
3.6	CONCLUSION	71
4	Réalisation	72
4.1	INTRODUCTION	73
4.2	Description du travail réalisé	73
4.2.1	Configuration de Microsoft Entra ID	73
4.2.2	Interface d'authentification à l'application	74
4.2.3	Interface de configuration du projet et attribution des rôles	74
4.2.3.1	Attribution des rôles	75
4.2.3.2	Ajout d'un nouveau projet	76
4.2.3.3	Gestion des projets : édition, archivage et restauration	77
4.2.4	Interfaces de lancement d'une exécution de test	78
4.2.4.1	Configuration de l'URL de l'application à tester	79
4.2.4.2	Mise à jour des UserStories	80
4.2.4.3	Génération des cas de test et des scripts de test	81
4.2.4.4	Édition et suppression des cas de test et de leurs scripts	82
4.2.4.5	Lancement d'un enregistrement (recording) d'un cas de test	83
4.2.4.6	Exécution automatisée et génération des rapports de test	84
4.2.5	Interface de consultation des projets pour le développeur	86

4.2.6	Interface du tableau de bord	88
4.3	CONCLUSION	89
	CONCLUSION GÉNÉRALE	90
	BIBLIOGRAPHIE	90
	ANNEXES	93
A.1	Détails des fonctionnalités décrites dans le diagramme de cas d'utilisation . . .	93
A.1.1	Fonctionnalité A : Configurer les projets	93
A.1.2	Fonctionnalité B : Générer des cas de test	94
A.1.3	Fonctionnalité C : Générer des scripts de test et exécuter les tests	94
A.2	Diagrammes de séquence de «Ajouter un projet»	95



LISTE DES FIGURES

1.1	Logo de Freedom Of Dev Services	4
1.2	Workflow des états d'une UserStory	9
1.3	Processus actuel de test	11
1.4	Problématique et critique de l'existant	12
1.5	Processus de la solution proposée	14
1.6	Enchaînement du processus Scrum	18
2.1	Environnements de développement intégrés	25
2.2	Technologies Frontend	26
2.3	Frameworks Backend	26
2.4	Logo de PostgreSQL	27
2.5	Outils de développement et d'intégration	27
2.6	Logo de REST	27
2.7	Logo Postman	27
2.8	Logo de Playwright	27
2.9	Logo de Microsoft Entra ID	28
2.10	Logo de Ollama	28
2.11	Illustrations de l'utilisation d'Azure DevOps pour le suivi du projet.	33
2.12	Diagramme des cas d'utilisation	35
2.13	Diagramme d'activité de « mettre à jour les UserStories »	42
2.14	Diagramme d'activité de « lancer un recording »	42
3.1	Architecture globale de « Smart QA »	45
3.2	Architecture microservices de « Smart QA »	48
3.3	Diagramme de classe	50
3.4	Diagramme de Séquence de « S'authentifier »	51
3.5	Diagramme de Séquence de « Générer un rapport de test »	52
3.6	Diagramme de Séquence de « Lancer un test automatisé »	53
3.7	Workflow de la génération des cas de test	61

3.8	Workflow général du module de génération de scripts de test	65
3.9	Illustration de la similarité cosinus entre deux vecteurs de cas de test	68
4.1	Interface de Microsoft Entra ID	73
4.2	Étapes du processus d'authentification	74
4.3	Interface de liste des projets pour le Super Admin	74
4.4	Liste des membres récupérés depuis Azure DevOps	75
4.5	Interface d'attribution des rôles	75
4.6	Processus d'ajout d'un projet	76
4.7	Affichage des projets avec numérotation des actions possibles	77
4.8	Interface d'affichage d'un projet en mode lecture seule	77
4.9	Interface de modification d'un projet en mode édition	78
4.10	Interface de liste des projets pour le QA	78
4.11	Configuration de l'URL de test par le QA	79
4.12	Interface principale de l'application après connexion	80
4.13	Interface principale d'exécution de tests	80
4.14	Mise à jour des UserStories depuis Azure DevOps	81
4.15	Affichage des cas de test générés	81
4.16	Apparence des cas de test avant et après génération de script	82
4.17	Restriction de génération de script selon le statut de la UserStory	82
4.18	Affichage et modification d'un cas de test et de son script	83
4.19	Enregistrement manuel d'un scénario avec Playwright	84
4.20	Exécution des cas de test et affichage des résultats dans l'interface	84
4.21	Génération et téléchargement du rapport de test	85
4.22	Aperçu des différentes pages du rapport de test généré en PDF	86
4.23	Liste des projets assignés au développeur	86
4.24	Consultation des rapports générés pour un projet	87
4.25	Consultation d'un rapport spécifique	87
4.26	Vue globale des indicateurs à travers tous les projets	88
4.27	Vue détaillée du tableau de bord d'un projet	89
A.1	Diagramme des cas d'utilisation pour la configuration des projets	93
A.2	Diagramme des cas d'utilisation pour la génération des cas de test	94
A.3	Diagramme des cas d'utilisation pour la génération de script et l'exécution des tests	95
A.4	Diagramme de Séquence de «Ajouter un projet»	96



LISTE DES TABLES

1.1	Comparaison des outils de génération de cas de test	10
1.2	Comparaison des outils d'automatisation des tests	10
1.3	Les sprints du projet	19
2.1	Environnement matériel	25
2.2	Liste des features et UserStories	29
2.3	Description textuelle de « S'authentifier »	36
2.4	Description textuelle de « Configurer les projets »	37
2.5	Description textuelle de « Générer les cas de test »	39
2.6	Description textuelle de « Configurer et générer le rapport de test »	40
2.7	Description textuelle de « Consulter le tableau de bord »	41
3.1	Comparaison des approches de génération des cas de test et des scripts de test .	55
3.2	Comparaison des LLM pour la génération de cas de test	58
3.3	Comparaison des LLM pour la génération de scripts de test	59
3.4	Exemples de cas de test générés	63



LISTE DES ABRÉVIATIONS

DevOps Development and Operations

FOD Freedom Of Dev Services

IA Intelligence Artificielle

KPIs Key Performance Indicators

LLM Large Language Model

NLU Natural Language Understanding

NLP Natural Language Processing

QA Quality Assurance

UML Unified Modeling Language



INTRODUCTION GÉNÉRALE

De nos jours, nous assistons à un progrès rapide dans divers domaines, notamment dans l'informatique, ce qui impacte fortement la gestion des projets. Pour qu'un projet soit mené efficacement, il doit suivre un cycle de vie structuré comprenant l'analyse des besoins, la conception, le développement, le test, le déploiement, la maintenance et le support.

Ces étapes ne constituent pas seulement des attentes, mais bien des impératifs pour satisfaire le client et garantir la qualité du logiciel ainsi que la performance du produit final.

Dans ce contexte, le **test** joue un rôle crucial. Il permet de valider les fonctionnalités développées, d'identifier les anomalies et de s'assurer que le produit répond aux exigences spécifiées.

Le **testeur ou ingénieur QA (Quality Assurance)** joue un rôle clé dans le processus de validation logicielle. Il est chargé de concevoir des cas de test pertinents à partir des exigences fonctionnelles, d'exécuter les tests et d'analyser les résultats afin de garantir la conformité du logiciel. Cette activité requiert une compréhension fine des besoins métier ainsi qu'une approche rigoureuse et structurée pour assurer une couverture complète et pertinente des scénarios à tester. Il existe plusieurs types de tests, couvrant à la fois les aspects fonctionnels et non fonctionnels. Dans le cadre de notre projet, l'accent est mis sur les **tests fonctionnels**, qui vérifient que le logiciel se comporte conformément aux spécifications métier, ainsi que sur les **tests de non-régression**, qui permettent de s'assurer que les nouvelles modifications n'ont pas altéré les fonctionnalités existantes. Sans une phase de test rigoureuse et bien organisée, les risques de dysfonctionnements, de régressions et d'insatisfaction client augmentent considérablement, compromettant ainsi la qualité, la fiabilité et la maintenabilité du produit.

C'est dans ce cadre que s'inscrit notre projet de fin d'études, intitulé « **Smart QA** », réalisé au sein de l'entreprise *Freedom Of Dev Services (FOD)* dans le cadre de l'obtention du diplôme national d'ingénieur en ingénierie des données et systèmes décisionnels de l'Ecole Nationale d'Electronique et des Télécommunications de Sfax pour l'année universitaire 2024/2025.

Notre projet vise à concevoir une application web intelligente dédiée à la phase de test, destinée aux équipes qualité et développement. En intégrant des technologies d'intelligence artificielle et d'automatisation, cette solution permet de générer et d'exécuter automatiquement des scénarios et des scripts de test, tout en produisant des rapports d'analyse détaillés. Elle vise à accélérer les cycles de test, améliorer la couverture des vérifications et renforcer la qualité des livrables.

Ce rapport est organisé en plusieurs chapitres qui retracent les différentes étapes de notre projet. Le premier chapitre, intitulé « **Contexte Général** », présente l'organisme d'accueil, *FOD*, et situe notre projet dans son environnement. Nous y étudions également l'existant, les défis rencontrés, et la solution proposée. Le deuxième chapitre, « **Spécification et Analyse des Exigences** », détaille les besoins fonctionnels, illustrés par des diagrammes UML, ainsi que les besoins non fonctionnels de l'application et les technologies retenues. Le troisième chapitre, « **Étude Conceptuelle** », expose l'architecture adoptée et la modélisation des besoins, avec un focus particulier sur les modules de génération intelligente des cas de test et de scripts de test basés sur l'Intelligence Artificielle. Enfin, le quatrième chapitre, « **Réalisation** », décrit la mise en œuvre des différentes fonctionnalités et les interfaces développées.

Nous clôturerons ce rapport par une conclusion générale, où nous récapitulerons les travaux réalisés, les défis rencontrés, et les perspectives d'amélioration pour notre solution.

Contexte Général

Sommaire

1.1 INTRODUCTION	4
1.2 Présentation de l'organisme d'accueil	4
1.2.1 Freedom Of Dev Services	4
1.2.2 Secteurs d'activité	5
1.3 Présentation du projet	6
1.3.1 Concepts Clés	6
1.3.2 Étude de l'existant	8
1.3.3 Critique de l'existant	12
1.3.4 Solution proposée	14
1.3.5 Objectifs de la solution	15
1.4 Méthodologie de travail	16
1.4.1 Choix de la méthodologie	16
1.4.2 Méthodologie Scrum	17
1.5 Langage de modélisation	18
1.6 Planification du projet par sprints	19
1.7 CONCLUSION	19

1.1 INTRODUCTION

Ce chapitre décrit le projet « Smart QA » dans son contexte général. Dans la première section, nous présentons l'organisme d'accueil. Ensuite, nous étudions l'existant afin d'identifier la problématique à résoudre par notre solution proposée. Nous concluons en présentant la méthodologie de travail et la planification adoptée tout au long du projet.

1.2 Présentation de l'organisme d'accueil

Afin de situer notre travail dans son environnement de réalisation, nous présentons dans cette section l'organisme d'accueil en exposant ses secteurs d'activité.

1.2.1 Freedom Of Dev Services

La société Freedom Of Dev Services (FOD) [1], fondée en 2005, est une plateforme d'experts en ingénierie logicielle « nearshore » destinée à des organisations professionnelles actives dans un large éventail de secteurs. Elle contribue efficacement à la transformation digitale des entreprises. Tout en conservant son agilité, FOD fournit des solutions logicielles personnalisées de haute qualité.



FIGURE 1.1 – Logo de Freedom Of Dev Services

L'équipe de FOD s'appuie sur les meilleures technologies, les méthodologies les plus appropriées et la vision la plus adéquate pour délivrer des projets de haute valeur de toute taille et complexité. Sa mission est de fournir des services d'ingénierie logicielle sur mesure, permettant à ses clients de réduire leurs coûts, de gagner en compétitivité et d'accélérer leurs délais de mise sur le marché.

Entreprise totalement exportatrice, FOD compte aujourd'hui une soixantaine d'ingénieurs logiciels, répartis au sein d'équipes pluridisciplinaires expérimentées.

1.2.2 Secteurs d'activité

C'est une société spécialisée dans l'étude et le développement des logiciels sur mesure ainsi que dans la mise en place et le support des solutions logicielles « NearShore » et « OffShore » pour des clients européens. FOD travaille dans plusieurs domaines métiers très variés (par exemple le domaine de l'énergie, le domaine médical, le secteur bancaire, le domaine de la logistique, etc.). Les activités de FOD couvrent les prestations suivantes :

- Développement agile de logiciels.
- Développement d'applications mobiles.
- Développement de workflow.
- Analytique de Big Data.
- Développement d'applications métiers.

FOD offre un service de qualité dans le respect des contraintes de temps et de budget de ses clients, dans le but d'atteindre leurs objectifs en termes de besoins. Parmi ses différents services, nous pouvons citer :

- Développement de solutions sur mesure.
- Services nearshore d'ingénierie logicielle.
- Maintenance des applications.
- Test et validation des logiciels.
- Monitoring des solutions en production chez les clients.
- Étude, conseil et assistance informatique.
- Support client.
- Recherche et développement.
- Analyse et modélisation des processus métiers.

Dans le cadre de la coopération permanente de FOD avec l'université, elle propose chaque année des sujets de projets de fin d'études. L'un de ces sujets porte sur la phase de test dans le cycle de vie d'un logiciel. Ce sujet fait l'objet de la section suivante.

1.3 Présentation du projet

La qualité logicielle étant un enjeu majeur, les entreprises doivent renforcer leurs processus de validation. Les tests, pilotés par l'équipe de **Quality Assurance (QA)** [2], jouent un rôle central pour garantir la fiabilité, prévenir les régressions et répondre aux attentes des utilisateurs. En élaborant et exécutant des scénarios adaptés, le QA identifie les anomalies le plus tôt possible afin d'assurer une livraison logicielle fiable, de qualité et conforme aux exigences.

Une fois une fonctionnalité développée, elle est transmise au testeur QA pour validation. Celui-ci détecte les éventuels écarts entre le comportement observé et le comportement attendu, qui sont ensuite corrigés par les développeurs. Cependant, ces derniers, concentrés sur l'aspect technique, peuvent négliger certains cas d'usage. D'où l'importance d'intégrer le QA de manière continue dans le cycle de développement : cette démarche permet d'anticiper les risques, de mieux couvrir les besoins fonctionnels et de maintenir un haut niveau de qualité tout au long du projet.

1.3.1 Concepts Clés

Pour faciliter la compréhension des notions techniques abordées dans ce rapport, nous présentons ci-dessous les définitions des principaux concepts liés au **QA** et à la gestion des tests.

- **UserStory** [3] : Une description simple d'une fonctionnalité du point de vue de l'utilisateur final, comprenant une description, souvent au format « En tant que [type d'utilisateur], je veux [objectif] afin de [bénéfice attendu] », des critères d'acceptation pour définir les conditions de validation, et un statut indiquant son avancement dans le cycle de développement.

Exemple : « En tant qu'administrateur, je veux réinitialiser le mot de passe d'un utilisateur afin d'assurer l'accès au compte en cas d'oubli. »

Un besoin fonctionnel peut être décomposé en plusieurs UserStories, chacune représentant une partie spécifique du comportement attendu à implémenter.

- **Azure DevOps [4]** : Est une plateforme de gestion de projet qui fournit des outils pour le développement de logiciels, la gestion du code source, la planification des travaux, l'intégration continue et la livraison continue (CI/CD).

Il est utilisé par plusieurs équipes pour gérer leurs backlogs et les UserStories de manière centralisée et collaborative, et il permet de gérer efficacement le cycle de vie complet d'un projet de développement logiciel.

Ces UserStories passent par plusieurs statuts dans le processus de développement, allant de la planification à la validation, afin de suivre leur progression et garantir leur réalisation selon les critères définis.

Définitions de quelques statuts des UserStories dans Azure DevOps [5] :

- *New* : La UserStory vient d'être créée et n'a pas encore été analysée.
 - *Ready* : Les critères et exigences de la UserStory sont clairement définis, la rendant prête à être prise en charge.
 - *Active* : La UserStory est en cours de développement.
 - *Resolved* : La UserStory est finalisée au niveau du développement et planifiée pour la phase de test.
 - *Closed* : La UserStory est terminée, validée et approuvée par le client, indiquant qu'elle est complètement finalisée.
- **Critères d'acceptation** : Conditions spécifiques à remplir pour valider une UserStory. Ils peuvent être listés ou suivre le format *Given/When/Then*, où *Given* décrit le contexte initial, *When* l'action déclenchée, et *Then* le résultat attendu.
- Exemple* : « Given un utilisateur sur la page de connexion, When il clique sur 'Mot de passe oublié' et saisit son e-mail, Then il reçoit un e-mail avec un lien de réinitialisation ».

- **Cas de Test (Test Case) [6]** : Ensemble d'étapes, de conditions préalables et de résultats attendus pour vérifier une fonctionnalité.

Exemple :

- ID : CT-001
 - Titre : Vérification de la réinitialisation du mot de passe
 - Préconditions : L'utilisateur possède un compte valide et a accès à sa boîte e-mail.
 - Étapes :
 1. Accéder à la page de connexion de l'application.
 2. Cliquer sur le lien « Mot de passe oublié ».
 3. Saisir une adresse e-mail valide associée au compte.
 4. Valider la demande de réinitialisation.
 - Résultat attendu : L'utilisateur reçoit un e-mail contenant un lien de réinitialisation sécurisé.
 - Statut : Succès/Échec après exécution.
- **Rapport de Test [7]** : Document récapitulatif des résultats des tests, incluant les cas exécutés, les succès/échecs et les anomalies.

Exemple : « Résumé des tests effectués sur la version 1.0 du logiciel. »

1.3.2 Étude de l'existant

Actuellement, après avoir tenu des réunions avec le client pour valider un besoin fonctionnel, une ou plusieurs UserStories sont créées afin de documenter la nouvelle fonctionnalité à implémenter. Chaque UserStory comprend une description détaillée de la fonctionnalité, des critères d'acceptation nécessaires à sa validation, ainsi qu'un statut indiquant son avancement dans le processus de développement.

Une fois la UserStory créée à l'état **New** (les critères d'acceptation ne sont pas encore définis et le besoin n'est pas validé), elle passe à l'état **Ready** après clarification et validation des besoins avec le client. La description de la UserStory permet aux développeurs de comprendre précisément le besoin exprimé, tandis que les critères d'acceptation servent de référence aux

testeurs QA pour évaluer la conformité de la fonctionnalité et valider le comportement attendu de la fonctionnalité développée. À l'état **Ready**, la UserStory est transmise à l'équipe de développement, qui commence son implémentation, faisant passer son statut à **Active**.

Une fois le développement finalisé, la UserStory atteint l'état **Resolved** et la fonctionnalité développée est transmise à l'équipe de test. Les testeurs QA exécutent alors une série de tests basés sur les critères d'acceptation, couvrant différents scénarios pour identifier d'éventuels *bugs* ou anomalies. Si un test échoue, elle est renvoyée aux développeurs pour correction, ce qui peut nécessiter un retour à l'état **Active**. Une fois les tests réussis et après une démonstration et validation du client, la UserStory passe à l'état **Closed**, et les QA préparent des rapports de test pour documenter les résultats et les anomalies détectées. Ce processus est illustré dans la figure 1.2.

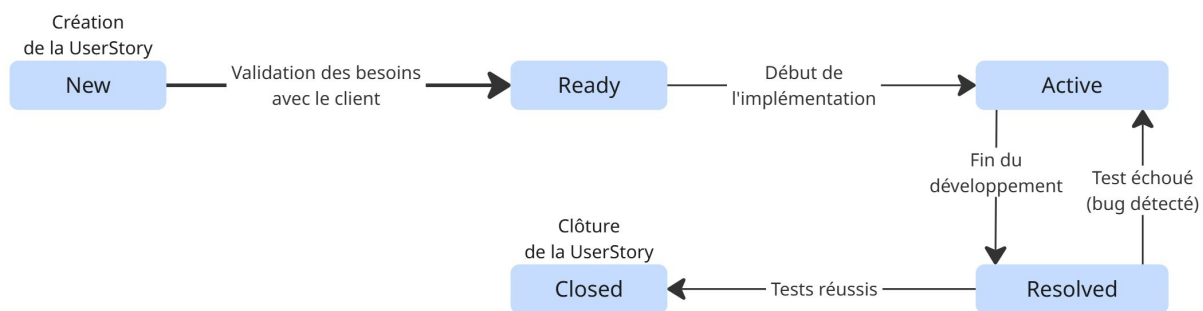


FIGURE 1.2 – Workflow des états d'une UserStory

Pour garantir la phase de test, les testeurs QA adoptent une stratégie centrée sur la préparation de scénarios fonctionnels, élaborés à partir des UserStories récupérées manuellement depuis Azure DevOps et consignées dans un document. Ils s'appuient à la fois sur leur description et sur les critères d'acceptation associés. Une fois les scénarios rédigés, ceux-ci sont exécutés soit manuellement, soit de manière automatisée pour les tests de non-régression, après la création des scripts, afin de comparer le comportement réel de l'application aux résultats attendus. Cette approche sera détaillée dans la section suivante.

Stratégie de test

Les équipes de test, qu'il s'agisse de testeurs manuels ou d'automatisation [14], analysent en détail les UserStories et leurs critères d'acceptation pour rédiger et structurer ces scénarios.

Par ailleurs, des outils peuvent être utilisés pour générer automatiquement des cas de test à partir des UserStories, facilitant ainsi la validation et l'organisation des cas dans un document centralisé regroupant l'ensemble des tests identifiés.

Au cours de notre recherche, nous présentons quelques outils que les QA peuvent utiliser pour optimiser la génération des cas de test. Ces outils sont comparés dans le tableau 1.1 ci-dessous, en mettant en évidence leurs descriptions, avantages et limites.

TABLE 1.1 – Comparaison des outils de génération de cas de test

Outil	Description	Avantages	Limites
ScopeMaster [8]	Analyse sémantique des UserStories pour générer des scénarios de test.	Réduction du temps d'analyse Suggestions basées sur le texte	Coût élevé (10€/UserStory) Pas d'exécution automatisée
AI Test Case Generator (Jira) [9]	Utilise l'IA (ChatGPT) pour générer des UserStories, cas de test et scripts pour Jira.	Intégration fluide avec Jira Génération automatisée	Limité à Jira Abonnement (10€/mois) Pas de réexécution multi-outils
GPT-4, Gemini, LLMs [10]	Génèrent des cas de test à partir de descriptions en langage naturel.	Usage flexible Adaptabilité élevée	Non spécialisé en test Résultats à valider manuellement

Une fois définis, ces cas de test sont exécutés soit manuellement, soit de manière automatisée, après la rédaction de scripts à l'aide d'outils tels que Selenium, Playwright, JUnit ou PyTest, dont quelques exemples sont présentés dans le tableau 1.2 ci-dessous.

TABLE 1.2 – Comparaison des outils d'automatisation des tests

Outil	Type	Avantages	Limites
Selenium [11]	Test UI automatisé	Open-source et puissant Large communauté	Configuration complexe Nécessite du code
Playwright [12]	Test End-to-End automatisé	Multi-navigateurs Supporte tests mobiles et API	Communauté plus petite
TestComplete [13]	Test visuel automatisé	Interface intuitive Supporte plusieurs langages	Payant

L'un des principaux défis du test manuel est non seulement l'exécution des tests de la nouvelle fonctionnalité, mais aussi la nécessité de rejouer systématiquement l'ensemble des tests précédents lorsqu'une nouvelle fonctionnalité est livrée. Ce processus permet de s'assurer que les fonctionnalités déjà validées ne sont pas impactées par des régressions (une régression désigne un bug ou un dysfonctionnement qui apparaît après une modification du code — mise à jour, correction de bug, ajout de fonctionnalité, etc. — ce qui signifie qu'une fonctionnalité qui marchait auparavant ne fonctionne plus correctement).

Cependant, cette approche devient rapidement lourde et chronophage à mesure que l'application évolue. C'est dans ce contexte que l'automatisation des tests prend toute son importance.

En mettant en place une stratégie de test automatisé, il devient possible d'exécuter rapidement et efficacement des tests de non-régression sur des fonctionnalités déjà testées manuellement, garantissant ainsi la stabilité du logiciel tout en réduisant le temps nécessaire aux validations. La transition d'une approche majoritairement manuelle vers une approche automatisée permet donc d'améliorer la fiabilité des tests, d'optimiser les ressources et d'accélérer le cycle de développement. Le processus décrit ci-dessus est illustré dans la figure 1.3.

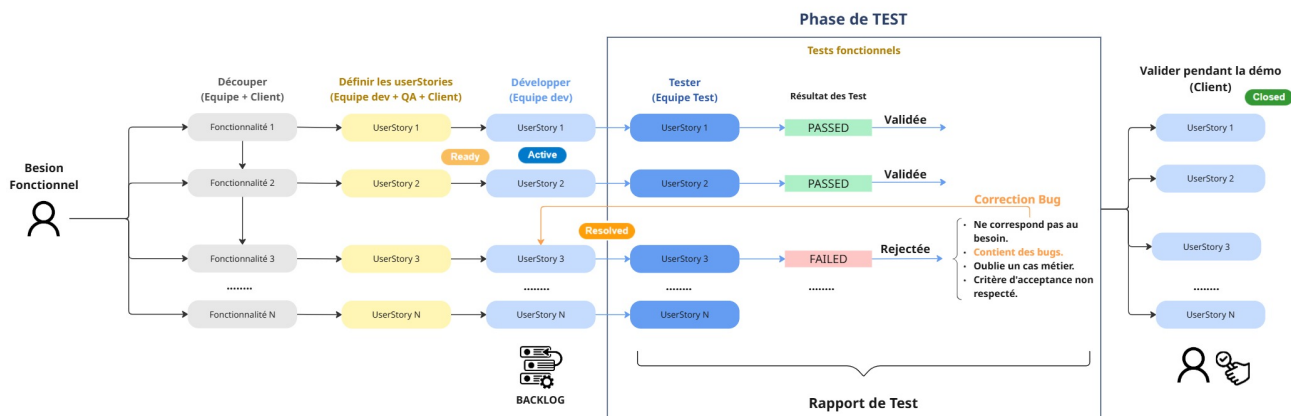


FIGURE 1.3 – Processus actuel de test

1.3.3 Critique de l'existant

Le processus actuel de génération et d'exécution des cas de test présente plusieurs limites, affectant l'efficacité, la qualité des tests et la productivité des équipes QA. Une analyse approfondie de la phase de test actuelle nous a permis d'identifier ses limites, présentées dans la figure 1.4 suivante.

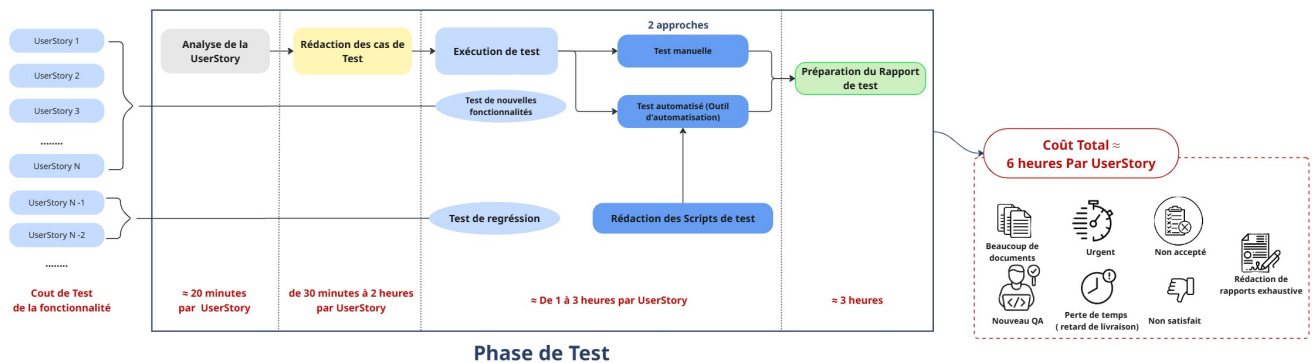


FIGURE 1.4 – Problématique et critique de l'existant

Les limitations identifiées dans ce processus sont :

- **Processus de test manuel long et dépendant de l'expertise humaine** : La création des scénarios de test repose sur une analyse approfondie des UserStories et des spécifications fonctionnelles, réalisée par les testeurs. Cette phase, dont la durée varie de 20 minutes à 2 heures par UserStory selon sa complexité, est particulièrement chronophage. À cela s'ajoutent l'exécution des tests et la génération des rapports, qui demandent également un temps conséquent. De plus, la qualité des scénarios dépend fortement de l'expertise des testeurs.
- **Extraction manuelle des UserStories** : Les testeurs doivent copier les UserStories depuis Azure DevOps vers un document séparé, ce qui alourdit la tâche et augmente les risques d'erreurs.
- **Couverture de test incomplète** : L'approche manuelle entraîne une couverture variable selon les projets. Certains scénarios ou critères d'acceptation peuvent être négligés, créant des failles potentielles dans la validation des fonctionnalités.

- **Méthodes de test non uniformes** : Chaque testeur utilise sa propre méthode, souvent basée sur des documents partagés, ce qui nuit à la cohérence et complique la synchronisation de l'équipe QA.
- **Intégration difficile des nouveaux membres** : Les nouveaux testeurs rencontrent des difficultés à s'approprier les processus en place et le travail déjà réalisé, ce qui ralentit et allonge le cycle de validation des tests.
- **Contraintes de délais et impact sur la productivité** : Les tests doivent être finalisés dans des délais stricts pour respecter les plannings de livraison, mobilisant ainsi des ressources importantes en temps, compétences et coordination, surtout pour des applications complexes ou des spécifications floues. Tout retard impacte la productivité, décale les livraisons, et peut entraîner des problèmes critiques avec le client, compromettant la qualité du produit.
- **Limites de l'automatisation actuelle** : Même si l'exécution des tests est souvent automatisée avec des outils comme Selenium ou Playwright, la rédaction des cas de test et surtout des scripts d'automatisation reste manuelle, complexe, et nécessite des compétences techniques avancées ainsi qu'un investissement en temps important. De plus, à chaque évolution d'une UserStory ou ajout de fonctionnalité, les cas de test et scripts doivent être mis à jour pour éviter les régressions. Sans suivi rigoureux, certains tests peuvent devenir obsolètes, compromettant la fiabilité de la validation.

De plus, les solutions actuelles se concentrent principalement sur l'exécution des tests ou leur intégration avec des outils spécifiques comme Jira, sans vraiment aider à créer et maintenir les cas de test. Cela complique leur utilisation, surtout que beaucoup de ces solutions sont payantes, ce qui peut être un obstacle pour les équipes avec un budget limité.

Dans ce contexte, automatiser et optimiser le processus de test devient indispensable pour améliorer l'efficacité des équipes QA.

1.3.4 Solution proposée

Pour pallier les limites de la méthode de travail actuelle, nous proposons de développer une application web intelligente dédiée aux équipes QA et Dev intitulée « Smart QA ».

Basée sur une architecture micro-services, cette solution intègre l'intelligence artificielle et l'automatisation pour optimiser et standardiser le processus de test en centralisant les exécutions ainsi que leurs résultats, tout en s'adaptant aux évolutions des UserStories. FOD nous a confié la mission de concevoir et d'implémenter cette solution, comme illustré à la figure 1.5.

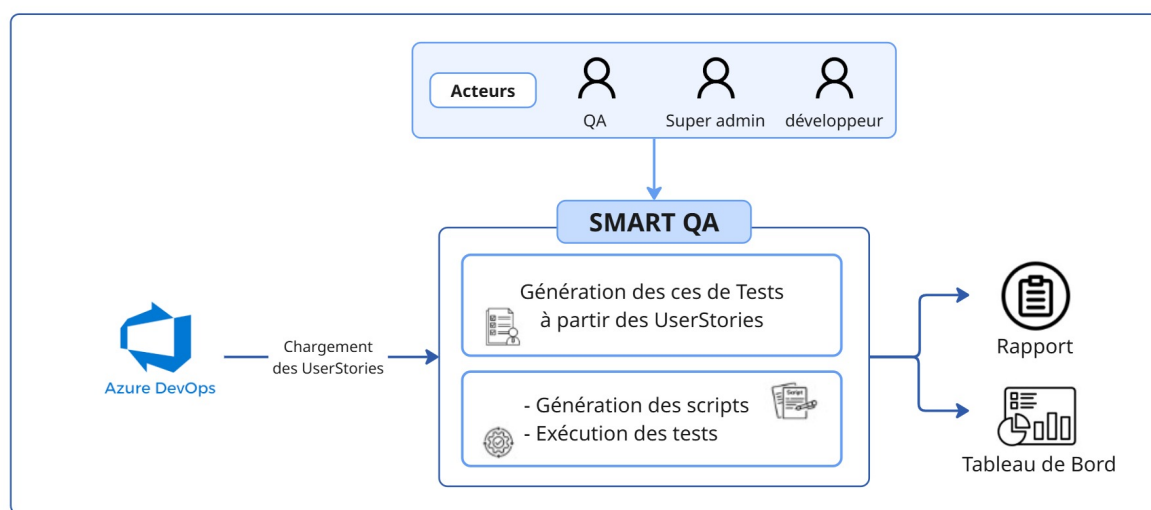


FIGURE 1.5 – Processus de la solution proposée

Notre solution apporte des améliorations significatives face aux limites identifiées dans l'existant. En automatisant la récupération des UserStories depuis Azure DevOps ainsi que la génération des cas de test, elle réduit considérablement le temps consacré aux phases de test tout en limitant la dépendance à l'expertise humaine, ce qui permet d'accélérer le processus de validation et d'assurer une meilleure qualité des livrables.

L'accès direct aux UserStories évite les tâches manuelles de copie et réduit les risques d'erreurs, tandis que la génération intelligente des cas de test garantit une couverture homogène et systématique. La génération automatique des scripts, l'exécution des tests et la production des rapports permettent de respecter les délais, même dans des contextes complexes, tout en renforçant la productivité globale.

De plus, la centralisation des UserStories, cas de test, scripts et rapports dans un environnement unifié favorise l'harmonisation des pratiques au sein de l'équipe QA. Cela facilite la collaboration entre les testeurs, en particulier sur des projets partagés, et permet aux nouveaux membres de s'adapter rapidement.

Enfin, la solution s'ajuste aux évolutions des UserStories, ce qui simplifie la maintenance des tests et limite les risques de régression. Un tableau de bord intégré offre une visibilité en temps réel sur l'état d'avancement du processus de test, facilitant ainsi le suivi et le pilotage global de la phase de tests.

1.3.5 Objectifs de la solution

Notre solution proposée vise à répondre aux défis identifiés dans le processus actuel en poursuivant des objectifs spécifiques, chacun adressant une limitation clé de l'existant :

- La récupération des UserStories depuis Azure DevOps pour centraliser les données nécessaires aux tests.
- La génération intelligente des cas de test à partir des UserStories en exploitant des approches d'intelligence artificielle.
- La génération intelligente des scripts de test associés aux cas de test générés, adaptés aux spécifications des UserStories, en exploitant des algorithmes d'intelligence artificielle basés sur un framework d'automatisation de tests.
- L'exécution automatisée des scripts de test après chaque modification ou développement pour tester les fonctionnalités.
- La génération de rapports détaillés permettant de suivre les résultats des tests et d'assurer un contrôle qualité rigoureux.
- L'intégration d'un tableau de bord interactif pour visualiser en temps réel l'état d'avancement des tests.

1.4 Méthodologie de travail

La gestion de projet est l'art de diriger et de coordonner des ressources humaines et matérielles tout au long de la vie d'un projet en utilisant des techniques de gestion modernes pour atteindre des objectifs prédéfinis d'envergure, de coût, de temps, de qualité et de satisfaction des participants. Pour y réussir, le recours à une méthodologie de gestion de projet est une approche indispensable au développement de tout projet.

1.4.1 Choix de la méthodologie

Le développement logiciel ne suit pas une méthode unique. Chaque projet présente ses propres contraintes, ce qui rend nécessaire l'adoption d'une méthodologie adaptée. L'un des principaux défis réside dans l'évolution fréquente des besoins, souvent mal définis au départ et susceptibles de changer. Cela rend difficile une planification rigide et exige une approche souple et itérative.

Face à ces incertitudes et à l'importance de respecter les délais, nous avons opté pour la méthodologie Agile [15]. Cette approche, fondée sur la collaboration, l'adaptabilité et l'amélioration continue, permet de répondre efficacement aux besoins du client tout en intégrant de manière flexible les évolutions en cours de développement.

Adopter Agile repose sur quatre principes fondamentaux :

- **Collaboration** : la communication et la cohésion au sein de l'équipe sont privilégiées par rapport aux processus et aux outils.
- **Implication du client** : l'interaction avec le client prime sur la simple négociation contractuelle.
- **Priorité au logiciel fonctionnel** : une application opérationnelle est préférée à une documentation exhaustive.
- **Acceptation du changement** : la flexibilité et l'adaptation sont favorisées plutôt que le suivi rigide d'un plan initial.

Parmi ses nombreux avantages, Agile permet une gestion du temps efficace grâce à un travail itératif, favorise la cohésion d'équipe et facilite la prise en compte rapide des retours clients. Ces qualités en font une méthode particulièrement adaptée aux projets nécessitant souplesse et réactivité, comme celui que nous avons mené.

1.4.2 Méthodologie Scrum

La société *Freedom Of Dev Services* (FOD) a choisi d'adopter la méthodologie Scrum [16] pour deux raisons principales. D'une part, FOD met en place des équipes expérimentées travaillant en étroite collaboration avec le client, conformément à la valeur agile qui privilégie la coopération à la négociation contractuelle. D'autre part, le contexte de notre projet ne permet pas de figer les spécifications dès le départ, il est donc essentiel d'accepter les changements en cours de développement et de les intégrer rapidement, ce qui reflète un principe fondamental de l'agilité.

Scrum repose sur une logique d'inspection et d'adaptation continues, où les sollicitations externes ne sont plus perçues comme des perturbations, mais comme des opportunités d'amélioration.

Trois rôles essentiels structurent Scrum :

- **Product Owner** : Représente les besoins du client, priorise les fonctionnalités à développer et gère le *Product Backlog*. Ce rôle est assuré par **Mme Randa Amich**.
- **Scrum Master** : Facilite l'application de la méthode Scrum, supprime les obstacles et soutient l'équipe. Ce rôle est tenu par **Mme Rim Mezghanni**.
- **Équipe de développement** : Chargée de concevoir, coder et tester les fonctionnalités. J'ai intégré cette équipe en tant que stagiaire.

Le pilotage s'appuie sur deux outils clés :

- **Product Backlog** : Liste évolutive et priorisée des fonctionnalités à développer.
- **Sprint Backlog** : Sous-ensemble du Product Backlog sélectionné pour un sprint.

Chaque itération suit un déroulement structuré :

- **Planification du sprint** : Définition des objectifs avec le *Product Owner*.

- **Réunions quotidiennes (Daily Scrum)** : Brève réunion journalière d'environ 15 minutes durant laquelle chaque membre de l'équipe partage :
 - Ce qu'il a accompli la veille ;
 - Ce qu'il prévoit de faire aujourd'hui ;
 - Les éventuelles difficultés rencontrées.
- **Revue de sprint** : Présentation de l'incrément produit au client.
- **Rétrospective** : Analyse des points à améliorer pour les prochains sprints.

La durée d'un sprint varie généralement entre deux et quatre semaines. Dans notre cas, nous avons opté pour une durée fixe de **deux semaines**. Avant le lancement de chaque sprint, une réunion de cadrage est tenue entre l'équipe projet et le client afin de définir les priorités et les spécifications à implémenter. Le processus Scrum appliqué dans notre projet est illustré dans la figure 1.6.

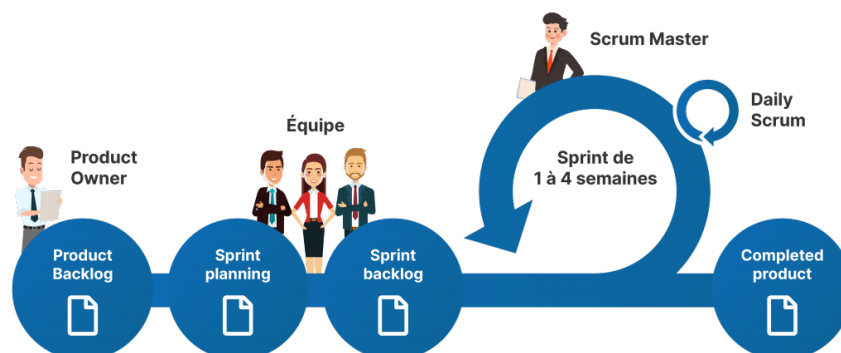


FIGURE 1.6 – Enchaînement du processus Scrum

1.5 Langage de modélisation

Avant de réaliser un système, il est essentiel de bien le comprendre à travers une phase de modélisation. Pour cela, nous avons utilisé le langage **UML (Unified Modeling Language)** [17], un langage de modélisation visuel standardisé, riche sur les plans sémantique et syntaxique.

UML est particulièrement adapté à la conception et à la mise en œuvre de systèmes logiciels complexes, notamment dans le cadre du développement orienté objet. Il permet de représenter aussi bien la structure que le comportement du système à travers différents types de diagrammes.

1.6 Planification du projet par sprints

Cette phase a été construite autour d'échanges réguliers avec le Product Owner (PO) afin de discuter des besoins et des fonctionnalités de la solution, garantissant ainsi une bonne compréhension des attentes pour le développement du projet.

Après avoir choisi la méthodologie Scrum, nous avons entamé la phase de découpage du projet en itérations appelées sprints. Le sprint 0 a été consacré à l'analyse des besoins, à la conception de l'architecture, ainsi qu'à la préparation du document d'analyse et du cahier des charges du projet, avec une preuve de la faisabilité de la solution et du backlog produit.

Par la suite, nous avons entamé la phase de développement, à travers plusieurs sprints successifs, permettant d'implémenter progressivement les UserStories définies dans le backlog. Chaque sprint s'est conclu par la livraison d'un incrément du produit, qui a été évalué puis validé.

TABLE 1.3 – Les sprints du projet

Sprint	Durée	Incrément
0	Du 10 février au 28 février	Accepté
1	Du 03 mars au 14 mars	Accepté
2	Du 17 mars au 28 mars	Accepté
3	Du 31 mars au 11 avril	Accepté
4	Du 14 avril au 25 avril	Accepté
5	Du 28 avril au 09 mai	Accepté
6	Du 12 mai au 23 mai	Accepté

1.7 CONCLUSION

Dans ce chapitre, nous avons présenté l'organisme d'accueil et introduit le cadre général de notre projet. Ensuite, nous avons étudié les solutions existantes afin de situer notre solution et de nous positionner par rapport à l'existant. Enfin, nous avons clôturé le chapitre par une présentation de la méthodologie de travail utilisée et une planification de notre projet.

Spécification et Analyse des exigences

Sommaire

2.1	INTRODUCTION	21
2.2	Spécification des exigences	21
2.2.1	Identification des acteurs	21
2.2.2	Exigences fonctionnelles	22
2.2.3	Exigences non fonctionnelles	24
2.3	Exigences techniques	25
2.3.1	Environnement matériel	25
2.3.2	Environnement logiciel	25
2.4	Pilotage du projet avec Scrum	28
2.4.1	Backlog du produit	29
2.4.2	Utilisation d'Azure DevOps pour le suivi du projet	33
2.4.3	Préparation de l'application à Tester <i>QA Task Manager</i>	34
2.5	Diagramme des cas d'utilisation	34
2.5.1	Description textuelle de « S'authentifier »	36
2.5.2	Description textuelle de « Configurer les projets »	37
2.5.3	Description textuelle de « Générer les cas de test »	39
2.5.4	Description textuelle de « Configurer et générer le rapport de test »	40
2.5.5	Description textuelle de « Consulter le tableau de bord »	40
2.6	Diagramme d'activité	41
2.7	CONCLUSION	43

2.1 INTRODUCTION

La phase d'analyse vise à définir, structurer et justifier la solution qui répondra efficacement aux besoins exprimés. Elle constitue une étape clé pour garantir la qualité et la réussite du projet.

Dans ce chapitre, nous présentons les principales exigences fonctionnelles et non fonctionnelles de notre application, ainsi que les technologies retenues. Ces exigences sont ensuite modélisées à l'aide de diagrammes UML afin de mieux comprendre la structure et le comportement attendu du système.

2.2 Spécification des exigences

Dans cette section, nous détaillons les exigences fonctionnelles et non fonctionnelles auxquelles notre application doit répondre. Ces exigences seront illustrées à l'aide d'un diagramme de cas d'utilisation. Nous procéderons également à l'identification des différents acteurs intervenant dans le système, afin de mener une analyse approfondie.

2.2.1 Identification des acteurs

Un acteur est une entité externe interagissant avec le système, qu'il s'agisse d'une personne (utilisateur, administrateur, etc) ou d'un autre système logiciel. Cette identification est essentielle pour comprendre les interactions possibles et définir les cas d'utilisation associés.

- **Super Admin** : Responsable de la gestion et de la configuration des projets, il attribue les rôles et les droits d'accès aux utilisateurs de l'application.
- **Ingénieur QA ou testeur** : Chargé de lancer un test sur un projet en validant les fonctionnalités développées, il génère les cas de test, lance des enregistrements manuels pour les cas de test via un outil de *recording*, initie la génération intelligente des scripts de test, exécute les scripts générés de test de façon automatisée et génère des rapports de test pour garantir la conformité aux exigences.

- **Développeur (Dev)** : Il consulte les rapports de test générés par les testeurs afin d'analyser les résultats fournis pour assurer la stabilité de l'application. Un tableau de bord est accessible à tous les acteurs afin d'offrir une vue sur le statut des tests des projets.
- **Azure DevOps** : Plateforme externe avec laquelle l'application interagit pour collecter les UserStories et les membres associés à un projet.

2.2.2 Exigences fonctionnelles

Les exigences fonctionnelles décrivent les fonctionnalités essentielles que l'application doit offrir. Elles sont organisées en sept axes principaux :

- **Authentification, configuration des projets et gestion des rôles**
 - *Authentification* : Le super admin, les QA et les développeurs accèdent à l'application après authentification via Microsoft Entra ID. Un système d'authentification robuste contrôle l'accès des utilisateurs. Les droits d'accès sont contrôlés finement selon les rôles attribués à chaque utilisateur.
 - *Gestion centralisée des projets* :
 - Le Super admin peut visualiser les projets et leurs détails (dernière modification, etc).
 - Le super admin peut configurer les projets en précisant leurs boards associés. Il peut lancer la récupération des membres affectés à ces projets suite à une interaction avec Azure DevOps.
 - Il peut affecter dynamiquement les rôles et permissions (Dev ou QA) aux membres du projet à configurer.
 - Le super admin peut modifier les configurations d'un projet, les archiver (rendre leur statut "Archivé") ou les restaurer (rendre leur statut "Actif").
- **Récupération des UserStories**
 - Les UserStories de ce projet sont automatiquement récupérées après la configuration du projet depuis Azure DevOps (board configuré) avec statuts et détails. Elles suivent une structure : titre, description et critères d'acceptation.

- Le QA peut relancer manuellement la récupération des UserStories d'un projet afin de rafraîchir et mettre à jour les données en cas de modifications, assurant ainsi un suivi en temps réel avec Azure DevOps.
 - Le QA peut consulter l'ensemble des UserStories du projet et accéder aux détails de chacune d'elles.
 - Le QA peut configurer l'URL de l'application à tester, associée à un projet. Un indicateur permet de savoir si cette URL a déjà été configurée par le Super Admin.
- **Génération intelligente des cas de test**
- Le QA peut lancer la génération des cas de test en sélectionnant une UserStory dédiée.
 - Le QA peut modifier le contenu des cas de test générés.
 - Le QA peut lancer une régénération des cas de test en cas de besoin.
 - Le QA peut supprimer des cas de test générés.
- **Génération de scripts et exécution automatisée des tests**
- *Recording* : Le QA peut lancer un enregistrement sur un cas de test à l'aide d'un outil de *recording* si l'URL de l'application est configurée. Le Recording va permettre de détecter les sélecteurs des éléments de l'application qui va servir ensuite comme base pour la génération des scripts par le modèle intelligent.
 - *Génération des scripts de test* : Le QA peut lancer la génération intelligente des scripts associés à chaque cas de test, afin de garantir un format homogène et de faciliter l'automatisation des tests. Cette génération s'effectue uniquement pour les UserStories ayant le statut **Closed** ou **Resolved**, afin de s'assurer que les cas de test sont stables et validés. La génération peut s'appuyer sur différentes sources : l'enregistrement (recording), des scripts issus de tests similaires précédents, ou, en dernier recours, uniquement sur la description du cas de test. Le script généré est ensuite compatible avec un outil permettant son exécution de manière automatisée.
 - *Exécution automatisée des tests* :
 - Le QA peut lancer l'exécution des scripts générés ou enregistrés, avec visualisation du résultat du test (succès ou échec).

– Génération de rapports de tests

- Le QA peut générer un rapport de test, configurable selon plusieurs critères. Il peut choisir entre un rapport détaillé (incluant les détails des cas de test) ou un rapport générique (mentionnant uniquement les cas de test sans leurs détails). Il peut également décider d'y inclure l'ensemble des tests ou uniquement les tests exécutés. Le rapport contient les UserStories, les cas de test et leurs résultats d'exécution.
- Le rapport généré par le QA est téléchargé.
- Le développeur peut consulter la liste des rapports générés pour chaque projet.

– Tableau de bord interactif

- Le QA, le Développeur et le Super admin peuvent visualiser en temps réel des métriques via un tableau de bord dynamique, facilitant le suivi des performances et la prise de décisions rapides.

2.2.3 Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les qualités attendues ainsi que les contraintes auxquelles l'application doit se conformer. Les points suivants détaillent ces exigences :

- **Performance** L'application doit garantir une navigation fluide et des temps de réponse courts, en particulier pour les actions critiques comme la synchronisation avec Azure DevOps, la génération intelligente de tests ou leur exécution. L'architecture web doit être scalable pour gérer une charge variable.
- **Maintenabilité** Le code côté serveur comme client doit être modulaire et facilement extensible. L'usage de composants React réutilisables, de bonnes pratiques de développement (tests, documentation, nommage) facilite la maintenance.
- **Ergonomie** Les interfaces utilisateur doivent être claires, intuitives et cohérentes pour permettre une prise en main rapide. La navigation se veut fluide, avec des retours visuels (notifications, validations) pour guider l'utilisateur et améliorer son expérience.
- **Sécurité** L'accès à l'application est sécurisé via Microsoft Entra ID. L'authentification OAuth assure la protection des données utilisateur et la confidentialité des projets. Les

rôles (Super Admin, QA, développeur) sont strictement contrôlés pour limiter l'accès aux actions autorisées.

2.3 Exigences techniques

Nous détaillons dans cette partie les logiciels et les langages utilisés pour le développement de notre application.

2.3.1 Environnement matériel

Dans cette section, nous présentons l'environnement matériel utilisé lors de la réalisation des différentes phases de notre projet. Comme montré dans le tableau 2.1, nous avons employé une machine dotée des caractéristiques suivantes :

TABLE 2.1 – Environnement matériel

Composant	Spécification
Processeur	Intel® Core™ i5-10300H
Mémoire vive	16 Go RAM
Stockage	512 Go SSD
Système d'exploitation	Windows 11 Professionnel 64 bits

2.3.2 Environnement logiciel

Nous abordons dans cette partie les technologies et outils utilisés pour la réalisation de ce projet.

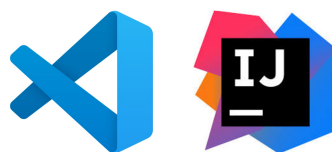


FIGURE 2.1 – Environnements de développement intégrés

Visual Studio Code est un éditeur de code source extensible, développé par Microsoft. Léger et hautement personnalisable, il facilite l'écriture, le débogage et la gestion de projets.

IntelliJ IDEA est un environnement de développement intégré (IDE) Java développé par JetBrains, misant sur la productivité avec des systèmes intelligents d'analyse de code en temps réel.



FIGURE 2.2 – Technologies Frontend

React est une bibliothèque JavaScript développée par Meta pour créer des interfaces utilisateur dynamiques et modulaires. Elle repose sur une architecture basée sur les composants et la gestion efficace du DOM virtuel.

HTML5 est le langage de balisage standard pour la création de pages web. Il définit la structure et le contenu sémantique des pages sur internet.

CSS3 est le langage de style qui contrôle la présentation visuelle des documents HTML. Il permet de gérer la mise en page, les couleurs, les polices et les animations.

JavaScript est un langage de programmation qui permet d'implémenter des fonctionnalités complexes sur les pages web, rendant les interfaces interactives et dynamiques.



FIGURE 2.3 – Frameworks Backend

Quarkus est un framework Java moderne utilisé pour nos microservices principaux. Conçu pour le cloud natif, il offre une empreinte mémoire réduite et un démarrage ultra-rapide.

FastAPI est notre framework Python pour les services d'intelligence artificielle. Moderne et rapide, il facilite la construction d'APIs RESTful performantes avec documentation automatique.



FIGURE 2.4 – Logo de PostgreSQL

PostgreSQL est le système de gestion de base de données relationnelle de notre application. Open-source et robuste, il offre les fonctionnalités avancées nécessaires pour notre projet.



FIGURE 2.5 – Outils de développement et d'intégration

Git est notre système de contrôle de version. Il permet de gérer efficacement l'historique des modifications et facilite la collaboration au sein de l'équipe.

Apache Maven gère les dépendances et automatise le build de nos projets Java, simplifiant l'intégration continue et le déploiement.



FIGURE 2.6 – Logo de REST

REST est l'architecture utilisée pour nos API. Ce style architectural basé sur HTTP permet une communication efficace entre nos différents services.



FIGURE 2.7 – Logo Postman

Postman est notre plateforme de test d'API. Elle permet de concevoir, tester et documenter nos API REST dans une interface ergonomique.



FIGURE 2.8 – Logo de Playwright

Playwright est une bibliothèque open-source développée par Microsoft permettant d'automatiser les tests end-to-end des applications web. Compatible avec plusieurs langages de programmation, elle offre une API unifiée pour interagir avec différents navigateurs tels que Chromium, Firefox et WebKit. Grâce à ses fonctionnalités avancées, Playwright permet de reproduire les actions des utilisateurs de façon fiable et réaliste.



FIGURE 2.9 – Logo de Microsoft Entra ID

Microsoft Entra ID (Azure AD) gère l'authentification et les accès de notre application. Cette solution cloud de Microsoft sécurise nos services via OAuth 2.0 et OpenID Connect.



FIGURE 2.10 – Logo de Ollama

Ollama nous permet d'exécuter localement des modèles de langage (LLMs) pour la génération de cas de test, offrant performance et confidentialité des données.

Flying Saucer est la bibliothèque Java utilisée pour générer nos rapports PDF à partir de documents HTML/CSS, automatisant ainsi la création de rapports professionnels.

2.4 Pilotage du projet avec Scrum

Cette section met en lumière la gestion du projet avec Scrum, en détaillant la structuration du backlog et l'utilisation d'Azure DevOps pour le suivi des sprints et des tâches.

2.4.1 Backlog du produit

Après plusieurs échanges avec notre client, nous avons clarifié et validé ses besoins. Nous avons ainsi défini les "Features" (fonctionnalités), qui regroupent différentes UserStories (histoires utilisateurs), pour constituer le backlog complet du projet. Une "UserStory" décrit simplement un besoin ou une attente d'un utilisateur. Une "Feature", quant à elle, rassemble plusieurs UserStories liées à une fonctionnalité clé, apportant une réelle valeur au produit. Grâce au backlog, nous pouvons :

- Utiliser Git pour gérer les versions de manière collaborative et efficace.
- Suivre facilement l'avancement du travail grâce à une organisation claire, permettant à notre équipe de gérer le projet avec fluidité.

Le tableau 2.2 ci-dessous présente toutes les UserStories de notre application.

TABLE 2.2 – Liste des features et UserStories

ID	Feature	ID	User Story
A.1	Authentification	A.1.1	En tant qu'utilisateur, je veux m'authentifier à l'application.
		A.1.2	En tant qu'utilisateur, je veux obtenir les permissions adéquates selon mon rôle.
B.1	Récupération des membres et gestion des rôles	B.1.1	En tant que Super Admin, je veux affecter des rôles aux membres.
		B.1.2	En tant que Super Admin, je veux récupérer et consulter les membres associés à un projet.
C.1	Configuration des projets	C.1.1	En tant que Super Admin, je veux créer un projet.
		C.1.2	En tant que Super Admin, je veux modifier un projet existant.
		C.1.3	En tant que Super Admin, je veux un tableau de bord récapitulatif des projets, et pouvoir filtrer les projets.

Suite à la page suivante

Table 2.2 – suite de la page précédente

ID	Feature	ID	User Story
		C.1.4	En tant que Super Admin, je veux pouvoir archiver et restaurer un projet.
		C.1.5	En tant que QA/Super Admin, je veux configurer l'url de l'application dans un projet.
D.1	Consultation et choix des projets	D.1.1	En tant que QA/Dev, je veux consulter la liste des projets auxquels je suis assigné, et pouvoir les filtrer ou les rechercher par nom.
		D.1.2	En tant que QA, je veux pouvoir lancer une exécution de test d'un projet et visualiser son dernier état afin de suivre son avancement.
		D.1.3	En tant que Dev, je veux consulter les rapports de test associés à un projet.
E.1	Génération Intelligente des Cas de Test	E.1.1	[SPIKE] choisir modèle IA pour la génération des cas de test.
		E.1.2	En tant que QA, je veux générer des cas de test à partir des UserStories.
		E.1.3	En tant que QA, je veux voir les progressions des exécutions en cours.
F.1	Gestion des Cas de Test Générés	F.1.1	En tant que QA, je veux consulter les détails d'un cas de test généré.
		F.1.2	En tant que QA, je veux que les cas de test générés à partir d'une UserStory s'affichent dans l'interface.
		F.1.3	En tant que QA, je veux régénérer les cas de test.

Suite à la page suivante

Table 2.2 – suite de la page précédente

ID	Feature	ID	User Story
		F.1.4	En tant que QA, je veux pouvoir modifier ou supprimer un cas de test généré.
G.1	Génération intelligente de script des tests	G.1.1	[SPIKE] choisir modèle IA pour la génération des scripts de test.
		G.1.2	[SPIKE] Étudier la performance du modèle intelligent.
		G.1.3	En tant que QA, je veux faire le recording d'un cas de test.
		G.1.4	En tant que QA, je veux que les scripts soient générés intelligemment à partir d'un recording du cas de test.
		G.1.5	En tant que QA, je veux que les scripts soient générés intelligemment sur base des anciens scripts des cas de test similaires.
H.1	Gestion de Scripts de Test Générés	H.1.1	En tant que QA, je veux visualiser les scripts de tests générés intelligemment.
		H.1.2	En tant que QA, je veux savoir si un cas de test possède un script généré ou recorded.
		H.1.3	En tant que QA, je veux éditer/valider les scripts de tests générés intelligemment.
I.1	Exécution de Test Automatisé	I.1.1	En tant que QA, je veux exécuter le script généré et validé d'un cas de test.
		I.1.2	En tant que QA, je veux pouvoir sélectionner les cas de test à exécuter automatiquement.

Suite à la page suivante

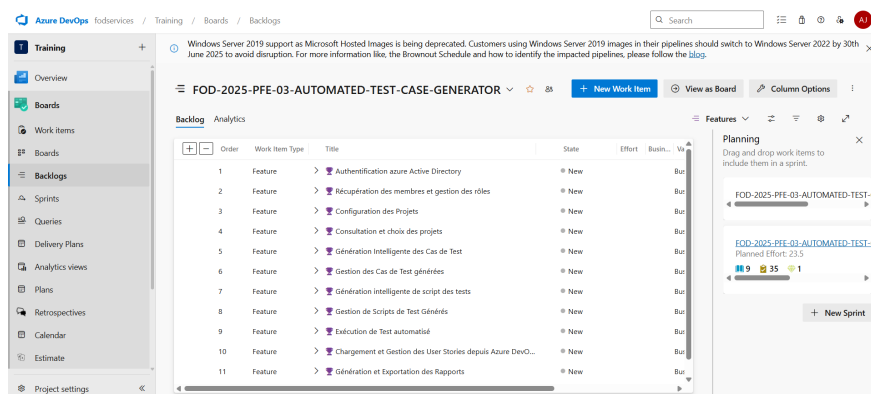
Table 2.2 – suite de la page précédente

ID	Feature	ID	User Story
		I.1.3	En tant que QA, je veux recevoir dans l'application une notification des 10 dernières exécutions terminées au cours des 24 dernières heures.
J.1	Chargement et Gestion des UserStories depuis Azure DevOps	J.1.1	En tant que QA, je veux pouvoir accéder en temps réel à toutes les informations détaillées d'une UserStory depuis Azure DevOps.
		J.1.2	En tant que QA, je veux recharger manuellement la liste des UserStories.
		J.1.3	En tant que QA, je veux visualiser automatiquement les UserStories de chaque projet depuis Azure DevOps.
		J.1.4	En tant que QA, je veux que les UserStories d'un projet soient récupérées de Azure DevOps au moment de la configuration du projet.
K.1	Génération et Exportation des Rapports	K.1.1	[SPIKE] analyser les bibliothèques de génération et d'exportation des rapports (PDF).
		K.1.2	En tant que QA, je veux configurer, générer et exporter un rapport de test en collectant les données depuis la base de données.
L.1	Tableau de Bord et Suivi	L.1.1	En tant qu'utilisateur, je veux avoir un tableau de bord interactif.

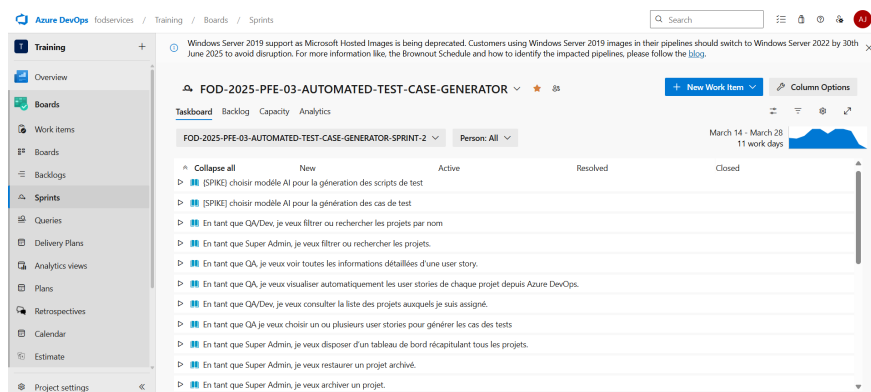
2.4.2 Utilisation d’Azure DevOps pour le suivi du projet

Pour piloter efficacement notre projet selon la méthodologie Scrum, nous avons utilisé l’outil Azure DevOps. Cet outil nous a permis de centraliser la gestion du backlog, l’organisation des sprints, et le suivi des tâches en temps réel.

- Les UserStories, définies dans le tableau précédent, ont été ajoutées dans le tableau de bord Azure DevOps, avec un lien explicite à chaque Feature.
- Les tâches ont été associées à chaque UserStory pour faciliter l’estimation, la planification et la répartition du travail entre les membres de l’équipe.
- Le backlog a été organisé en sprints, planifiés selon notre capacité et les priorités définies avec le client.
- Des revues de sprint et rétrospectives ont été menées régulièrement à la fin de chaque sprint pour évaluer les progrès réalisés et ajuster notre approche.



(a) Backlog dans Azure DevOps



(b) Exemple d'un sprint configuré dans Azure DevOps

FIGURE 2.11 – Illustrations de l’utilisation d’Azure DevOps pour le suivi du projet.

2.4.3 Préparation de l'application à Tester *QA Task Manager*

En parallèle, un second board Azure DevOps a été créé pour l'application simple **QA Task Manager**, que nous avons développée spécifiquement afin de servir de système à tester pour Smart QA. Nous y avons défini un backlog dédié, structuré en fonctionnalités et UserStories, dans le but de simuler un projet réel et de valider les fonctionnalités de génération de cas de test, de génération de scripts, ainsi que de suivi de qualité proposées par notre outil principal.

Grâce à Azure DevOps, nous avons pu assurer une traçabilité complète du projet, favoriser la collaboration entre les membres de l'équipe, et garantir une visibilité continue pour le client.

2.5 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation fournit une vue d'ensemble du fonctionnement du système en représentant les besoins des utilisateurs sous forme de cas d'utilisation, ainsi que les relations entre eux.

En raison de sa complexité, le diagramme global du système Smart QA, illustré à la figure 2.12, a été décomposé en plusieurs sous-diagrammes pour en faciliter la lecture et la compréhension. Ce schéma général présente les principales fonctionnalités et les interactions entre les différents acteurs.

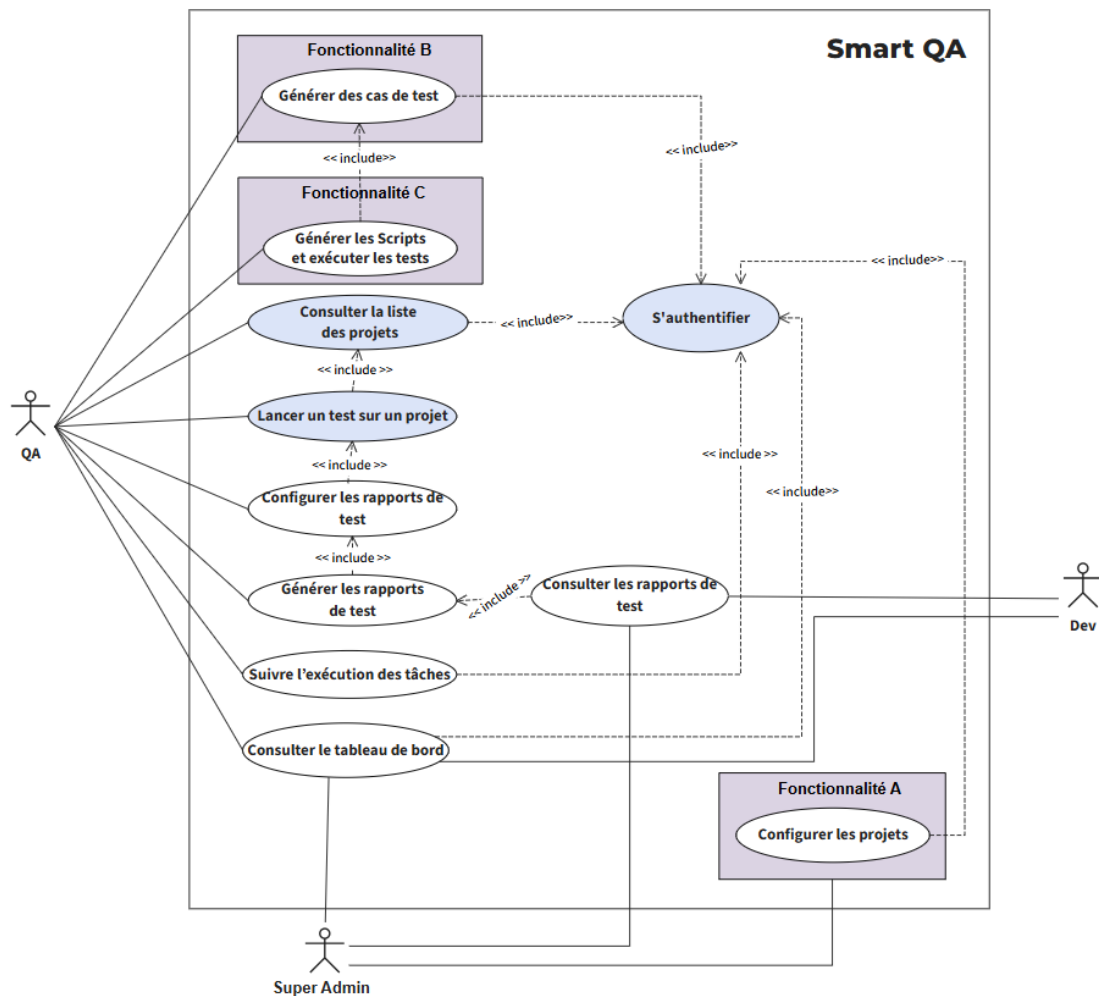


FIGURE 2.12 – Diagramme des cas d'utilisation

L'accès à l'application commence par une authentification, préalable à tous les cas d'utilisation. Trois types d'utilisateurs sont définis : le Super Admin, le QA et le Développeur (Dev), chacun disposant de droits spécifiques.

Le Super Admin peut notamment configurer les projets (fonctionnalité A), qui englobe plusieurs sous-fonctionnalités détaillées ultérieurement. Le QA est chargé de générer des cas de test (fonctionnalité B), de produire et d'exécuter des scripts (fonctionnalité C), de configurer et générer les rapports et de suivre l'exécution des tests. Les fonctionnalités B et C seront approfondies dans les sections suivantes. Le Développeur a accès aux rapports de test générés. Enfin, tous les utilisateurs peuvent consulter le tableau de bord.

La description détaillée des cas d'utilisation associés à chacune de ces fonctionnalités est présentée en Annexe A.1.

Dans ce qui suit, nous présentons les descriptions textuelles de de quelques cas d'utilisation.

2.5.1 Description textuelle de « S'authentifier »

Pour garantir la sécurité et un accès contrôlé au système Smart QA, il est indispensable de s'authentifier. Le tableau 2.3 illustre la description textuelle de ce cas d'utilisation.

TABLE 2.3 – Description textuelle de « S'authentifier »

Acteurs	Super Admin, QA, Dev
Description	Le membre s'authentifie à l'application via Microsoft Entra ID.
Pré-condition	Le membre doit exister comme utilisateur dans le tenant (organisation) de Microsoft Entra ID. Cela signifie qu'il doit être créé ou invité dans ce tenant.
Post-condition	Le membre est authentifié. Il sera redirigé vers l'application selon son rôle (Super Admin, QA ou Dev).
Scénario nominal	1. Le système détecte que l'utilisateur doit être authentifié via Microsoft Entra ID. 2. Le système redirige l'utilisateur vers le point de terminaison d'autorisation (Authorization Endpoint) d'Entra ID. 3. Microsoft Entra ID affiche l'interface de connexion (page de login). 4. L'utilisateur saisit ou sélectionne son adresse e-mail (ou est automatiquement connecté selon la configuration SSO). 5. L'utilisateur saisit ses identifiants ou utilise un facteur d'authentification si MFA est activé (Il doit saisir un code SMS ou approuver dans une app mobile après le mot de passe). 6. Microsoft Entra ID vérifie les informations d'identification. 7. En cas de succès, Microsoft Entra ID génère un jeton d'authentification (ID token / access token). 8. Le système reçoit et valide le jeton d'authentification via le redirect URI. 9. Le système établit la session utilisateur et redirige l'utilisateur vers la page d'accueil de l'application.

Scénarios alternatifs	A1 : SSO actif avec connexion automatique 1. L'utilisateur est reconnu automatiquement par Microsoft Entra ID sans saisir de mot de passe (connexion SSO active). 2. Il est redirigé immédiatement vers l'application.
Scénarios d'erreurs	E1 : Membre non reconnu 1. Le membre ne pourra pas s'authentifier car il ne sera pas reconnu comme membre de l'annuaire. 2. Le système affiche un message d'erreur : « Utilisateur non reconnu ». 3. Le scénario se termine avec échec. E2 : Problème de configuration SSO 1. La configuration SSO (redirect URI, Client ID) est invalide. 2. Le système ne peut pas établir la connexion avec Microsoft Entra ID. 3. Le message d'erreur « Problème de configuration SSO » est affiché. 4. Le scénario se termine avec échec.

2.5.2 Description textuelle de « Configurer les projets »

Le Super Admin peut configurer les projets dans Smart QA. Le tableau 2.4 illustre la description textuelle de ce cas d'utilisation.

TABLE 2.4 – Description textuelle de « Configurer les projets »

Acteurs	Super Admin
Description	Le Super Admin configure les projets existants sur Azure DevOps à partir des liens des boards configurés, afin de les intégrer dans Smart QA.
Pré-condition	Le Super Admin doit être authentifié et disposer des permissions nécessaires pour accéder aux données sur Azure DevOps.
Post-condition	Le projet est correctement configuré et accessible aux utilisateurs affectés.

Scénario nominal	1. Le Super Admin accède à l'interface de gestion des projets. 2. Il ajoute un nouveau projet. 3. Il renseigne les paramètres du projet, notamment les liens vers les boards Azure DevOps. 4. Il récupère les membres associés au projet depuis Azure DevOps. 5. Le Super Admin peut affecter les rôles aux membres récupérés. 6. Le Super Admin enregistre la configuration. 7. Le système vérifie si le projet existe déjà dans Smart QA. 8. Si le projet n'existe pas, le système l'enregistre ainsi que les membres associés. 9. Les UserStories du projet sont automatiquement récupérées depuis Azure DevOps et enregistrées en base de données.
Scénarios alternatifs	A1 : Projet déjà existant dans Smart QA 1. Lors de la tentative d'ajout, le système détecte que le projet existe déjà. 2. Un message d'erreur « Projet déjà configuré » est affiché. 3. Le scénario nominal reprend au point 2.
Scénarios d'erreurs	E1 : Échec de connexion à Azure DevOps 1. Une erreur réseau ou d'authentification empêche la connexion. 2. Le système affiche un message : « Impossible de se connecter à Azure DevOps ». 3. Le scénario se termine avec échec. E2 : Échec de récupération des membres 1. Le système ne parvient pas à récupérer la liste des membres du projet depuis Azure DevOps. 2. Un message d'erreur « Erreur lors de la récupération des membres » est affiché. 3. Le scénario se termine avec échec.

2.5.3 Description textuelle de « Générer les cas de test »

Le QA peut générer des cas de test à partir des UserStories d'Azure DevOps. Le tableau 2.5 illustre la description textuelle de ce cas d'utilisation.

TABLE 2.5 – Description textuelle de « Générer les cas de test »

Acteurs	QA
Description	Le QA génère des cas de test à partir des UserStories récupérées dans un projet Azure DevOps. Les cas de test sont générés automatiquement en analysant le contenu de chaque UserStory, notamment les critères d'acceptation.
Pré-condition	Le QA est authentifié et a sélectionné un projet contenant des UserStories.
Post-condition	Les cas de test correspondant à la UserStory sélectionnée sont générés et affichés.
Scénario nominal	1. Le QA sélectionne un projet dans la liste. 2. Le système affiche les UserStories du projet. 3. Le QA sélectionne une UserStory. 4. Le QA clique sur « Generate Test Case ». 5. Le système génère automatiquement les cas de test. 6. Les cas de test générés sont affichés à l'interface.
Scénarios alternatifs	A1 : La UserStory ne contient pas de critères d'acceptation 1. Le système affiche un message : « Acceptance criteria not found. » 2. Le scénario nominal reprend au point 3.
Scénarios d'erreurs	E1 : Échec lors de la génération 1. Une erreur technique survient lors du traitement (ex. : modèle indisponible). 2. Le système affiche le message : « Error while generating test cases ».

2.5.4 Description textuelle de « Configurer et générer le rapport de test »

Le QA peut générer un rapport de test configurable selon ses préférences. Ce rapport est ensuite téléchargé automatiquement et sauvegardé dans le système. Le tableau ?? présente la description détaillée de ce cas d'utilisation.

TABLE 2.6 – Description textuelle de « Configurer et générer le rapport de test »

Acteurs	QA (acteur principal), Super Admin, Dev (consultation uniquement)
Description	Le QA configure et génère un rapport de test personnalisé depuis l'interface de Smart QA. Le rapport est téléchargé automatiquement et reste accessible aux autres membres autorisés.
Pré-condition	Le QA est authentifié et a accès à un projet.
Post-condition	Un rapport est généré, sauvegardé et téléchargé localement. Il est aussi consultable depuis le système par les autres rôles.
Scénario nominal	1. Le QA accède à un projet. 2. Il clique sur le bouton « Exporter le rapport ». 3. Une fenêtre modale s'affiche, proposant des options de configuration du rapport (sections à inclure, etc.). 4. Le QA sélectionne les options souhaitées et confirme la génération. 5. Le système compile les données selon la configuration. 6. Le rapport est généré et automatiquement téléchargé sur le poste du QA. 7. Le rapport est également enregistré dans la base. 8. Le Super Admin et les développeurs peuvent y accéder en lecture.

2.5.5 Description textuelle de « Consulter le tableau de bord »

Le membre (QA, Développeur ou Super Admin) accède au tableau de bord dans Smart QA, qui fournit une vue synthétique des projets à travers des indicateurs clés (KPIs). Le tableau 2.7 détaille ce cas d'utilisation.

TABLE 2.7 – Description textuelle de « Consulter le tableau de bord »

Acteurs	QA, Développeur, Super Admin
Description	Le tableau de bord fournit une vue synthétique de l'état des projets via des KPIs interactifs (taux de réussite des tests, statistiques globales, etc.). L'utilisateur peut appliquer un filtre par projet pour affiner les données affichées, mises à jour dynamiquement par le système.
Pré-condition	L'utilisateur est authentifié.
Post-condition	Les indicateurs du tableau de bord sont affichés et filtrés selon le projet sélectionné.
Scénario nominal	1. Le membre clique sur « Dashboard » dans le menu principal. 2. Le système récupère les données pertinentes depuis la base. 3. Les indicateurs sont présentés sous forme de graphiques et tableaux. 4. L'utilisateur sélectionne un projet. 5. Le tableau de bord se met à jour avec les données correspondantes.

2.6 Diagramme d'activité

Le diagramme d'activités est un diagramme UML qui fournit une représentation graphique du comportement d'un système. Il permet de le décomposer de manière fonctionnelle afin d'analyser sa mise en œuvre. Il est également dédié à la description d'un processus métier ou d'un cas d'utilisation. La figure 2.13 présente le diagramme d'activités décrivant le processus métier de la mise à jour des UserStories. Cette fonctionnalité récupère les dernières informations depuis Azure DevOps, permettant aux équipes QA de détecter rapidement toute modification dans les UserStories, afin d'adapter ensuite les cas de test en fonction de ces changements.

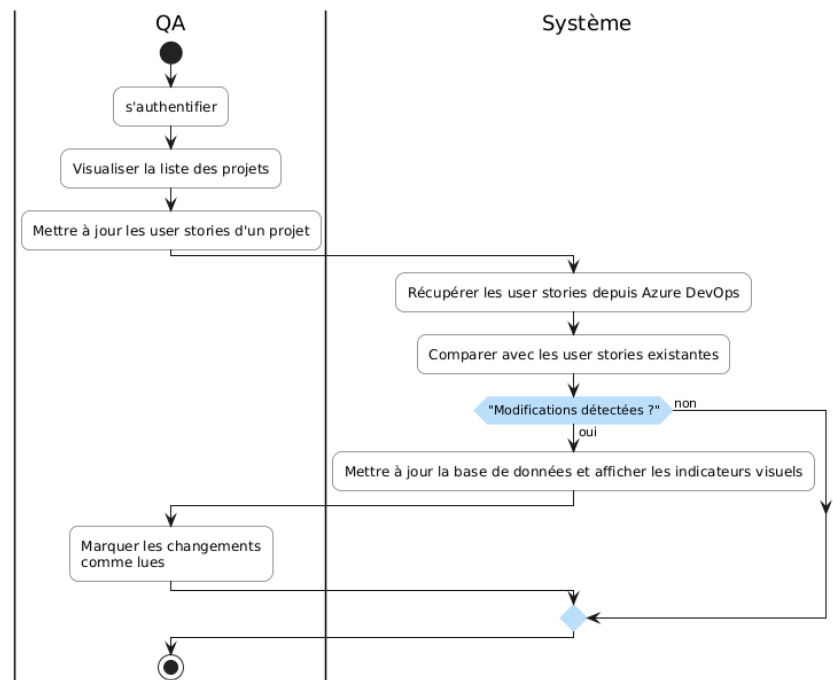


FIGURE 2.13 – Diagramme d'activité de « mettre à jour les UserStories »

Le processus de lancement d'un "recording" est représenté par le diagramme d'activité 2.14. Cette fonctionnalité permet d'enregistrer automatiquement les interactions réalisées sur l'interface utilisateur afin de générer les scripts de test correspondants.

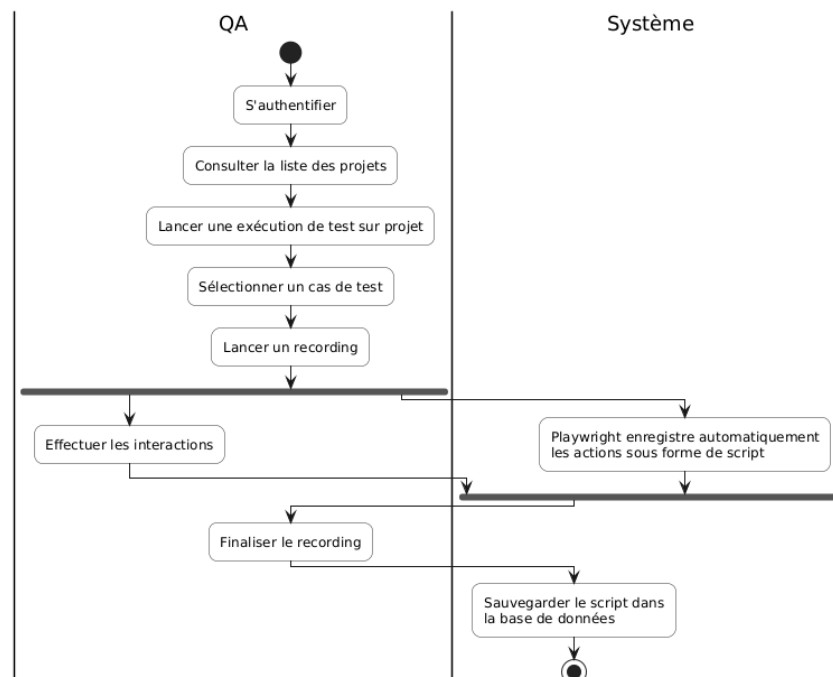


FIGURE 2.14 – Diagramme d'activité de « lancer un recording »

2.7 CONCLUSION

Ce chapitre a permis de décrire les fonctionnalités de notre application à travers la présentation du diagramme des cas d'utilisation, accompagnée des descriptions textuelles des différents cas d'utilisation, ainsi que des diagrammes d'activités. Ces éléments ont jeté les bases nécessaires à une compréhension claire des besoins fonctionnels. Dans le chapitre suivant, nous aborderons la phase de conception, qui vise à garantir une mise en œuvre efficace d'un système fonctionnel répondant pleinement aux exigences identifiées.

Étude Conceptuelle

Sommaire

3.1	INTRODUCTION	45
3.2	Conception générale	45
3.2.1	Architecture globale et choix technologiques	45
3.2.2	Architecture logique	47
3.3	Conception détaillée	49
3.3.1	Diagramme de classes	49
3.3.2	Diagrammes de séquence	51
3.4	Étude théorique des modules intelligents de génération des cas de test et des scripts de test	53
3.4.1	Approches théoriques communes pour les deux modules	53
3.4.2	Comparaison des approches	55
3.4.3	Prompt Engineering dans le contexte des LLM	56
3.4.4	Choix du LLM : Évaluation des modèles et résultats	57
3.4.5	Modèle adopté : LLM DeepSeek-R1-Distill-Llama-8B	59
3.5	Mise en Œuvre des modules intelligents de génération des cas de test et des scripts de test	60
3.5.1	Outil utilisé	60
3.5.2	Mise en Œuvre du module de génération des cas de test	61
3.5.3	Mise en œuvre du module de génération des scripts de test	64
3.6	CONCLUSION	71

3.1 INTRODUCTION

Ce chapitre expose la phase de conception, étape cruciale dans le développement de notre application, permettant de structurer un système complexe en sous-systèmes cohérents. À travers une série de diagrammes UML, nous présentons l'architecture générale adoptée, justifions ce choix, détaillons les interactions entre les composants via des diagrammes de séquence, et nous consacrons une attention particulière au module de génération intelligente des cas de test et des scripts.

3.2 Conception générale

Dans cette section, nous décrivons la conception générale de notre système, en présentant l'architecture globale de l'application ainsi que les choix technologiques adoptés.

3.2.1 Architecture globale et choix technologiques

L'architecture d'un système informatique définit sa structure fondamentale, en mettant en évidence l'organisation des différents composants ainsi que les relations et interactions entre eux. La figure 3.1 illustre l'architecture globale de notre application avec ses différents composants et leurs relations.

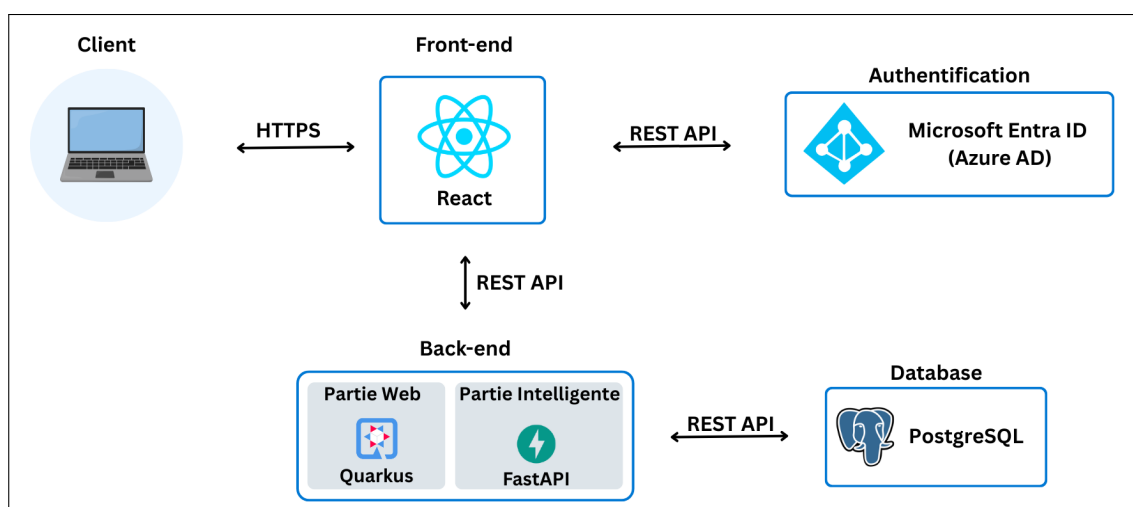


FIGURE 3.1 – Architecture globale de « Smart QA »

Composants de l'architecture et technologies utilisées :

- **Base de données** : Les données de l'application sont stockées dans une base de données relationnelle, **PostgreSQL**, choisie pour sa robustesse et sa capacité à gérer des relations complexes entre les entités. Le back-end interagit avec PostgreSQL via des requêtes SQL pour sauvegarder et récupérer les informations nécessaires.
- **Service d'authentification** : L'authentification des utilisateurs est assurée par Microsoft Entra ID, un service sécurisé de gestion des identités et des accès. Le front-end communique avec Entra ID (anciennement Azure Active Directory) pour authentifier les utilisateurs (Super Admin, QA, Dev), garantissant que seules les personnes autorisées accèdent aux fonctionnalités du système selon leur profil.
- **Front-end** : La couche de présentation est développée avec **React**, une bibliothèque JavaScript moderne, en combinaison avec l'outil de bundling **Vite**, afin d'optimiser la vitesse de développement et les performances de l'application. Cette architecture permet de concevoir une interface dynamique, réactive et modulaire. Le front-end communique avec le back-end via des **API REST** pour échanger des données. Il interagit également avec **Microsoft Entra ID** pour sécuriser l'accès des utilisateurs et gérer leurs sessions.
- **Back-end** : Le back-end constitue le cœur de l'application, en assurant la gestion de la logique métier, la communication avec la base de données, ainsi que l'exposition des **API REST** utilisées par le front-end. Il est structuré en deux parties complémentaires :
 - **La partie web**, développée avec **Quarkus** (Java), assure la gestion des utilisateurs, des projets et des données. Ce framework a été choisi pour ses performances, sa légèreté et son intégration avec les environnements cloud.
 - **La partie intelligente**, implémentée avec **FastAPI** (Python), est dédiée à la génération automatique des cas de test et des scripts. Ce framework a été choisi pour sa simplicité d'utilisation, sa rapidité de développement, la clarté de son code et la documentation automatique offerte via **OpenAPI**.

3.2.2 Architecture logique

Afin de garantir la flexibilité, la maintenabilité et la scalabilité de notre application, nous avons adopté une **architecture microservices** [18]. Il s'agit d'un modèle architectural dans lequel le système est divisé en services indépendants, chacun étant responsable d'une fonctionnalité précise. Cette approche favorise le découplage des composants, permettant leur développement, déploiement et mise à jour de manière autonome, tout en assurant une meilleure résilience globale. Ce choix repose sur plusieurs avantages essentiels :

- **Modularité et scalabilité** : Le système est divisé en services indépendants, permettant de gérer et de faire évoluer chaque composant séparément selon les besoins.
- **Responsabilité unique** : Chaque service est conçu pour remplir une tâche spécifique (par exemple, gestion des projets, génération des cas de test, exécution des tests), ce qui simplifie la maintenance et améliore la clarté de l'architecture.
- **Flexibilité** : L'architecture permet d'intégrer de nouveaux composants ou technologies en fonction des besoins futurs des clients, sans impacter l'ensemble du système.
- **Configuration indépendante** : Chaque service peut être configuré et déployé séparément, facilitant les mises à jour et la maintenance.
- **Découplage de la partie Web et du module IA** : Le module de génération intelligente des cas de test et des scripts, basé sur l'intelligence artificielle, est indépendant. Ainsi, en cas de panne, de maintenance ou d'optimisation du module IA, l'application principale reste opérationnelle, permettant aux utilisateurs de consulter l'historique des tests et de gérer les projets sans interruption.

La figure 3.2 illustre cette architecture, en mettant en évidence les principaux microservices et leurs interactions.

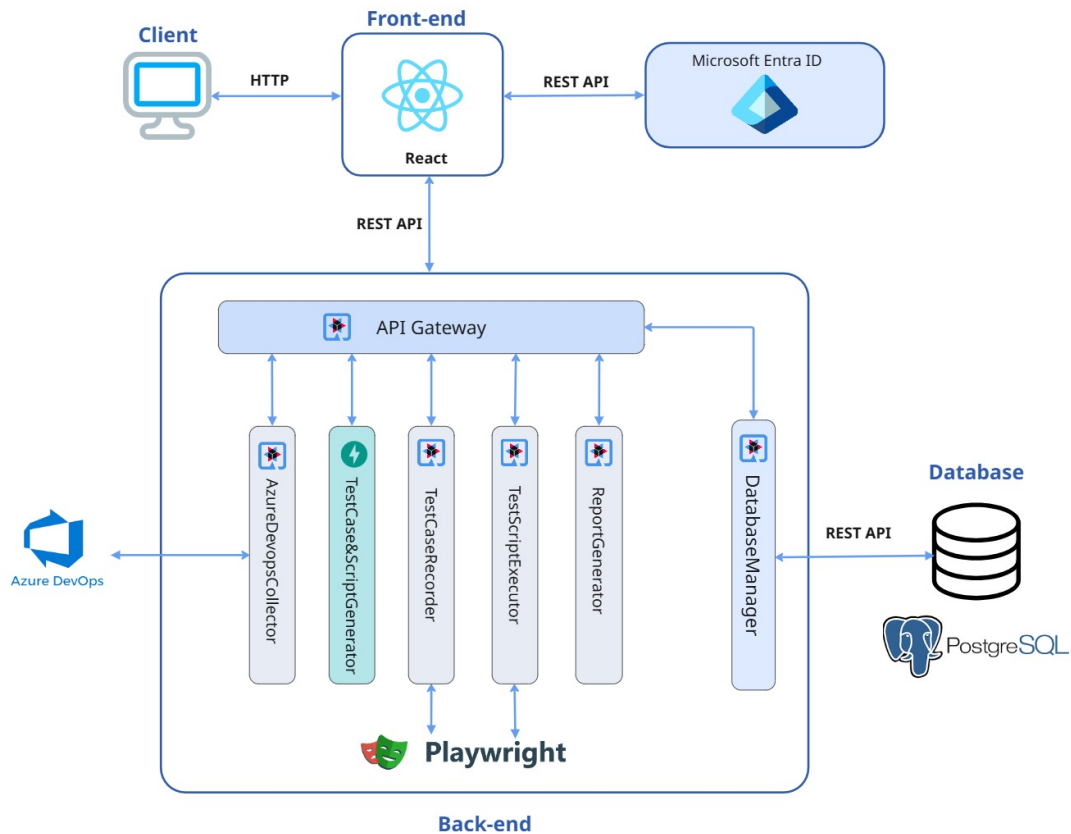


FIGURE 3.2 – Architecture microservices de « Smart QA »

Les microservices suivants ont été conçus pour répondre aux besoins fonctionnels de notre application :

- **APIGateway** : Ce microservice constitue le point d’entrée central du système. Il assure la coordination entre le front-end, les différents microservices et la base de données. Il a été développé pour faciliter la distribution des requêtes vers les services concernés et gérer la logique métier globale de l’application.
- **DatabaseManager** : Responsable des interactions avec la base de données PostgreSQL, ce microservice gère le stockage, la mise à jour et la récupération des données générées ou utilisées par les autres services.
- **AzureDevopsCollector** : Ce composant est dédié à la récupération des UserStories et des membres de projets depuis la plateforme Azure DevOps.
- **TestCaseAndScriptGenerator** : Basé sur l’intelligence artificielle, ce microservice génère des cas de test à partir des UserStories récupérées, et il est chargé de produire

intelligemment les scripts de test associés, prêts pour leur automatisation dans l'exécution des tests.

- **TestCaseRecorder** : Ce microservice prend en charge le lancement d'une session de capture (recording) pour un cas de test donné. Il initialise l'outil Playwright, ouvre l'interface de l'application à tester, et enregistre les interactions effectuées par l'utilisateur.
- **TestScriptExecutor** : Il exécute automatiquement les scripts de test générés. Il se charge de lancer les tests et de fournir les résultats obtenus.

Justification du choix de Playwright : Playwright a été retenu pour son efficacité en automatisation des tests fonctionnels. Plus performant que Selenium, il prend en charge nativement les navigateurs modernes (Chromium, Firefox, WebKit) et s'adapte bien aux applications complexes comme les applications monopages (SPA). Il offre également **Codigen**, un outil qui enregistre les interactions utilisateur et génère automatiquement des scripts JavaScript avec des sélecteurs précis, réduisant ainsi l'effort manuel et améliorant l'efficacité de l'automatisation.

- **TestReportGenerator** : Ce microservice s'occupe de la configuration et de la génération des rapports de test, à partir des UserStories, des cas de test et des résultats d'exécution. Il fournit une vue détaillée de l'avancement et de la qualité des tests.

3.3 Conception détaillée

Dans cette section, nous présentons le diagramme de classes de notre solution Smart QA, ainsi que plusieurs diagrammes de séquence associés à des scénarios d'utilisation clés du système.

3.3.1 Diagramme de classes

Un diagramme de classe est une représentation graphique qui modélise les structures statiques d'un système en décrivant ses classes, leurs attributs, méthodes, et les relations entre elles. La figure 3.3 illustre le diagramme de classes de notre application.

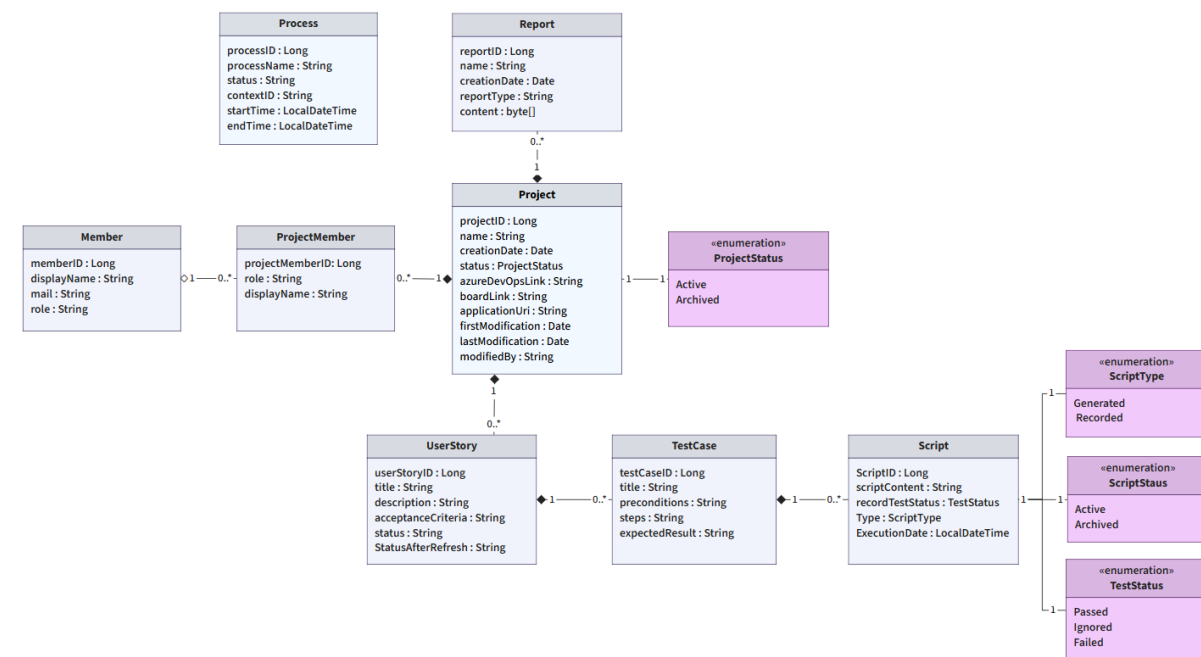


FIGURE 3.3 – Diagramme de classe

Dans ce diagramme, nous avons illustré les relations entre les différentes classes de notre système. En effet, nous distinguons les classes suivantes :

- **Process** : Représente un processus de test, qu’il s’agisse de la génération de cas de test ou de scripts de test, défini par un identifiant unique, un nom, un contexte spécifique (par exemple, une combinaison d’un identifiant de UserStory et d’un identifiant de cas de test), ainsi que des dates de début et de fin.
- **Project** : Modélise un projet avec un identifiant, un nom, un statut (actif ou archivé), et des informations telles que les liens Azure DevOps vers le tableau de bord.
- **Member** : Représente un utilisateur, caractérisé par un identifiant, un nom, une adresse email et un rôle (QA, DEV ou SuperAdmin).
- **UserStory** : Décrit une UserStory, avec un identifiant, un titre et des critères d’acceptation, servant de base à la génération des cas de test.
- **TestCase** : Représente un cas de test, avec un identifiant, un titre, des préconditions, des étapes et un résultat attendu.
- **Script** : Encapsule un script Playwright pour automatiser les tests, avec un identifiant, le contenu du script, un statut (actif ou archivé) et un type (généré ou enregistré).

- **Report** : Modélise un rapport généré, avec un identifiant, un nom, un type, un contenu et une date de création.

Les relations suivent une logique de composition ou d'agrégation selon le degré de dépendance entre les entités. Des énumérations comme ProjectStatus, ScriptType ou ScriptStatus garantissent la cohérence des données dans Smart QA.

3.3.2 Diagrammes de séquence

Le principal but d'un diagramme de séquence est de représenter les interactions entre les objets et les processus d'un point de vue temporel.

3.3.2.1 Diagramme de séquence de « S'authentifier »

Le diagramme de séquence du cas d'utilisation « S'authentifier » est représenté par la figure 3.4.

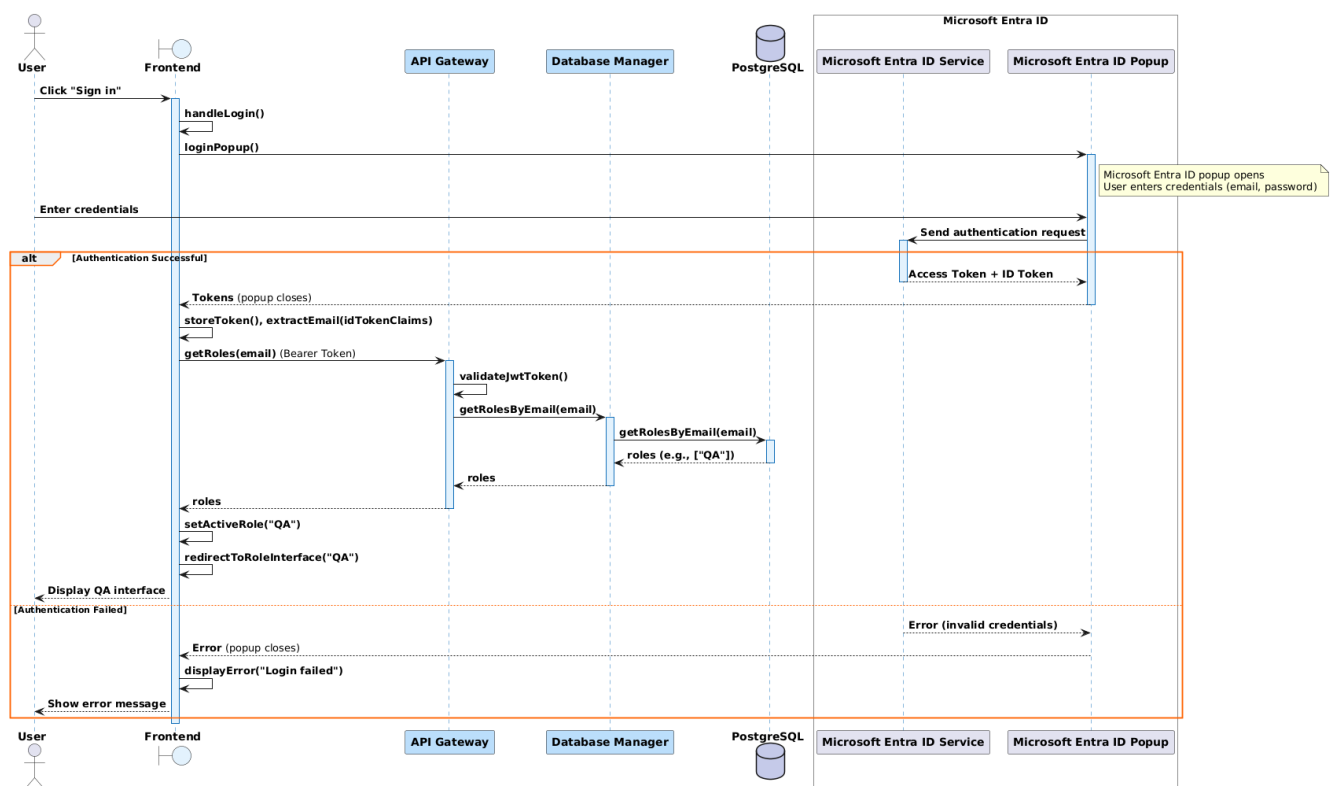


FIGURE 3.4 – Diagramme de Séquence de « S'authentifier »

3.3.2.2 Diagramme de séquence de « Générer un rapport de test »

Le diagramme de séquence du cas d'utilisation « Générer un rapport de test » est représenté par la figure 3.5.

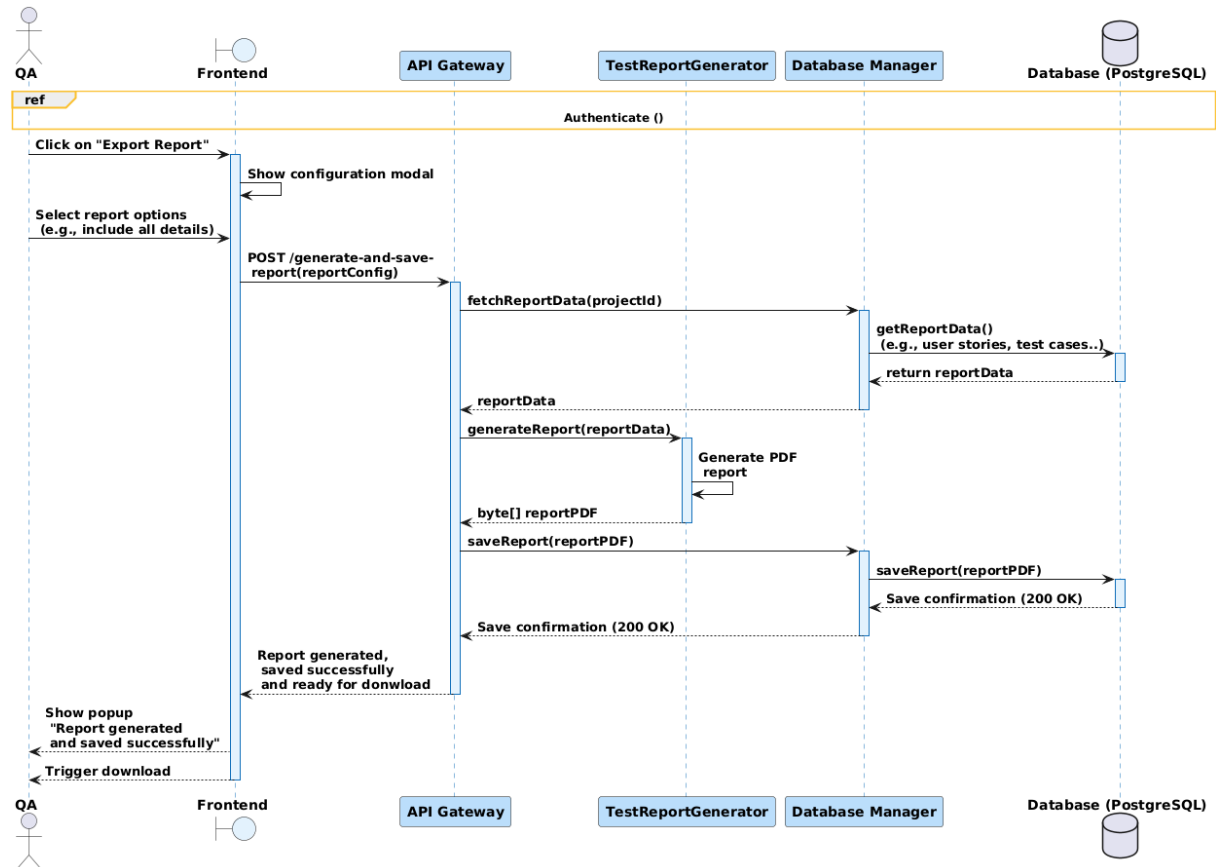


FIGURE 3.5 – Diagramme de Séquence de « Générer un rapport de test »

3.3.2.3 Diagramme de séquence de « Lancer un test automatisé »

Le diagramme de séquence du cas d'utilisation « Lancer un test automatisé » est représenté par la figure 3.6.

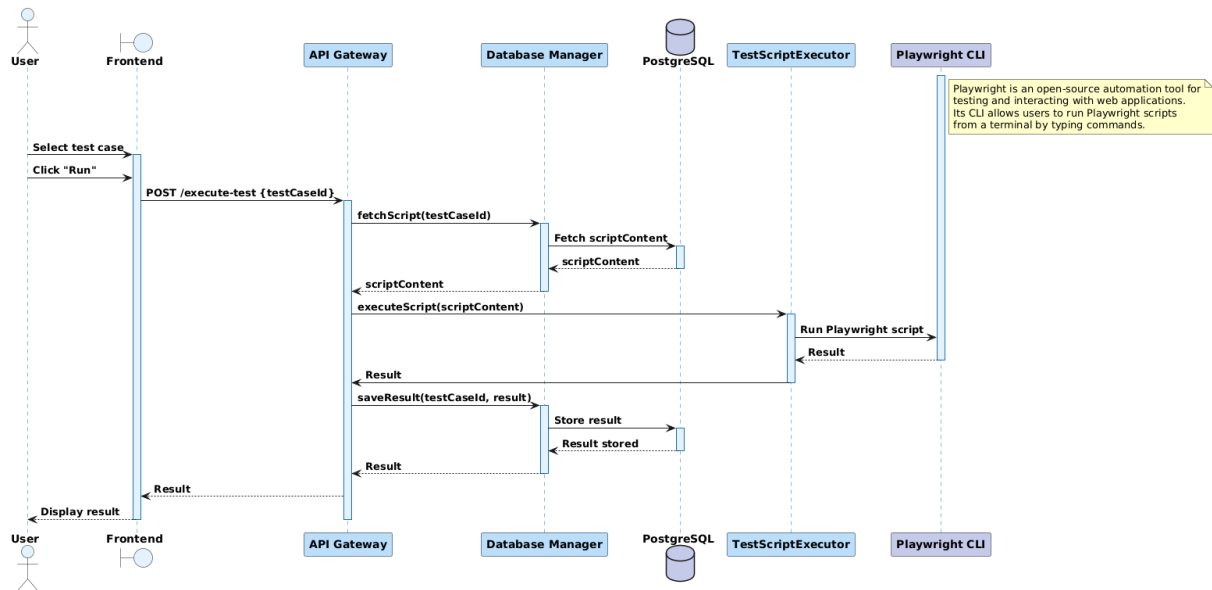


FIGURE 3.6 – Diagramme de Séquence de « Lancer un test automatisé »

3.4 Étude théorique des modules intelligents de génération des cas de test et des scripts de test

L'objectif de cette section est de présenter notre recherche préliminaire sur les approches d'**intelligence artificielle** permettant de générer des cas de test à partir des UserStories, ainsi que des scripts de test à partir des cas de test. Nous explorerons les méthodes communes entre ces deux modules, synthétiserons l'étude réalisée et les évaluations des modèles, puis présenterons notre démarche conceptuelle et les résultats obtenus. Les modules de génération seront détaillés dans les sous-sections suivantes.

3.4.1 Approches théoriques communes pour les deux modules

Dans le cadre de Smart QA, les UserStories constituent la base textuelle à partir de laquelle sont générés intelligemment les cas de test (sous forme de descriptions structurées) et les scripts de test (sous forme de code exécutable). Ce processus s'appuie sur des techniques de traitement automatique du langage naturel (NLP), afin d'extraire, interpréter et transformer les informations contenues dans les UserStories. L'objectif est d'accélérer la validation logicielle tout en

réduisant les efforts manuels. Deux approches principales issues de l'intelligence artificielle ont été étudiées dans ce cadre.

1. **NLP / NLU Classique** [19] : Cette méthode repose sur des chaînes de traitement déterministes du langage naturel, utilisant des outils comme spaCy ou NLTK pour analyser la syntaxe et la sémantique. Elle s'applique à l'extraction de structures (préconditions, étapes) pour les cas de test et à la génération de squelettes de code pour les scripts, en s'appuyant sur des règles métiers prédéfinies.

- **Analyse syntaxique** : Identification des verbes, sujets et compléments pour structurer les descriptions ou les actions de code.
- **Extraction d'entités** : Détection d'objets métiers (ex. « utilisateur », « mot de passe ») pour instancier les cas ou les données des scripts.
- **Règles heuristiques** : Application de templates (ex. « Si X alors tester Y » pour les cas, ou « Si action X alors écrire Y » pour les scripts).

2. **Large Language Models (LLM)** [20] : Les LLM sont des réseaux de grande taille (plusieurs milliards de paramètres) pré-entraînés sur d'immenses corpus textuels. Ils traitent les UserStories comme des prompts pour générer directement des cas de test structurés ou des scripts de code. Leur capacité à comprendre le contexte métier est clé pour les deux modules.

- **Compréhension contextuelle** : Les LLMs interprètent les intentions métier à partir du langage naturel, sans nécessiter de règles explicites, même dans des formulations variées ou incomplètes.
- **Génération unifiée** : Un même modèle peut produire l'ensemble des champs d'un cas de test (ID, préconditions, étapes, résultats attendus) ou générer directement des blocs de code cohérents pour les scripts d'automatisation.
- **Apprentissage en contexte** : Les LLMs sont capables de s'adapter dynamiquement aux exemples fournis dans le prompt, sans nécessiter de réentraînement. Cela permet de guider la génération en fournissant quelques cas de test modèles ou formats souhaités, ce qui améliore la qualité et la cohérence des résultats.

3.4.2 Comparaison des approches

Cette section présente une synthèse des avantages et des limites des deux principales approches utilisées pour la génération des cas et des scripts de test, comme illustré dans le tableau 3.1, afin de justifier le choix retenu pour le projet.

TABLE 3.1 – Comparaison des approches de génération des cas de test et des scripts de test

Approche	Avantages	Limites
NLP / NLU classique	– Processus transparent et contrôlable. – Réduit le risque d’erreur dans les structures prédéfinies. – Implémentation légère sans dépendances lourdes.	– Gestion limitée de la variabilité des formulations. – Difficulté avec les cas complexes ou ambigus. – Nécessite une mise à jour manuelle des règles. – Couverture insuffisante des cas limites et absence d’adaptation automatique aux nouveaux scénarios.
Large Language Models (LLM)	– Compréhension fine du contexte et du langage naturel. – Génération de cas variés et adaptés. – Réduction importante des efforts humains.	– Risque de génération incorrecte (hallucinations). – Infrastructure coûteuse requise.

Suite à cette comparaison, les LLM se distinguent par leur flexibilité et leur capacité à interpréter avec finesse les exigences fonctionnelles. Contrairement aux approches classiques basées sur le NLP traditionnel, souvent rigides et limitées dans la gestion de la diversité des formulations, les LLM offrent une meilleure compréhension contextuelle et une adaptation plus naturelle aux différents cas d’usage.

C'est cette richesse d'analyse et cette capacité à générer des cas de test variés et pertinents qui justifient leur adoption pour le module intelligent de Smart QA. Néanmoins, leur intégration implique des défis techniques, notamment en termes de ressources nécessaires et de maîtrise des risques liés à la génération incorrecte d'informations, qui doivent être soigneusement gérés.

Dans la suite, nous procéderons à une analyse comparative des principaux LLM afin d'identifier celui qui répond le mieux aux contraintes et aux objectifs du projet.

3.4.3 Prompt Engineering dans le contexte des LLM

Le succès des grands modèles de langage (LLM) dans la génération de contenu repose en grande partie sur la qualité des instructions fournies, appelées *prompts*. Le prompt engineering [21], ou l'art de concevoir ces instructions, est une technique essentielle pour optimiser les performances des LLM. Cette sous-section présente les principes théoriques du prompt engineering et son application dans notre contexte.

3.4.3.1 Importance du prompt engineering

Les LLM sont des modèles génératifs qui produisent du texte en fonction des instructions reçues. La formulation du prompt influence directement la pertinence, la structure et la qualité de la sortie générée. Un prompt bien conçu permet de :

- Guider le modèle vers la tâche spécifique à accomplir.
- Imposer des contraintes de format (par exemple, champs structurés pour les cas de test, syntaxe correcte pour les scripts).
- Réduire les hallucinations en fournissant un contexte clair et des exemples pertinents.
- Adapter le modèle à des domaines spécifiques sans nécessiter de réentraînement complet.

Dans notre projet, le prompt engineering est crucial pour garantir que les LLM produisent des sorties conformes aux exigences des modules de génération des cas de test et des scripts de test. Il permet également d'exploiter au mieux les capacités des modèles sans avoir à les réentraîner, ce qui serait coûteux en temps et en ressources.

3.4.3.2 Techniques de prompt engineering

Pour optimiser les performances des grands modèles de langage (LLM), plusieurs techniques de prompt engineering existent afin de mieux guider le modèle dans l'exécution de tâches spécifiques. Les principales méthodes incluent :

- **Zero-shot prompting** : le modèle reçoit uniquement une instruction sans exemple, adapté aux tâches simples mais limité pour les cas complexes.
- **One-shot prompting** : un seul exemple est fourni pour clarifier les attentes, offrant un bon équilibre entre simplicité et efficacité.
- **Few-shot prompting** : plusieurs exemples sont donnés, ce qui améliore la précision pour les tâches techniques mais augmente le coût en tokens.

Deux techniques avancées complètent ces approches :

- **Role prompting** : le modèle adopte un rôle spécifique (ex. expert en automatisation), ce qui renforce la pertinence des réponses dans des contextes spécialisés.
- **Prompt chaining** : une tâche complexe est décomposée en étapes successives, augmentant la cohérence et la précision des résultats.

Ces techniques, appliquées de manière stratégique, sont essentielles pour la génération automatique de cas de test et de scripts, et seront détaillées dans les sections suivantes.

3.4.4 Choix du LLM : Évaluation des modèles et résultats

Cette section détaille les modèles de langage large (LLMs) évalués pour la génération des cas de test et des scripts de test. Nous expliquons le protocole de benchmarking utilisé, la sélection des modèles, ainsi que les performances obtenues.

3.4.4.1 Protocole de benchmarking commun

Pour évaluer les LLM, un protocole de benchmarking systématique est mis en place. Trois à quatre UserStories représentatives, couvrant diverses fonctionnalités de Smart QA, servent de base pour tester la généralisabilité. Les prompts d'entrée sont standardisés pour garantir une comparaison équitable, avec des ajustements selon le type de sortie (cas de test ou script).

Les critères d'évaluation incluent :

- **Conformité au format** : Respect des structures attendues (champs pour les cas, syntaxe pour les scripts).
- **Clarté et cohérence** : Compréhensibilité des descriptions ou du code généré.
- **Couverture fonctionnelle** : Capacité des cas de test et des scripts à couvrir et tester toutes les fonctionnalités clés du système.
- **Détail** : Niveau de précision dans les étapes ou les assertions.

En l'absence de métriques objectives pour évaluer la pertinence des sorties générées par les LLMs dans ce contexte spécifique, l'évaluation repose exclusivement sur une analyse humaine réalisée par des experts QA (*human feedback*). Les résultats issus de cette analyse guident le choix du modèle le plus adapté pour chaque module, tel que détaillé dans les sections suivantes.

3.4.4.2 Modèles évalués pour la génération des cas de test

Nous avons retenu et comparé cinq modèles open-source : Google/mt5-small, Mistral-7B, Llama-2-7B-Chat, DeepSeek-R1-Distill-Qwen-7B et DeepSeek-R1-Distill-Llama-8B, en raison de leur diversité architecturale, de leur accessibilité open-source et de leur compatibilité avec des environnements aux ressources limitées. Les résultats sont synthétisés dans le tableau 3.2.

TABLE 3.2 – Comparaison des LLM pour la génération de cas de test

Modèle	Format	Clarté	Couverture	Détail
Google/mt5-small	Format souvent incorrect.	Explications peu claires.	Nombreuses fonctionnalités non couvertes.	Étapes incomplètes ou générales.
Mistral-7B	Format assez stable.	Explications parfois ambiguës.	Couvre les principaux scénarios.	Détail parfois insuffisant.
Llama-2-7B-Chat	Format correct mais inégal.	Lisibilité moyenne, manque de précision.	Couvre certains cas standards.	Détail limité pour les scénarios complexes.
DeepSeek-R1-Distill-Qwen-7B	Format stable et clair.	Explications structurées.	Couvre la plupart des cas utiles.	Bon niveau de détail global.
DeepSeek-R1-Distill-Llama-8B	Format très rigoureux.	Explications précises et claires.	Couvre la plupart des cas attendus.	Détail complet et bien structuré.

Le modèle **DeepSeek-R1-Distill-Llama-8B** s'est révélé être le meilleur choix, avec des performances maximales sur l'ensemble des critères. Il a donc été retenu pour la suite de notre approche.

Il est à noter que la qualité des UserStories et des critères d'acceptation impacte directement la qualité des cas de test générés, quel que soit le modèle. Des entrées claires et précises améliorent la pertinence des sorties et réduisent le temps de génération en évitant l'interprétation d'informations ambiguës.

3.4.4.3 Modèles évalués pour la génération des scripts de test

Pour la génération des scripts, trois modèles ont été évalués : StarCoder, DeepSeek-Distill-Llama-8B et Mistral-7B. Leurs performances sont résumées dans le tableau 3.3.

TABLE 3.3 – Comparaison des LLM pour la génération de scripts de test

Modèle	Format	Clarté	Couverture	Détail
StarCoder	Format assez stable, structure claire mais parfois à revoir.	Code généralement lisible, certains passages peuvent manquer de précision.	Principales fonctionnalités bien identifiées.	Détail correct, mais certaines étapes restent sommaires.
DeepSeek-Distill-Llama-8B	Format très rigoureux, structure homogène.	Explications précises et faciles à suivre.	Large couverture incluant les scénarios complexes.	Détail complet, chaque étape et assertion est bien explicitée.
Mistral-7B	Format bien respecté, structure logique.	Explications claires dans l'ensemble.	Bon niveau de couverture des fonctionnalités importantes.	Détail généralement suffisant, quelques cas à approfondir.

DeepSeek-Distill-Llama-8B a été retenu pour sa performance globale supérieure, cohérente avec ses résultats pour les cas de test.

3.4.5 Modèle adopté : LLM DeepSeek-R1-Distill-Llama-8B

Le modèle DeepSeek-R1-Distill-Llama-8B [22] appartient à la famille DeepSeek, basée sur l'architecture Transformer, reconnue pour ses mécanismes d'attention qui permettent une compréhension contextuelle avancée du langage. Cette version optimisée combine deux techniques clés : la distillation, qui consiste à entraîner un modèle compact (8 milliards de paramètres) à reproduire les performances d'un modèle beaucoup plus volumineux, et la quantification, qui

réduit la taille des poids du modèle pour diminuer la consommation mémoire et accélérer les calculs. Grâce à ces optimisations, DeepSeek-R1-Distill-Llama-8B offre un excellent compromis entre performances et efficacité, ce qui le rend particulièrement adapté aux environnements à ressources limitées (machines avec mémoire et puissance de calcul réduites). Cette capacité à maintenir une génération cohérente et pertinente, even dans ces conditions, en fait un choix pertinent pour des applications exigeantes comme la génération automatique de cas de test et de scripts.

3.5 Mise en Œuvre des modules intelligents de génération des cas de test et des scripts de test

Après l'étude théorique des approches d'intelligence artificielle, cette section décrit la mise en œuvre pratique des deux modules intelligents au sein de Smart QA en utilisant le modèle justifié. L'objectif est double : d'une part, générer à partir des UserStories et leurs critères d'acceptation des cas de test structurés, et d'autre part, générer des scripts automatisés et exécutables à partir de ces cas de test. Nous détaillons ci-dessous l'implémentation technique de chacun de ces modules.

3.5.1 Outil utilisé

Nous avons utilisé le modèle **DeepSeek-R1-Distill-Llama-8B** avec la plateforme **Ollama** [23], une plateforme permettant d'exécuter localement des modèles de langage.

L'un des avantages majeurs d'Ollama est sa souplesse de configuration : il suffit d'un simple paramétrage pour basculer vers un autre modèle, sans modifier l'architecture de l'application.

De plus, les modèles sont exécutés localement, ce qui permet d'adapter leur utilisation aux ressources matérielles disponibles. Il est ainsi possible de déployer une version quantifiée du modèle, réduisant significativement l'empreinte mémoire et les besoins en calcul, un point crucial pour les LLM, dont la taille est généralement très importante.

Enfin, l'exécution en local garantit que les données traitées ne quittent jamais l'environnement de travail, ce qui assure une confidentialité totale : les informations du projet ne sont pas exposées à des serveurs distants, et ne peuvent donc pas être utilisées à des fins d'entraînement ou d'exploitation par des modèles hébergés.

3.5.2 Mise en Œuvre du module de génération des cas de test

Dans cette section, nous détaillons l'implémentation technique du module de génération des cas de test à partir des UserStories.

Pour la génération des cas de test, nous avons intégré le modèle DeepSeek-R1-Distill-Llama-8B avec Ollama. Les entrées fournies au modèle sont les UserStories, composées d'un titre et de critères d'acceptation. À partir de ces informations, le modèle génère automatiquement des cas de test respectant un format défini : identifiant, titre, préconditions, étapes numérotées et résultat attendu, comme illustré dans la figure 3.7.

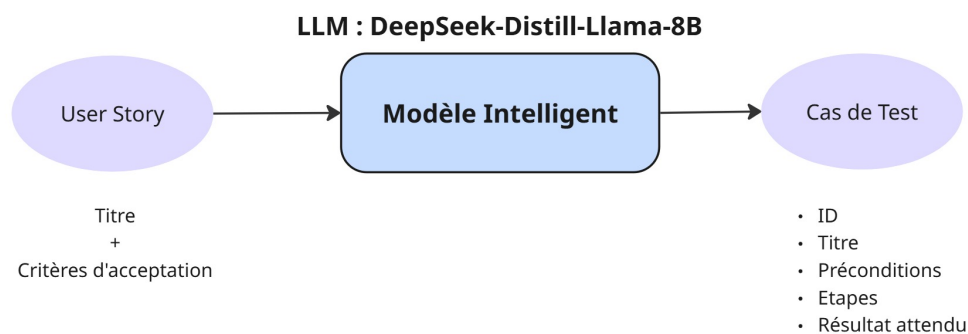


FIGURE 3.7 – Workflow de la génération des cas de test

3.5.2.1 Conception du prompt

La conception du prompt a été un élément clé pour garantir la qualité des cas de test générés. Une approche itérative a été adoptée afin d'optimiser les performances du modèle. Initialement, des prompts simples ont été testés, comme :

Générer des cas de test pour la UserStory suivante : [Titre et critères d'acceptation]

Cependant, ces prompts produisaient des réponses souvent incomplètes ou mal formatées, ne respectant pas les standards attendus. Pour remédier à cela, une structure plus détaillée a été introduite, en intégrant un rôle, un format strict et un contexte clair.

Le prompt final utilisé est le suivant :

```
Vous êtes un expert en tests logiciels.  
À partir du titre et des critères d'acceptation d'une UserStory,  
générez une liste de cas de test en français,  
en suivant exactement ce format :  
ID du test :  
Titre :  
Préconditions :  
Étapes : (liste numérotée)  
Résultat attendu :  
Générez uniquement les cas de test pertinents.
```

Cette formulation a permis d'améliorer considérablement la structure et la pertinence des cas de test générés, confirmant l'importance d'un prompt bien conçu pour obtenir des résultats exploitables.

3.5.2.2 Résultats et validation des cas de test générés

Pour illustrer le fonctionnement du module, nous présentons ci-dessous un exemple de UserStory issu de notre backlog, ainsi que les cas de test générés automatiquement à partir de celle-ci.

a) UserStory :

"En tant que Super Admin, je veux créer un projet à partir du Dashboard afin de gérer efficacement mes projets dans le système". Les critères d'acceptation sont les suivants :

- À partir du Dashboard, le Super Admin peut saisir :

- L'URL du projet dans Azure DevOps.
 - L'URL du Board associé.
 - Le nom doit être récupéré à partir de l'URL du Board associé.
- Vérifier si le projet existe déjà (un projet ne peut pas être ajouté deux fois).
 - Afficher un pop-up de succès après création réussie.
 - Afficher un pop-up d'erreur en cas d'échec.

b) Cas de test générés :

Le tableau 3.4 présente un extrait représentatif des cas de test générés pour valider le bon fonctionnement de la UserStory relative à la création de projet dans l'application.

TABLE 3.4 – Exemples de cas de test générés

ID	Titre	Précondition	Étapes	Résultat attendu
TC-1	Saisie des URLs Azure DevOps et du Board	L'utilisateur est connecté en tant que Super Admin et accède au formulaire de création de projet.	1. Saisir une URL valide dans le champ Azure DevOps. 2. Saisir une URL valide du Board associé. 3. Soumettre le formulaire.	Les deux URLs sont acceptées et le processus de création continue.
TC-2	Récupération automatique du nom du projet	Une URL valide du Board a été saisie.	1. Saisir l'URL du Board. 2. Vérifier que le nom du projet est automatiquement extrait.	Le nom du projet est correctement affiché.
TC-3	Vérification de l'existence du projet	Un projet avec les mêmes paramètres existe déjà.	1. Tenter de créer un projet avec une URL déjà utilisée. 2. Soumettre le formulaire.	Un message d'erreur indique que le projet existe déjà.

Pour garantir la qualité des cas générés, une phase de validation automatique a été mise en place. Chaque cas est vérifié afin de s'assurer qu'il respecte la structure attendue (identifiant,

titre, précondition, étapes, résultat). Les cas incomplets ou non conformes sont automatiquement écartés, assurant ainsi la fiabilité des cas de test avant leur automatisation.

3.5.3 Mise en œuvre du module de génération des scripts de test

Cette section présente la mise en œuvre pratique du module de génération automatique de scripts de test au sein de Smart QA. L'objectif est de produire, à partir des cas de test spécifiés, des scripts JavaScript exploitables via **Playwright**, en tenant compte de la structure réelle des sélecteurs et des éléments UI de l'application à tester.

Un des principaux défis dans ce processus réside dans l'identification et l'exploitation des sélecteurs des éléments de l'interface utilisateur. Pour qu'un LLM puisse générer un script cohérent et fonctionnel, il est indispensable qu'il dispose d'informations précises sur les éléments à manipuler dans la page. L'absence ou l'inexactitude des sélecteurs rend la génération automatique peu fiable, voire inefficace. Afin de surmonter cette limitation, plusieurs stratégies complémentaires ont été mises en place.

Trois approches distinctes ont été développées, classées par priorité en fonction de leur robustesse et efficacité :

- **Génération à base d'enregistrement** : reposant sur des scripts générés automatiquement à partir d'enregistrements via l'outil *Codegen* de Playwright. Cette approche est privilégiée pour son efficacité à capturer les interactions réelles et précises avec l'interface utilisateur.
- **Génération fondée sur la similarité sémantique** : exploitant un ensemble de scripts existants des anciens tests pour générer un nouveau script par analogie. Cette méthode est utile lorsque des cas de test similaires existent déjà dans le référentiel.
- **Génération simple** : basée uniquement sur le contenu du cas de test. Bien qu'elle soit la moins robuste, elle est utilisée en dernier recours lorsque les deux autres approches ne peuvent être appliquées.

En pratique, le choix de la stratégie de génération dépend de l'existence d'un script associé au cas de test :

- Si le cas de test possède déjà un script associé issu d'un enregistrement (*recorded*), la génération s'appuie sur ce script (génération à base d'enregistrement). Si ce script avait déjà été généré automatiquement, il s'agit alors d'une régénération.
- Si le cas de test ne dispose pas de script associé, le système recherche dans l'ensemble des scripts existants pour identifier un cas similaire (génération fondée sur la similarité sémantique).
- Si aucune similarité pertinente n'est trouvée ou si aucun script n'existe dans le référentiel, la génération simple, basée uniquement sur le contenu textuel du cas de test, est alors appliquée.

Ces approches, illustrées dans le workflow général du système présenté à la Figure 3.8, seront détaillées dans les sections suivantes. Pour chacune d'elles, nous décrirons les principes de fonctionnement, la conception du prompt adressé au LLM, ainsi que le résultat final obtenu.

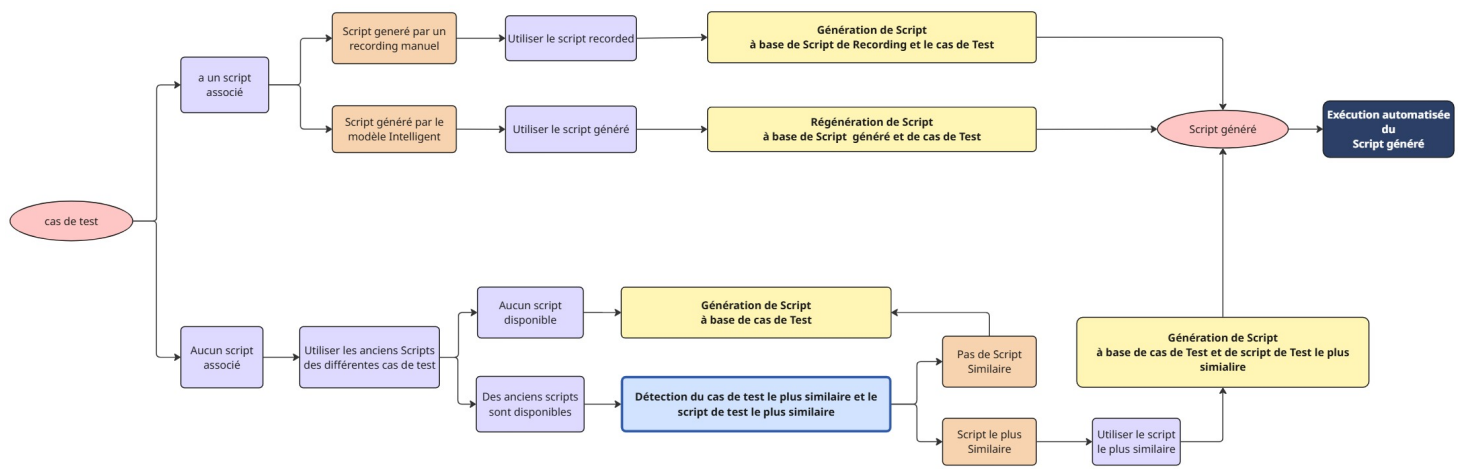


FIGURE 3.8 – Workflow général du module de génération de scripts de test

3.5.3.1 Conception de prompt

Le prompt est conçu pour guider le modèle vers une génération de script structurée et conforme aux bonnes pratiques d'automatisation avec Playwright. Les éléments principaux incluent :

- **Structure Playwright** : Les scripts générés respectent les conventions Playwright, incluant la déclaration des modules nécessaires (`const`, `test`, `expect`) et l'utilisation de sélecteurs robustes comme `getByRole`, `getByLabelText`, ou `page.locator`.
- **Actions et assertions** : Les actions (`page.fill`, `page.click`, etc) sont dérivées des étapes du cas de test, et l'insertion d'assertions explicites (`expect`) est systématique pour valider les résultats attendus.
- **Adaptation au contexte** : Les scripts doivent refléter les spécificités des cas de test, comme la navigation vers une URL (`page.goto`), les actions requises, et les validations.

Chaque approche adapte ce cadre commun en fonction des données disponibles et des besoins spécifiques, comme décrit dans les sections suivantes.

3.5.3.2 Génération à base d'enregistrement

Cette approche consiste à générer un script Playwright en s'appuyant sur un enregistrement initial réalisé à l'aide de l'outil **Codegen** de Playwright. Cet outil capture automatiquement les interactions de l'utilisateur avec l'interface de l'application (navigation, saisie de texte, clics, etc) lors de l'exécution manuelle d'un cas de test par un QA. Toutefois, le code produit par *Codegen* est purement descriptif et ne contient généralement pas d'assertions permettant de valider le comportement attendu.

Le modèle **DeepSeek-R1-Distill-Llama-8B** exploite un enregistrement (recording) comme base pour générer un script de test enrichi et aligné avec le cas de test défini.

Le **recording** désigne une capture automatique des interactions utilisateur avec l'interface de l'application (clics, saisies, navigations, etc), réalisée via un outil d'enregistrement de sessions ou de tests. Cette trace fournit une vue détaillée du parcours utilisateur ainsi que des sélecteurs précis des éléments de l'interface (boutons, champs, liens, etc).

En s'appuyant sur cet enregistrement, le modèle conserve les éléments contextuels pertinents (identifiants CSS/XPath, structure DOM, etc), ce qui facilite l'interprétation du scénario par le LLM. Il est ensuite capable d'éliminer les actions superflues, d'ajouter les assertions manquantes, et de structurer le script généré de manière plus cohérente et exploitable.

a) Adaptation du prompt

Pour cette approche, le prompt est ajusté afin de :

- **Conserver les locators, textes saisis, et clics du script enregistré** sans modification.
- **Ajouter des assertions** pour vérifier les résultats attendus du cas de test.
- **Supprimer les actions inutiles** qui ne correspondent pas aux étapes du cas de test.

Cette méthode permet au modèle d’avoir une vision concrète de l’application à tester, en ancrant la génération dans le réel, et constitue une base fiable pour les générations ultérieures de scripts optimisés ou personnalisés.

3.5.3.3 Génération basée sur la similarité

Cette approche s’appuie sur l’exploitation d’un référentiel de cas de test existants, dans lequel chaque cas est associé à un ou plusieurs scripts d’automatisation déjà validés. L’objectif est d’identifier le cas de test le plus proche sur le plan sémantique de celui pour lequel un nouveau script doit être généré.

Une fois cette similarité établie, le script correspondant est réutilisé comme base, puis adapté automatiquement au nouveau contexte, en tenant compte des différences fonctionnelles ou techniques entre les deux cas. Cette stratégie permet de capitaliser sur les travaux existants et de réduire significativement le coût de génération des scripts tout en garantissant une certaine cohérence et fiabilité.

a) Étude de similarité

La similarité sémantique est évaluée à l’aide du modèle **all-MiniLM-L6-v2** [24], une version compacte et distillée de **BERT**, implémentée via la bibliothèque **SentenceTransformers**. Ce modèle est spécifiquement optimisé pour produire des *embeddings* (représentations vectorielles) de phrases, permettant de capturer efficacement leur sens dans un espace vectoriel de dimension réduite, avec un bon compromis entre performance et rapidité.

Chaque cas de test est représenté par un vecteur calculé à partir de ses champs textuels (titre, préconditions, étapes, et résultat attendu). La mesure de similarité entre deux cas est ensuite calculée via la **similarité cosinus** [25], qui quantifie l'angle entre les vecteurs correspondants. Formellement, étant donné u le vecteur du cas de test d'entrée et v_i celui d'un cas de test du référentiel, la similarité est donnée par :

$$\text{similarite} = \frac{u \cdot v_i}{\|u\| \|v_i\|}$$

Un score proche de 1 indique une forte similarité (petit angle), tandis qu'un score proche de 0 traduit une dissimilarité marquée. La figure 3.9 illustre ce principe.

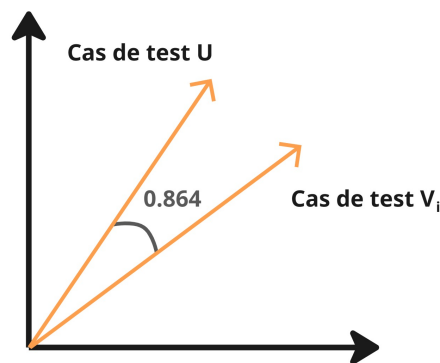


FIGURE 3.9 – Illustration de la similarité cosinus entre deux vecteurs de cas de test

Le processus complet se déroule en quatre étapes principales :

1. **Encodage** : Le cas de test cible est encodé en un vecteur u à l'aide de `all-MiniLM-L6-v2`. Les cas du référentiel sont également encodés en vecteurs v_i .
2. **Calcul de similarité** : La similarité cosinus est calculée entre u et chaque v_i . Les cas trop proches (score = 1) sont exclus, afin d'éviter de réutiliser un script identique. On privilégie des cas similaires mais distincts (score typiquement $\geq 0,7$).
3. **Sélection** : Le cas de test ayant la similarité maximale est sélectionné. Parmi ses scripts associés, on identifie celui dont la description est également proche du cas cible (score $\geq 0,4$).
4. **Adaptation** : Le script sélectionné est adapté par le modèle **DeepSeek-R1-Distill-Llama-8B**, qui :

- conserve les sélecteurs spécifiques du DOM (ex. `page.locator(...)`).
- ajuste la navigation si besoin (ex. `page.goto(...)`).
- insère des assertions (`expect`) correspondant au résultat attendu.
- supprime les actions redondantes ou non pertinentes.

En s'appuyant sur des cas de test similaires présents dans le référentiel, cette approche combine la recherche sémantique et la génération assistée par LLM pour produire un script pertinent et adapté aux spécificités du cas cible.

b) Adaptation du prompt

Pour cette approche, le prompt est ajusté afin de :

- **Adapter un script existant** au nouveau cas de test en conservant les sélecteurs pertinents.
- **Insérer des assertions** pour valider les résultats attendus.
- **Supprimer les actions redondantes** ou non pertinentes.

Cette approche combine la recherche sémantique et la génération assistée par LLM pour produire un script pertinent et adapté.

3.5.3.4 Génération simple (basée uniquement sur le cas de test, sans script de référence)

Cette approche consiste à générer un script de test automatisé en se basant uniquement sur la description textuelle du cas de test : le titre, les préconditions, les étapes à suivre et le résultat attendu. Le modèle **DeepSeek-R1-Distill-Llama-8B** est utilisé pour interpréter ces informations et produire un script conforme aux conventions de Playwright.

a) Adaptation du prompt

Pour cette approche, le prompt est ajusté afin de :

- **Extraire les actions** à partir des étapes du cas de test.
- **Insérer des assertions** explicites pour valider le comportement attendu.
- **Utiliser des sélecteurs sémantiques** robustes comme `getByRole` ou `getByLabelText`.

Cette méthode est utilisée en dernier recours lorsque ni un enregistrement ni un cas similaire ne sont disponibles.

3.5.3.5 Résultat obtenu

Pour illustrer le fonctionnement du module de génération de scripts, nous présentons ci-dessous un exemple de cas de test issu de notre référentiel, ainsi que le script Playwright généré automatiquement à partir de celui-ci.

- **Titre :** Récupération du nom du projet à partir de l'URL du Board associé.

Précondition : Une URL valide du Board a été saisie.

Étapes :

1. Saisir l'URL du Board.
2. Vérifier que le nom du projet est automatiquement extrait.

Résultat attendu : Le nom du projet est correctement affiché.

Script généré

```
import { test, expect } from '@playwright/test';

test('Extraction automatique du nom de projet',
  'depuis l'URL du Board', async ({ page }) => {
  await page.goto('https://exemple-app.fr/board');

  await page.fill('input[name="boardUrl"]',
    'https://board.exemple.fr/projet123');

  await page.press('input[name="boardUrl"]', 'Enter');

  const projectName = await page.locator('#project-name').textContent();

  expect(projectName).toContain('projet123');
});
```

La validation du script généré est effectuée en vérifiant la conformité syntaxique (absence d'erreurs, respect des mots-clés essentiels comme `const`, `test`, `expect`, `page`), l'exécutabilité dans un environnement Playwright, et la correspondance fonctionnelle entre les actions du script et les étapes décrites dans le cas de test.

Ce script, généré sans intervention humaine, répond aux critères de validation définis : il est structuré correctement, fonctionne sans erreur, et reflète fidèlement les objectifs du cas de test pour produire un code exploitable et fiable.

3.6 CONCLUSION

À travers ce chapitre, nous avons présenté l'architecture de l'application Smart QA, en illustrant ses différents modules et leurs interactions à l'aide de diagrammes UML. Nous avons ensuite mené une étude approfondie des techniques d'intelligence artificielle, en explorant les approches de génération de cas de test et de scripts de test, basées sur le modèle DeepSeek-R1-Distill-Llama-8B. Le prochain chapitre portera sur la mise en œuvre de notre application.

Réalisation

Sommaire

4.1	INTRODUCTION	73
4.2	Description du travail réalisé	73
4.2.1	Configuration de Microsoft Entra ID	73
4.2.2	Interface d'authentification à l'application	74
4.2.3	Interface de configuration du projet et attribution des rôles . .	74
4.2.4	Interfaces de lancement d'une exécution de test	78
4.2.5	Interface de consultation des projets pour le développeur . . .	86
4.2.6	Interface du tableau de bord	88
4.3	CONCLUSION	89

4.1 INTRODUCTION

La phase de développement, aussi appelée phase technique, marque la concrétisation de tout le travail de conception effectué pendant le projet. Ce chapitre présente en détail la réalisation de notre solution logicielle. Nous y décrivons les différentes étapes du travail, accompagnées de captures d'écran pour mieux illustrer nos actions.

4.2 Description du travail réalisé

Dans cette section, nous présentons le travail réalisé durant toute la phase d'implémentation en décrivant les interfaces de notre système par des captures d'écran.

4.2.1 Configuration de Microsoft Entra ID

Nous avons configuré Microsoft Entra ID pour gérer l'authentification et sécuriser l'accès à notre application. La figure 4.1 illustre l'interface de configuration, où nous retrouvons l'ID de l'application (client), une information essentielle permettant l'authentification et l'enregistrement sécurisé de l'application au sein du système.

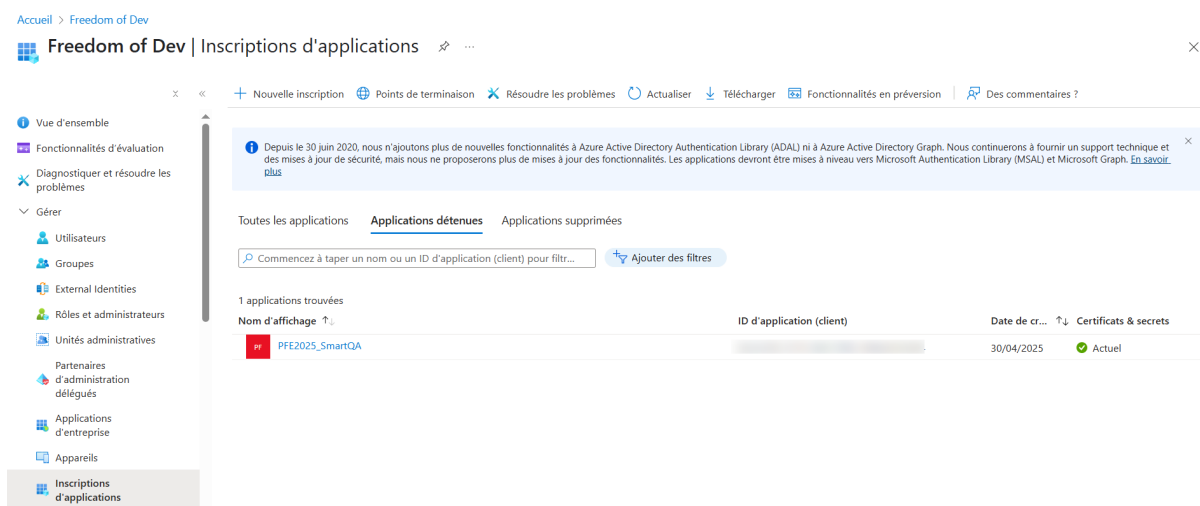


FIGURE 4.1 – Interface de Microsoft Entra ID

4.2.2 Interface d'authentification à l'application

Pour accéder à l'application, l'utilisateur doit cliquer sur le bouton «Sign in with Microsoft», ce qui le redirige vers un processus d'authentification sécurisé. Les différentes étapes de ce processus sont illustrées à la figure 4.2.

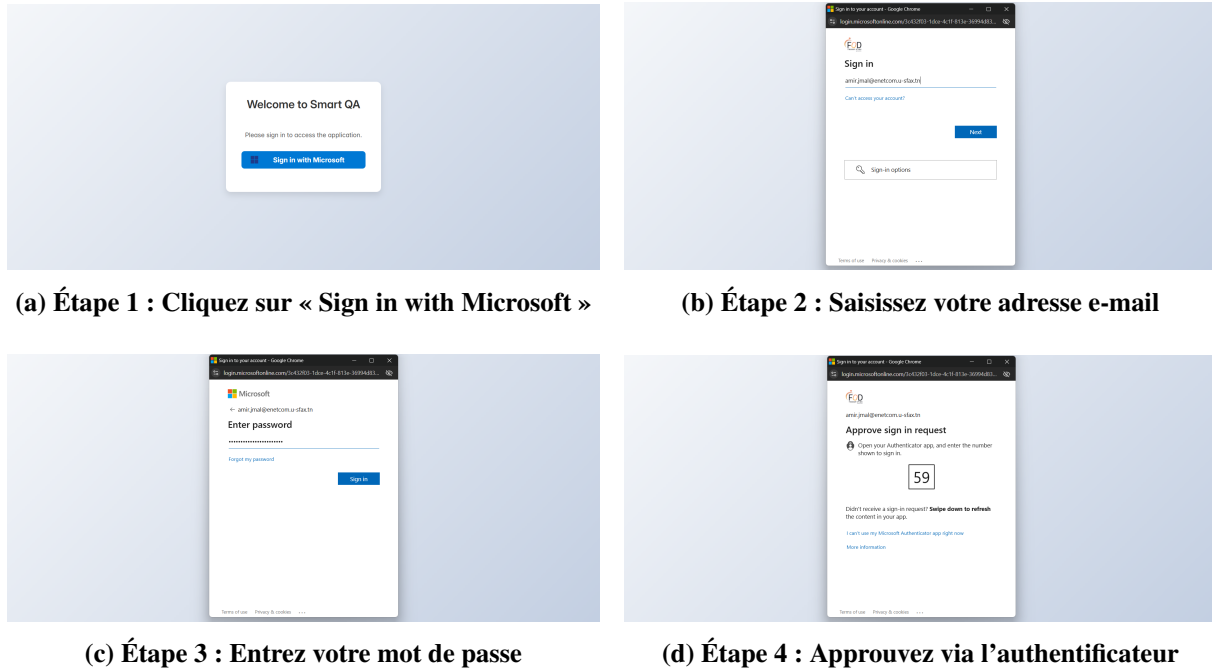


FIGURE 4.2 – Étapes du processus d'authentification

4.2.3 Interface de configuration du projet et attribution des rôles

Après une authentification réussie, le Super Admin est redirigé vers l'interface de liste des projets. Cette interface permet d'afficher les projets existants. La figure 4.3 présente cette vue.

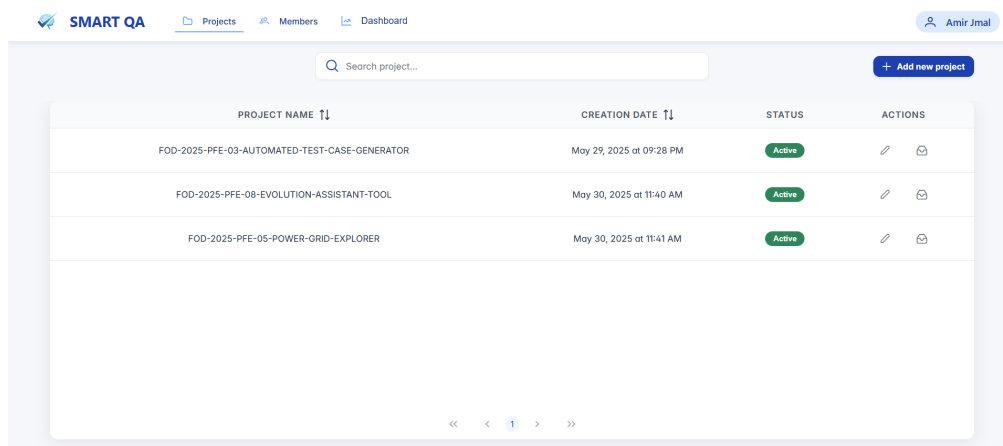
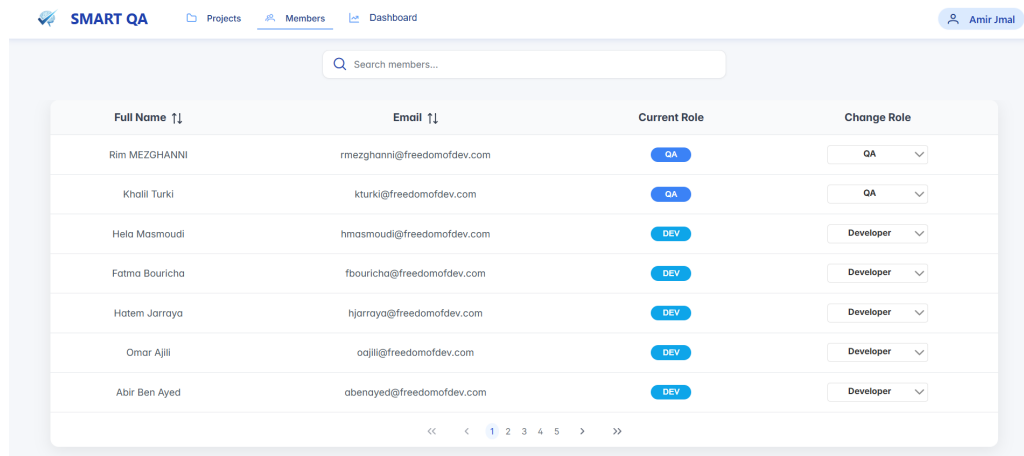


FIGURE 4.3 – Interface de liste des projets pour le Super Admin

4.2.3.1 Attribution des rôles

De plus, le Super Admin peut accéder aux membres collectés de l'organisation depuis Azure DevOps et attribuer des rôles. En fait, tous les membres de l'organisation sont récupérés avec le rôle Dev par défaut, comme illustré dans la figure 4.4.



Full Name ↑↓	Email ↑↓	Current Role	Change Role
Rim MEZGHANNI	rmezghanni@freedomofdev.com	QA	QA
Khalil Turki	kturki@freedomofdev.com	QA	QA
Hela Masmoudi	hmasmoudi@freedomofdev.com	DEV	Developer
Fatma Bouricha	fbouricha@freedomofdev.com	DEV	Developer
Hatem Jarraya	harraya@freedomofdev.com	DEV	Developer
Omar Ajili	oajili@freedomofdev.com	DEV	Developer
Abir Ben Ayed	abenayed@freedomofdev.com	DEV	Developer

FIGURE 4.4 – Liste des membres récupérés depuis Azure DevOps

Le Super Admin peut modifier ce rôle et attribuer à chaque membre soit le rôle Dev, soit le rôle QA. Si le Super Admin veut changer un rôle d'un membre de Dev à QA, cela se passe normalement. Cependant, changer un rôle de QA à Dev n'est pas autorisé sauf si ce membre ne possède aucun rôle QA dans un projet, comme montré dans la figure 4.5.

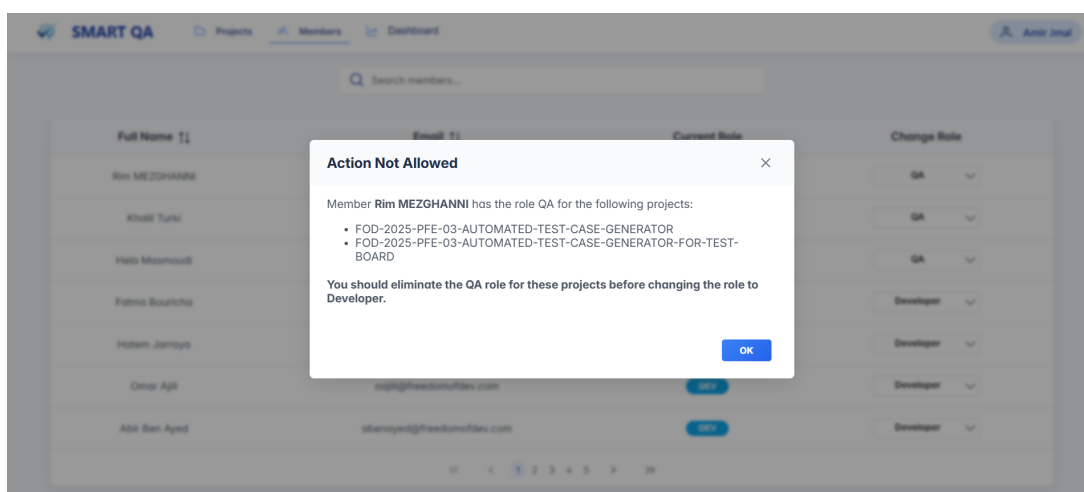


FIGURE 4.5 – Interface d'attribution des rôles

4.2.3.2 Ajout d'un nouveau projet

Le Super Admin peut ajouter un nouveau projet, comme illustré dans la figure 4.6. Cette action ouvre une interface dédiée où l'admin remplit les champs de formulaire nécessaires :

- **Azure DevOps Link** : lien vers le projet dans Azure DevOps.
- **Board Link** : lien vers le tableau associé (le nom du projet est automatiquement extrait du « Board Link »).
- **Application URL (optionnel)** : lien vers l'application à tester, si disponible.

Ensuite, le Super Admin clique sur « Load members » pour récupérer les membres associés au projet dans Azure DevOps. Ces membres sont chargés avec les rôles prédéfinis attribués par l'admin dans la section « Members », puis il confirme l'ajout.

(a) Saisie des informations du projet

Name	Email	Role
Khalil Turki	kturki@freedomofdev.com	QA
Khouloud BOUAZIZ	khouloud_bouaziz@enetcom.u-sfax.tn	DEV
Randa Amich	ramich@freedomofdev.com	DEV
Amir JMAL	amir.jmal@enetcom.u-sfax.tn	Admin
Rim MEZGHANNI	rmezghanni@freedomofdev.com	QA

(b) Chargement des membres et confirmation

FIGURE 4.6 – Processus d'ajout d'un projet

4.2.3.3 Gestion des projets : édition, archivage et restauration

L'interface de gestion des projets présentée dans la figure 4.7 permet au Super Admin de réaliser différentes actions via les icônes numérotées.

PROJECT NAME ↑↓	CREATION DATE ↑↓	STATUS	ACTIONS
FOD-2025-PFE-03-AUTOMATED-TEST-CASE-GENERATOR	May 29, 2025 at 09:28 PM	Active	1
FOD-2025-PFE-08-EVOLUTION-ASSISTANT-TOOL	May 30, 2025 at 11:40 AM	Active	2
FOD-2025-PFE-05-POWER-GRID-EXPLORER	May 30, 2025 at 11:41 AM	Archived	3
FOD-2025-PFE-03-AUTOMATED-TEST-CASE-GENERATOR-FOR-TEST-BOARD	May 30, 2025 at 11:57 AM	Active	

FIGURE 4.7 – Affichage des projets avec numérotation des actions possibles

L'icône 1 (voir figure 4.7) permet au Super Admin d'éditer un projet existant. Lors du clic, une fenêtre modale s'ouvre, présentant les données du projet (URLs, membres, historique des rapports). L'interface dispose d'un mode lecture (figure 4.8) et d'un mode édition (figure 4.9).

Edit Project Read Only ×

Project Name
FOD-2025-PFE-03-AUTOMATED-TEST-CASE-GENERATOR-FOR-TEST-BOARD

Azure DevOps Link
<https://dev.azure.com/fodservices/Training>

Board Link
https://fodservices.visualstudio.com/Training/_boards/board/t/FOD-2025-PFE-03-AUTOMATED-TES

Application URL (Optional)

Assigned Members

Name ↑↓	Email	Role
Khalil Turki	kturki@freedomofdev.com	QA
Khouleut RYIA717	khouleut.hounziz@anatom.us-cfex.fr	DEV

Edit

FIGURE 4.8 – Interface d'affichage d'un projet en mode lecture seule

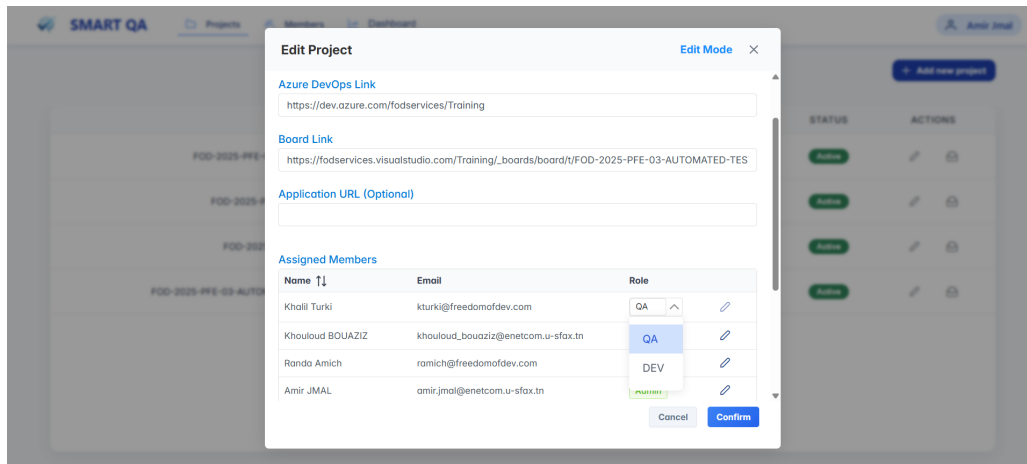


FIGURE 4.9 – Interface de modification d'un projet en mode édition

Le Super Admin peut modifier ce rôle et attribuer à chaque membre soit le rôle Dev, soit le rôle QA. Si le Super Admin souhaite changer un rôle d'un membre de Dev à QA, il doit confirmer cette action, car ce membre obtiendra les privilèges de QA.

Les icônes 2 et 3 (voir figure 4.7) permettent respectivement d'archiver et de restaurer un projet. Lorsqu'un projet est archivé (icône 2), il est retiré de la liste des projets actifs et n'est plus visible pour les QA ou Dev. L'icône 3 permet ensuite de restaurer le projet, le rendant à nouveau accessible.

4.2.4 Interfaces de lancement d'une exécution de test

Après s'être authentifié, le QA est redirigé vers l'interface listant ses projets, comme illustré par la figure 4.10.

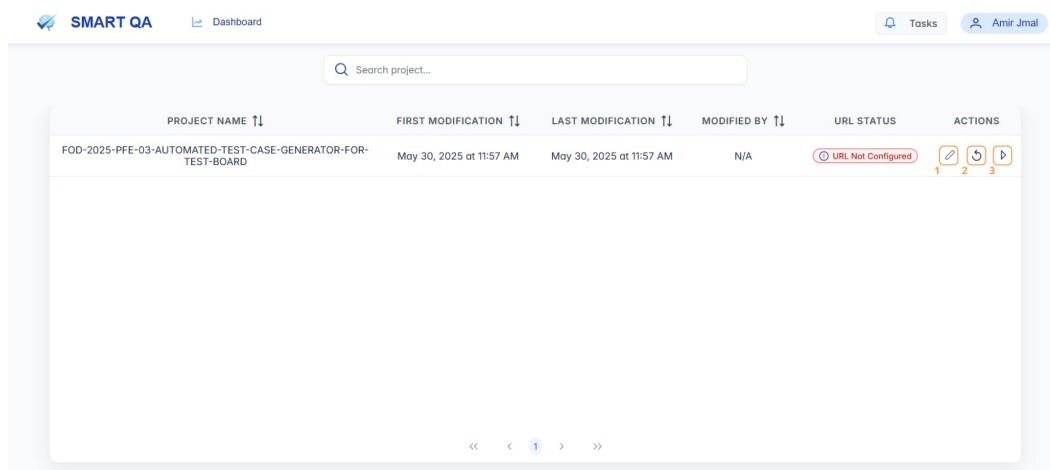
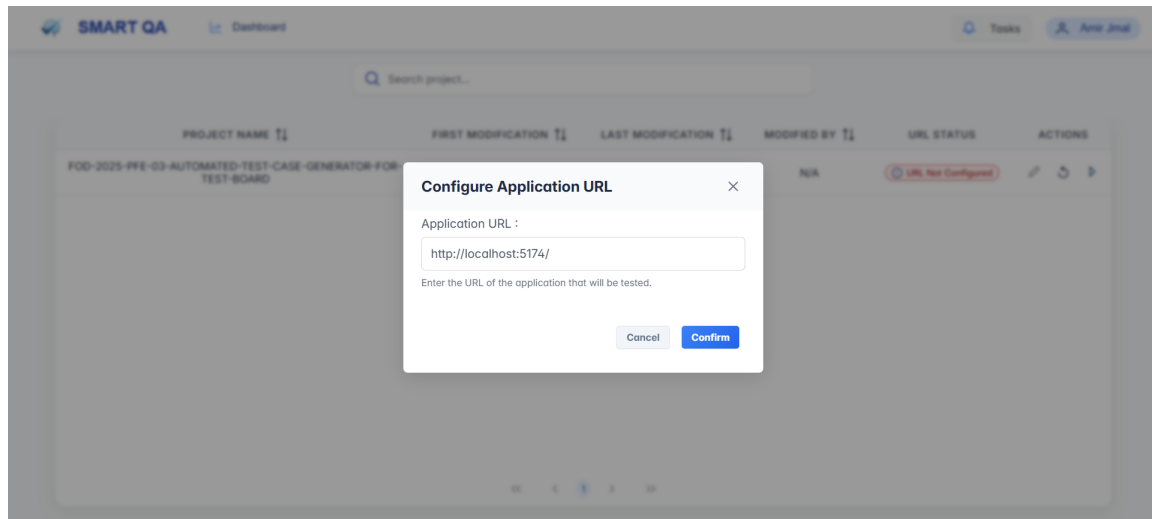


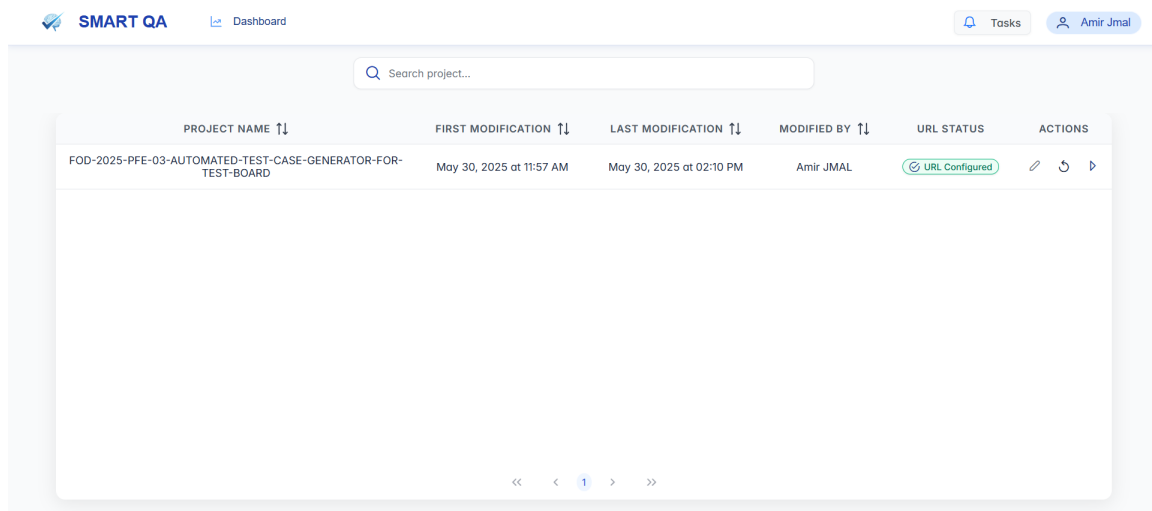
FIGURE 4.10 – Interface de liste des projets pour le QA

4.2.4.1 Configuration de l'URL de l'application à tester

L'icône 1 (voir figure 4.10) permet au QA de configurer ou d'éditer l'URL de l'application à tester. Une fois l'URL renseignée, elle sera utilisée lors de l'exécution des tests automatisés. La figure 4.11 illustre ce processus en deux étapes : la saisie de l'URL et son affichage après enregistrement.



(a) Saisie de l'URL de l'application à tester



(b) URL sauvegardée et affichée

FIGURE 4.11 – Configuration de l'URL de test par le QA

La figure 4.12 présente l'interface principale de l'application web que nous avons développée pour la phase de test automatisé. Cette application a servi de système à tester afin de valider le bon fonctionnement de Smart QA.

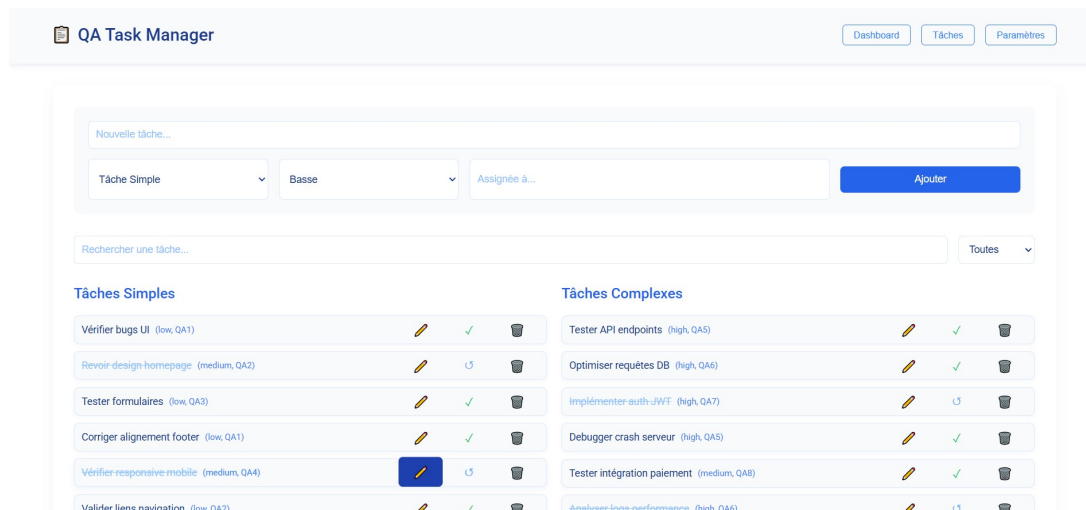


FIGURE 4.12 – Interface principale de l’application après connexion

4.2.4.2 Mise à jour des UserStories

L’icône 3 (voir figure 4.10) permet au QA de lancer une session de test. Il est alors redirigé vers l’interface principale de test, où il peut consulter les UserStories récupérées lors de l’ajout du projet par le Super Admin (figure 4.13). Le QA peut y rechercher une UserStory, consulter son statut et accéder aux détails.

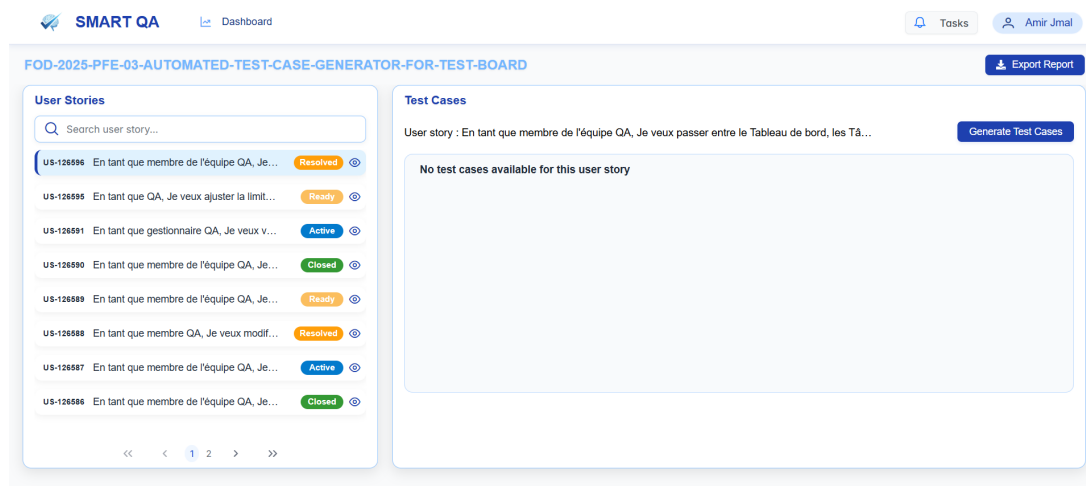


FIGURE 4.13 – Interface principale d’exécution de tests

L’icône 2 (voir figure 4.10) permet de mettre à jour la liste des UserStories depuis Azure DevOps. Toute modification (ajout, suppression, changement de statut) est alors reflétée dans l’interface. Le QA est informé des changements et peut les marquer comme lus, assurant une traçabilité continue des évolutions, comme présenté dans la figure 4.14.

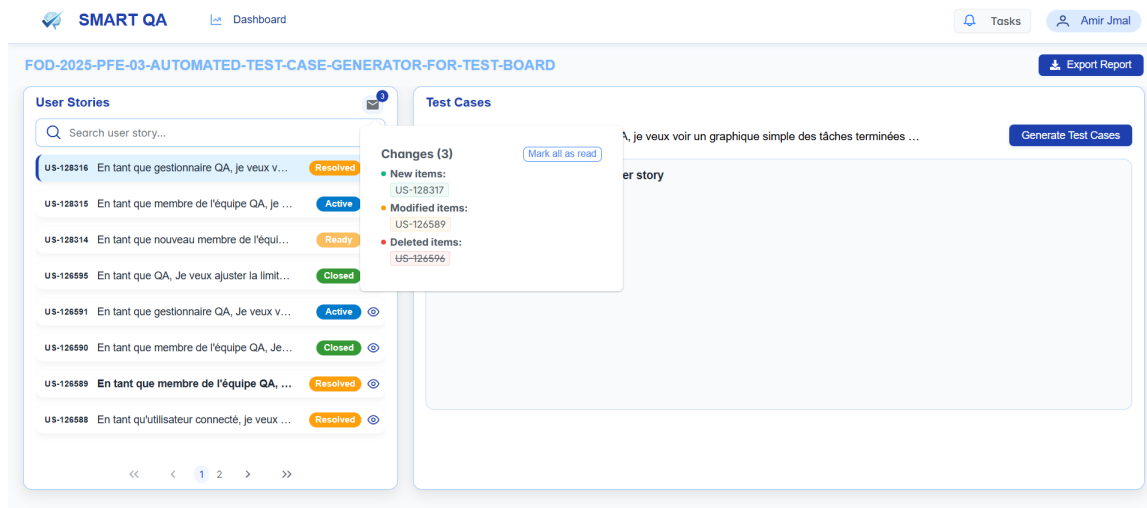


FIGURE 4.14 – Mise à jour des UserStories depuis Azure DevOps

4.2.4.3 Génération des cas de test et des scripts de test

Via le bouton "Generate Test Cases", les cas de test sont générés automatiquement par le LLM. Une fois le traitement terminé, ils sont affichés directement dans l'interface (figure 4.15). Ils apparaissent initialement en grisé, indiquant l'absence de script associé.

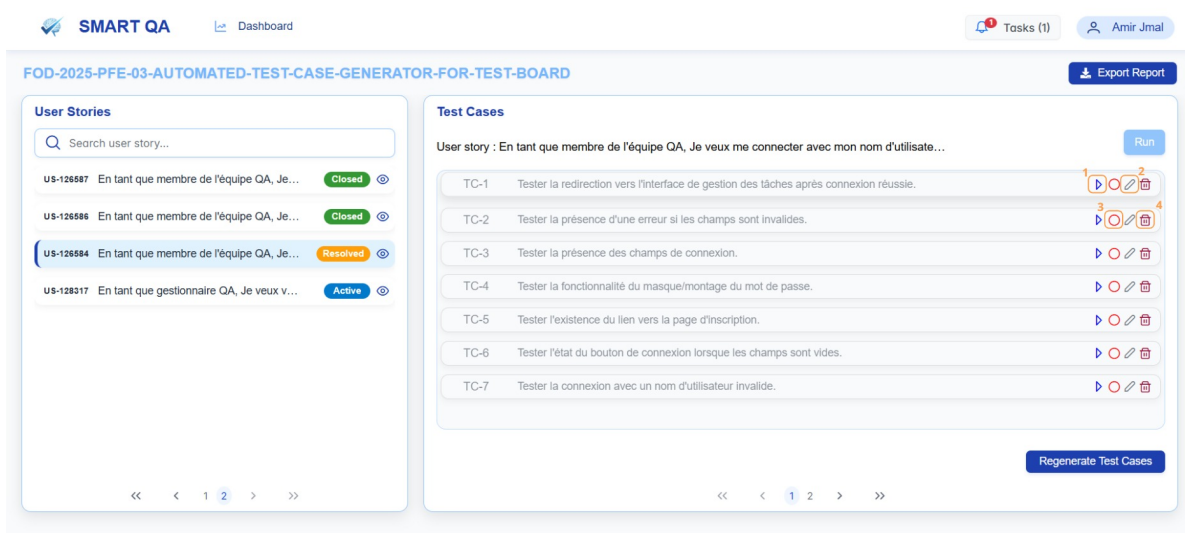


FIGURE 4.15 – Affichage des cas de test générés

L'icône 1 permet au QA de générer un script de test automatiquement. Une fois la génération réussie, l'apparence du cas de test change (il n'est plus grisé) et l'icône se transforme en une icône de régénération (figure 4.16).

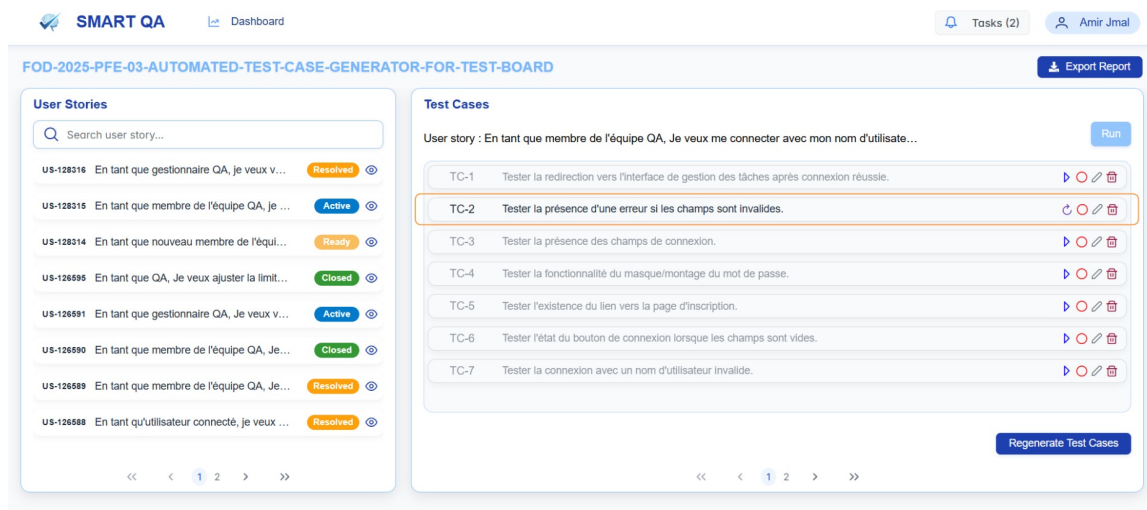


FIGURE 4.16 – Apparence des cas de test avant et après génération de script

À noter que la génération de script n'est autorisée que pour les UserStories dont le statut est *Resolved* ou *Closed*. En effet, les statuts *Ready* et *Active* indiquent que la UserStory est encore en phase de développement, ce qui rend la génération de script non pertinente. Cette restriction est illustrée dans la figure 4.17.

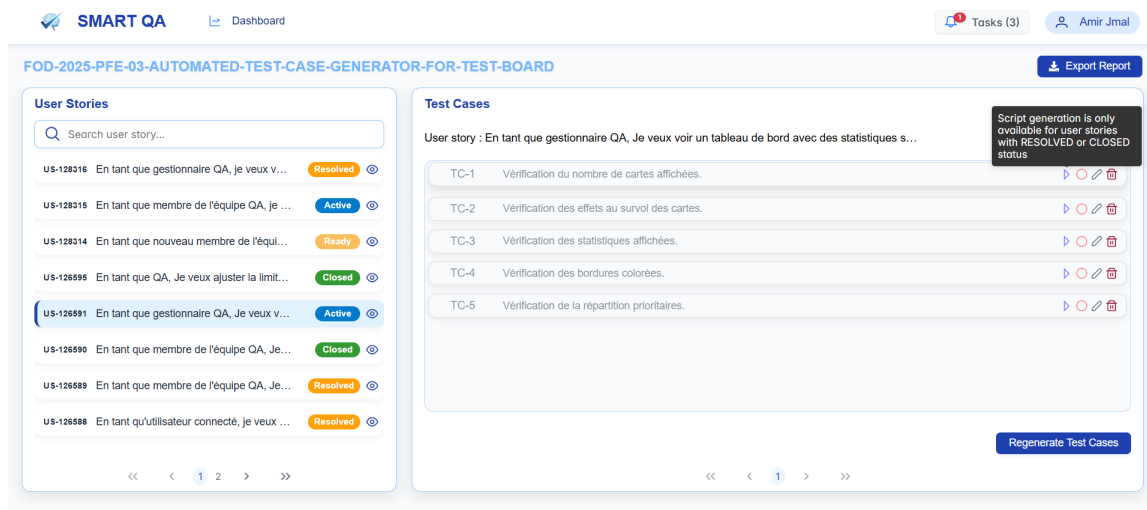
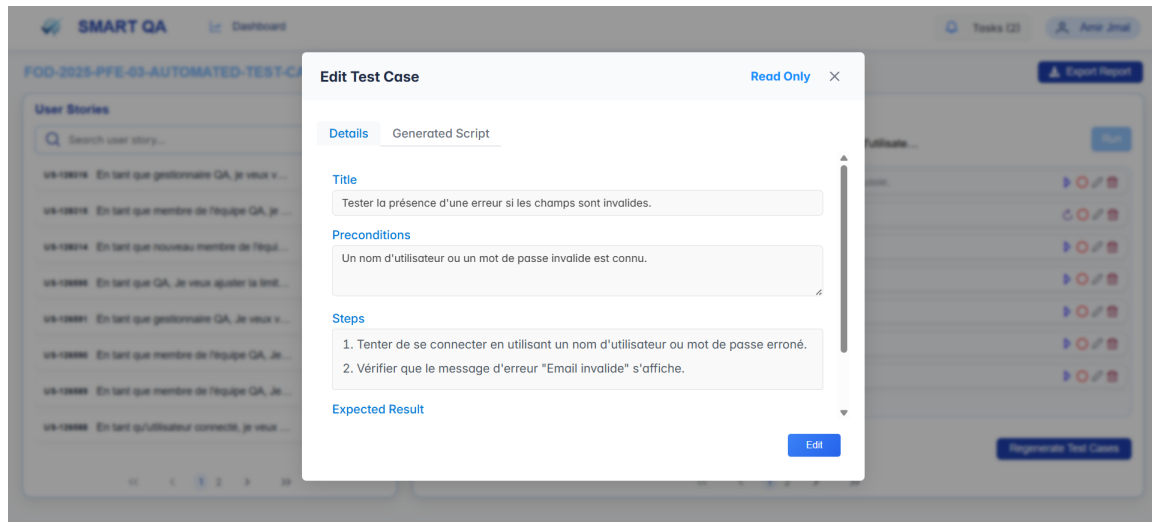


FIGURE 4.17 – Restriction de génération de script selon le statut de la UserStory

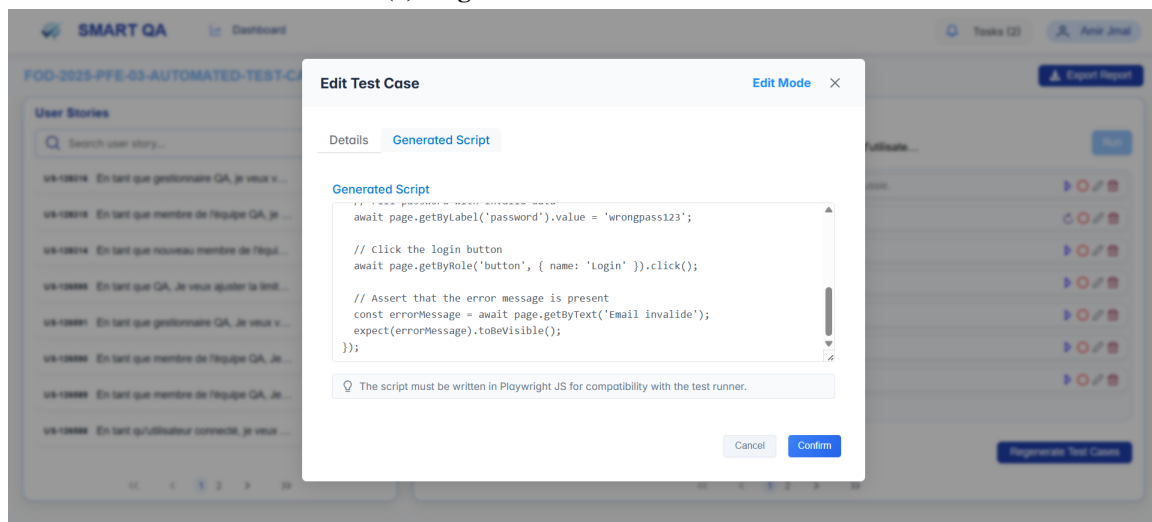
4.2.4.4 Édition et suppression des cas de test et de leurs scripts

L'icône 2 de modification (voir figure 4.15) ouvre une fenêtre composée de deux onglets : le premier pour les détails du cas de test (figure 4.18a) , le second pour le script généré (fi-

gure 4.18b). Chaque onglet dispose d'un mode lecture et d'un mode édition, permettant à l'utilisateur de consulter ou de modifier les informations.



(a) Onglet des détails du cas de test



(b) Onglet d'édition du script généré

FIGURE 4.18 – Affichage et modification d'un cas de test et de son script

Par ailleurs, l'**icône 4** (voir figure 4.15) permet au QA de supprimer un cas de test lorsqu'il est jugé non pertinent ou obsolète. Une confirmation est exigée avant toute suppression.

4.2.4.5 Lancement d'un enregistrement (recording) d'un cas de test

L'**icône 3** (voir figure 4.15) permet au QA de lancer un enregistrement manuel à l'aide de Playwright. Cela ouvre automatiquement l'interface de l'application (via l'URL configurée) et

déclenche le mode d'enregistrement de Playwright. Le QA peut alors effectuer ses interactions, qui serviront de référence pour le modèle LLM (figure 4.19).

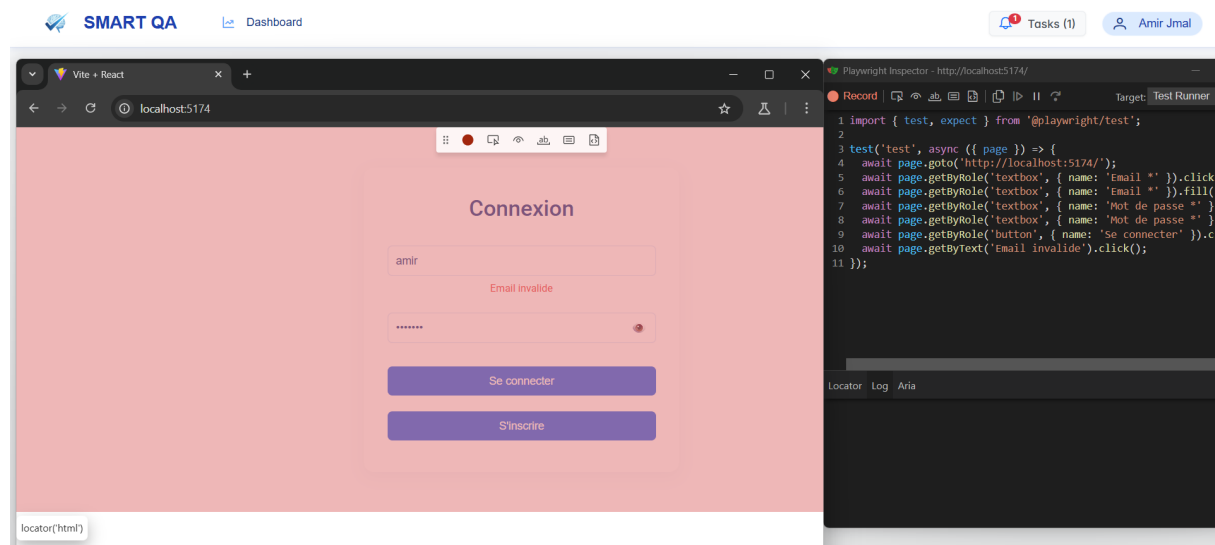


FIGURE 4.19 – Enregistrement manuel d'un scénario avec Playwright

4.2.4.6 Exécution automatisée et génération des rapports de test

Après la génération des scripts, le QA peut lancer l'exécution automatisée. Il sélectionne les cas de test prêts (ceux avec un script), puis clique sur le bouton "Run", qui s'active alors. L'exécution démarre et les résultats s'affichent en temps réel (Passed/Failed), comme montré dans la figure 4.20.

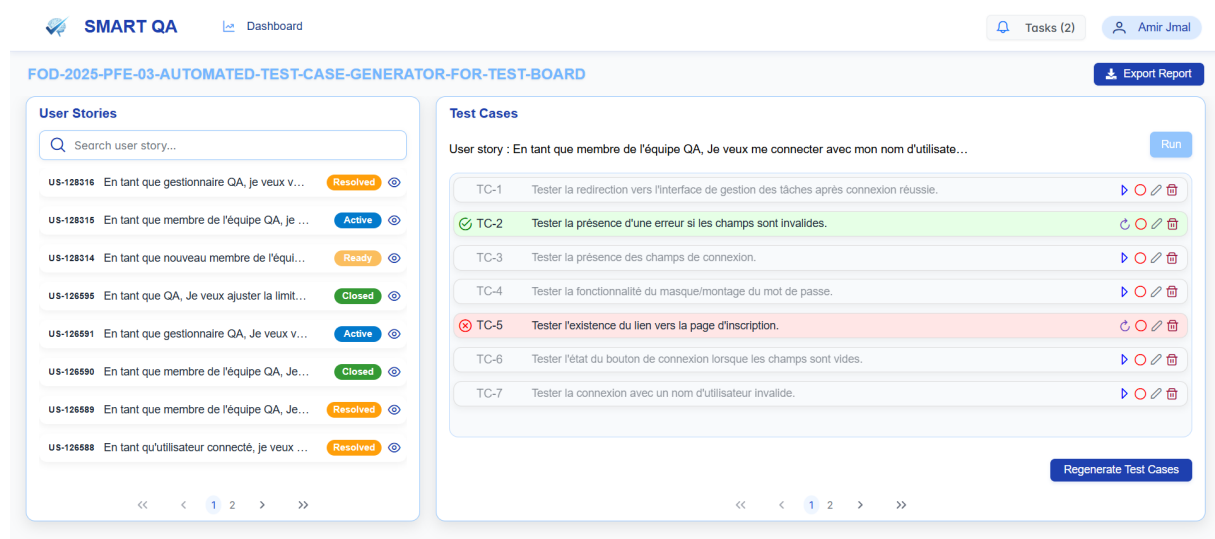
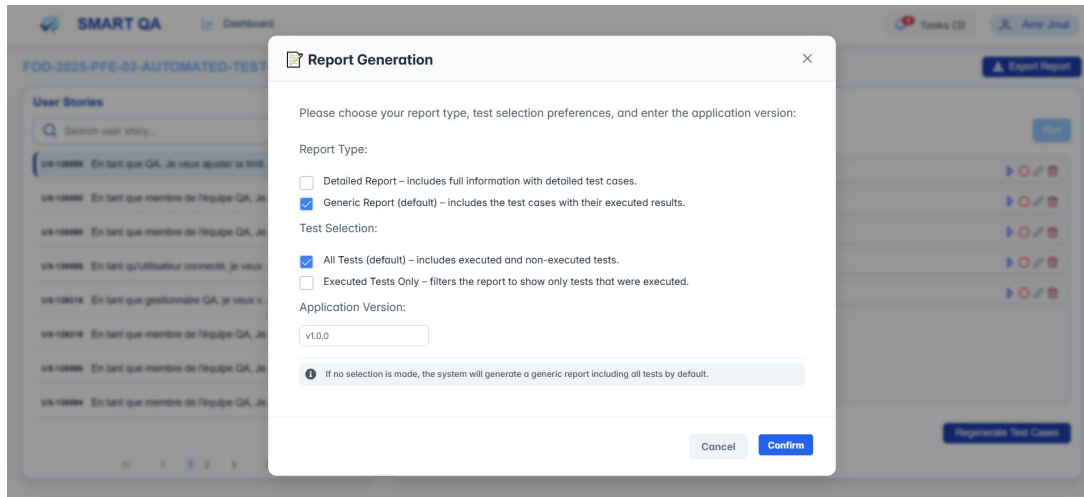
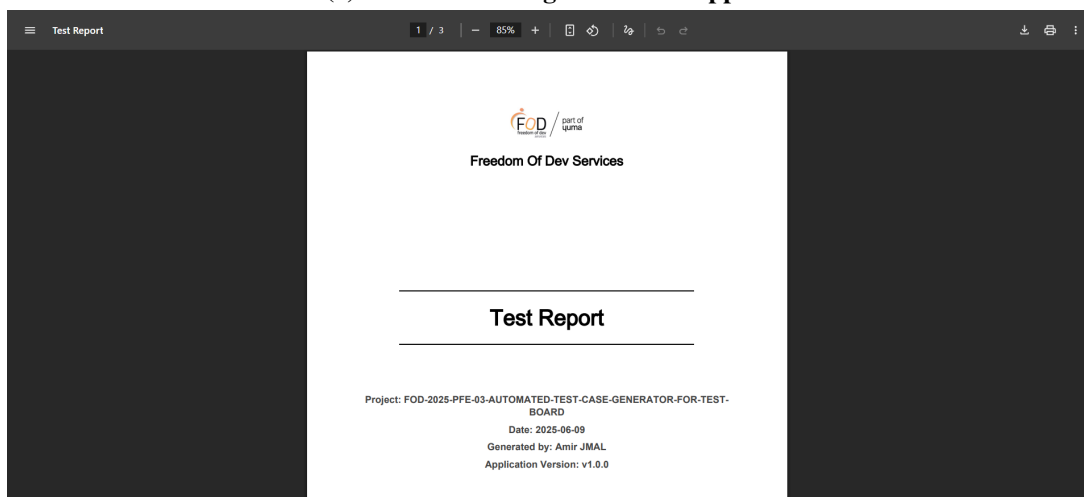


FIGURE 4.20 – Exécution des cas de test et affichage des résultats dans l'interface

En fin du processus, le QA génère un rapport de test via le bouton « Export Report ». Un clic ouvre une fenêtre de configuration (figure 4.21a) où le QA choisit la structure du rapport. Une fois validé, le rapport est généré et proposé au téléchargement (figure 4.21b).



(a) Modale de configuration du rapport.



(b) Téléchargement du rapport généré.

FIGURE 4.21 – Génération et téléchargement du rapport de test

Le rapport généré utilise un modèle personnalisé, converti en PDF avec la bibliothèque Flying Saucer [26]. Il contient les informations essentielles : nom du projet, nom du QA, date, version testée, statistiques de réussite/échec, et le détail des UserStories et cas de test. Un aperçu du rapport final est présenté dans la figure 4.22.

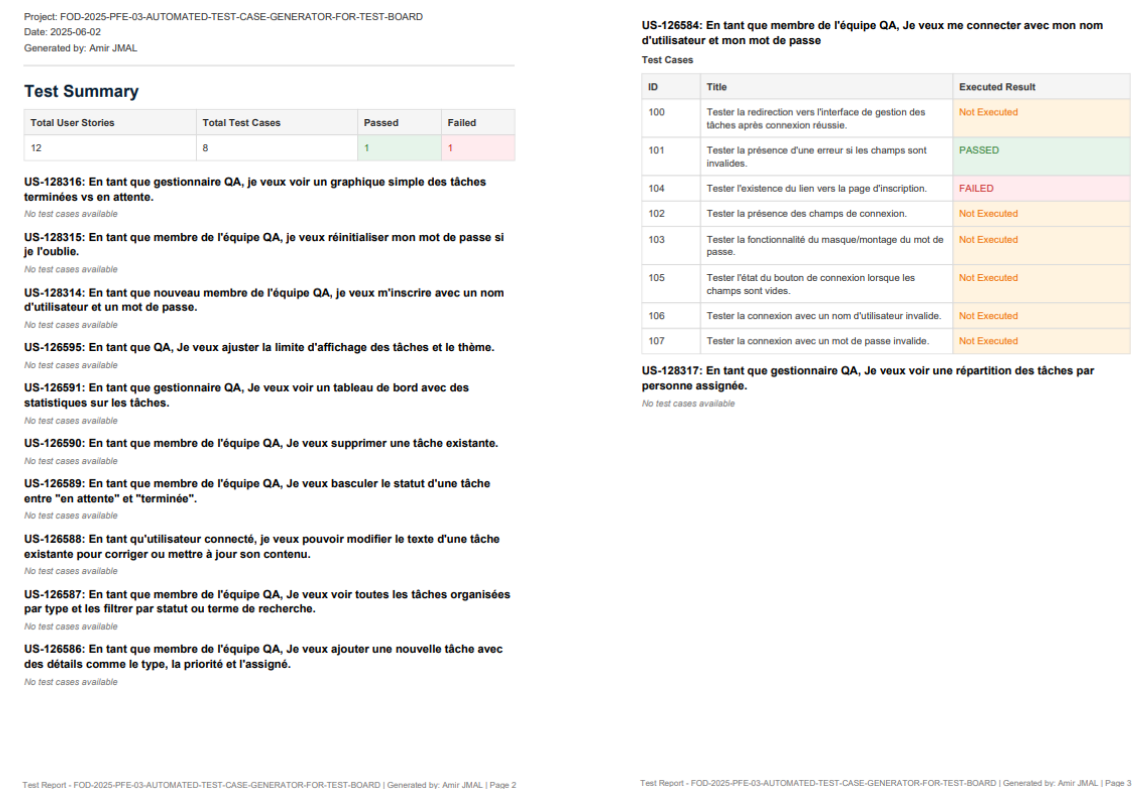


FIGURE 4.22 – Aperçu des différentes pages du rapport de test généré en PDF

4.2.5 Interface de consultation des projets pour le développeur

Après authentification, le développeur est redirigé vers une interface affichant la liste des projets qui lui sont assignés, comme illustré dans la figure 4.23

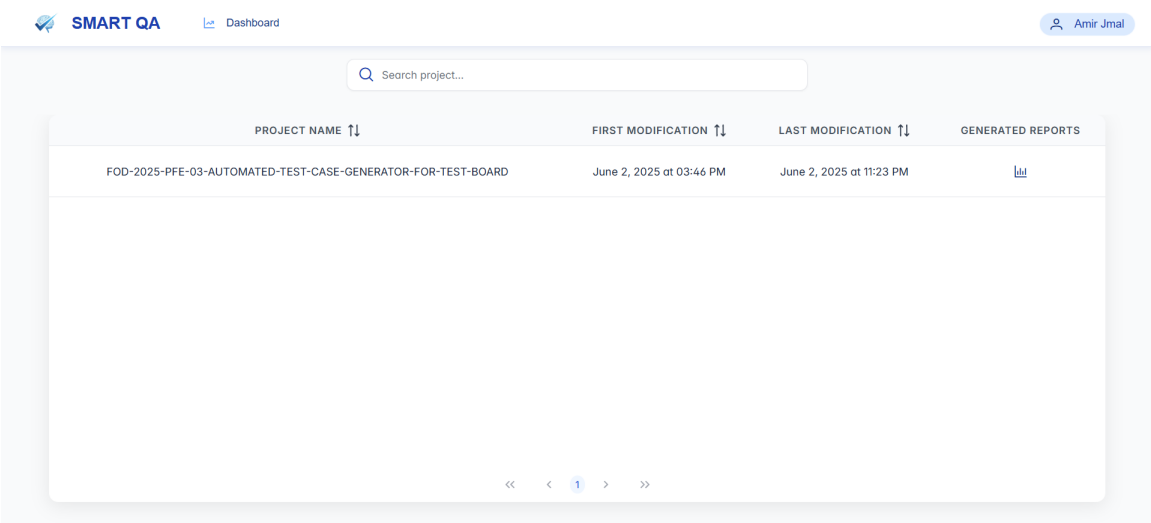


FIGURE 4.23 – Liste des projets assignés au développeur

Dans la colonne "Generated Reports", l'icône "View Reports" permet de consulter l'ensemble des rapports générés pour un projet, comme représenté dans la figure 4.24.

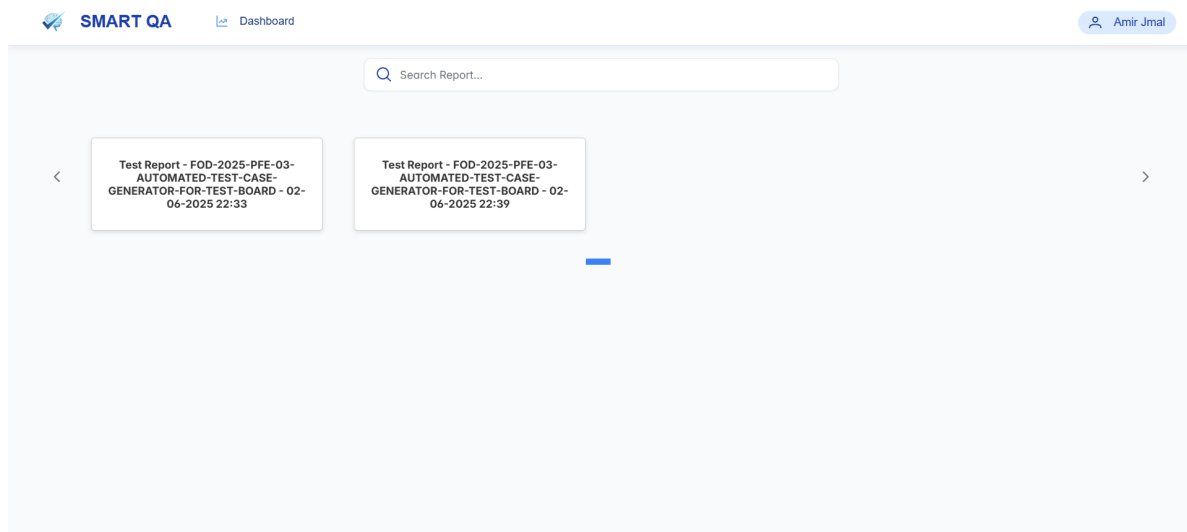


FIGURE 4.24 – Consultation des rapports générés pour un projet

L'utilisateur peut également effectuer une recherche par nom afin de retrouver rapidement un rapport spécifique et le visualiser en détail, ainsi que le montre la figure 4.25.

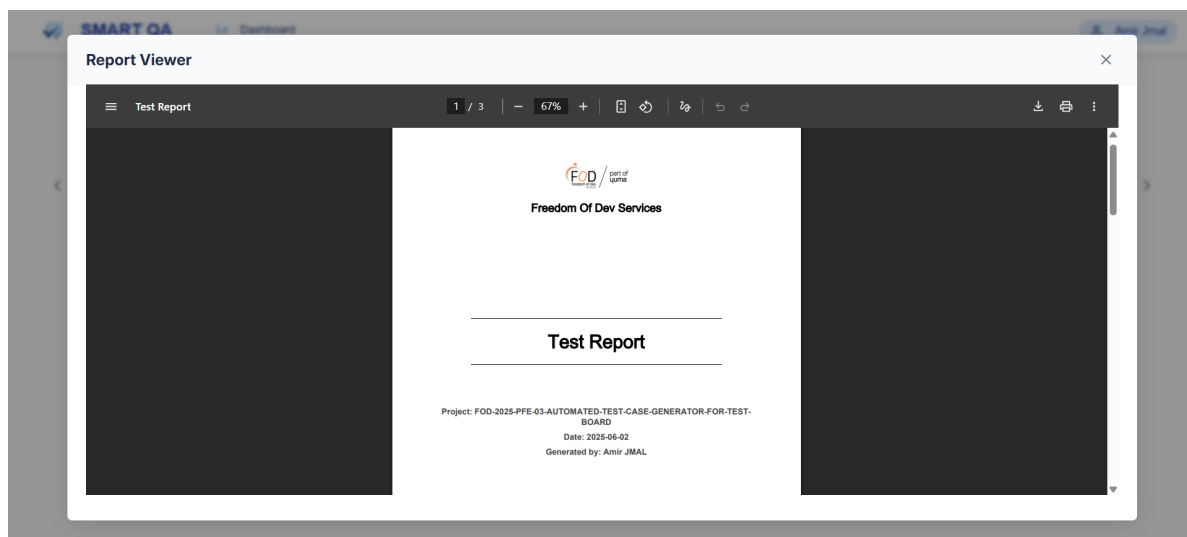


FIGURE 4.25 – Consultation d'un rapport spécifique

Enfin, tous les rapports sont archivés dans l'historique accessible par le Super Admin depuis l'interface d'édition d'un projet.

4.2.6 Interface du tableau de bord

Nous avons mis en place une interface de tableau de bord affichant plusieurs indicateurs clés (KPIs) liés à l'activité de test. Deux niveaux sont proposés :

- Un **tableau de bord global**, offrant une vue synthétique des KPIs sur l'ensemble des projets.
- Un **tableau de bord par projet**, affichant les métriques spécifiques à un projet sélectionné.

La figure 4.26 illustre la vue globale.

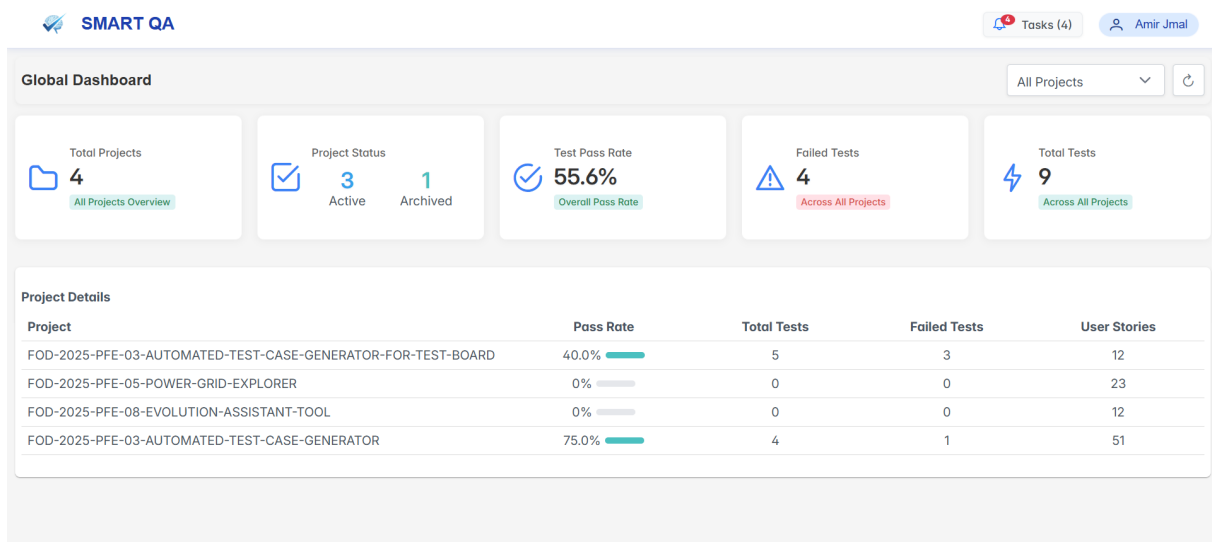


FIGURE 4.26 – Vue globale des indicateurs à travers tous les projets

En sélectionnant un projet, l'utilisateur accède à une vue détaillée présentant des informations spécifiques comme le nombre de cas de test, le taux de réussite (*pass rate*), et la liste des membres impliqués (figure 4.27).

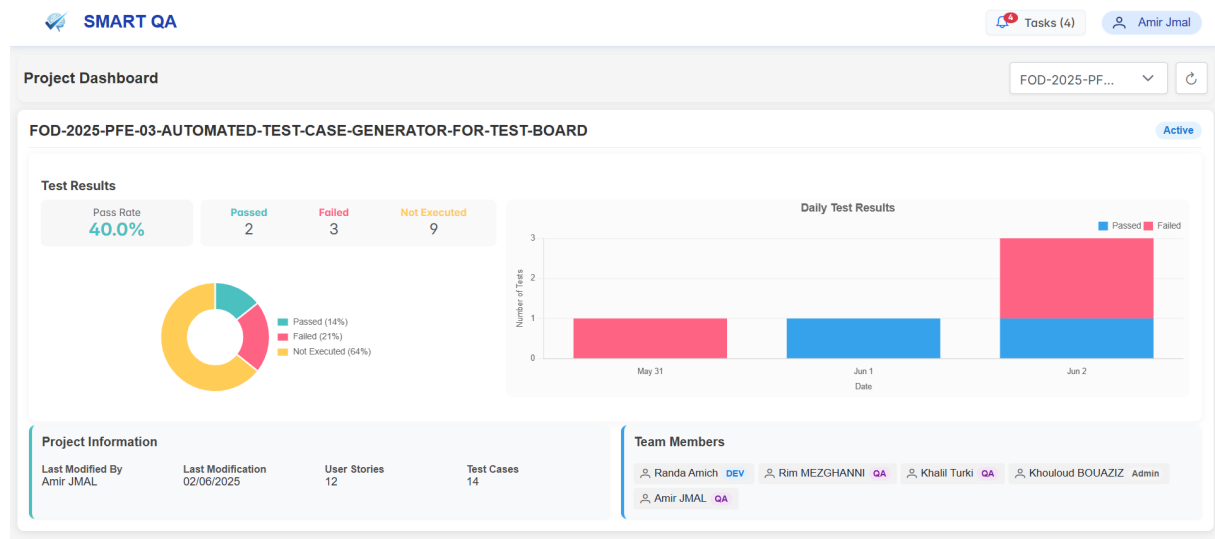


FIGURE 4.27 – Vue détaillée du tableau de bord d'un projet

4.3 CONCLUSION

Dans ce dernier chapitre, nous avons présenté le travail réalisé ainsi que les différentes interfaces de notre application Smart QA. Cette partie illustre concrètement les résultats obtenus au cours du projet. Pour clôturer ce rapport, nous proposons une conclusion générale accompagnée de quelques perspectives d'évolution pour la suite.



CONCLUSION GÉNÉRALE

Ce projet de fin d'études a consisté à développer *Smart QA*, une application intelligente intégrant l'intelligence artificielle et l'automatisation pour optimiser les processus de test des équipes QA et Dev, en centralisant les exécutions et leurs résultats.

Pour mener à bien ce projet, nous avons débuté par une phase d'analyse du métier, des problématiques et des objectifs, aboutissant à un cahier des charges structuré couvrant les besoins fonctionnels et non fonctionnels.

Notre application, développée selon une architecture microservices, permet de configurer les projets existants dans Azure DevOps et de collecter automatiquement les UserStories associées, tout en assurant leur mise à jour pour suivre l'évolution des besoins métier. Les équipes QA peuvent générer intelligemment les cas de test et les scripts correspondants, en s'appuyant sur des techniques d'intelligence artificielle. L'application prend également en charge l'exécution automatisée des tests via Playwright, la génération de rapports configurables, et offre un tableau de bord centralisant les indicateurs clés de performance (KPIs) de chaque projet.

Les objectifs fixés ont été globalement atteints. Toutefois, certaines pistes d'amélioration peuvent être envisagées, notamment l'ajout d'un module d'analyse d'anomalies pour les tests échoués, le renforcement des performances du modèle LLM via des techniques comme le RAG, le fine-tuning ou l'apprentissage par renforcement, ainsi que la mise en production dans l'environnement de travail pour une utilisation en conditions réelles.

En guise de conclusion, ce projet a constitué une passerelle concrète entre le monde universitaire et celui de l'entreprise. Il nous a permis d'approfondir notre compréhension des enjeux de l'automatisation des tests logiciels, de renforcer nos compétences en analyse, conception et implémentation de solutions complexes, tout en expérimentant le travail en équipe dans un cadre professionnel.



Webographie

- [1] **Freedom Of Dev.** <https://www.freedomofdev.com/>. (Dernier accès : Avril 2025).
- [2] **QA.** <https://www.amiltone.com/tech-place/qa>. (Dernier accès : Mai 2025).
- [3] **UserStory.** <https://www.atlassian.com/agile/project-management/user-stories>. (Dernier accès : Mai 2025).
- [4] **Azure DevOps.** <https://learn.microsoft.com/en-us/azure/devops/?view=azure-devops>. (Dernier accès : Mai 2025).
- [5] **Status des UserStories.** <https://learn.microsoft.com/en-us/azure/devops/boards/work-items/about-work-items?view=azure-devops&tabs=agile-process>. (Dernier accès : Juin 2025).
- [6] **Cas de test.** <https://www.geeksforgeeks.org/test-case/>. (Dernier accès : Avril 2025).
- [7] **Rapport de test.** <https://katalon.com/resources-center/blog/what-is-a-test-report>. (Dernier accès : Mai 2025).
- [8] **ScopeMaster.** <https://www.scopemaster.com/features/test-scenarios-generator/>. (Dernier accès : Mars 2025).
- [9] **AI Test Case Generator (Jira).** <https://marketplace.atlassian.com/apps/1232448/ai-test-case-generator-unlimited-for-jira?tab=overview&hosting=cloud>. (Dernier accès : Mai 2025).
- [10] **LLMs for test case generation.** <https://www.contextqa.com/useful-resource/chatgpt-test-case-generation/>. (Dernier accès : Avril 2025).
- [11] **Selenium.** <http://geeksforgeeks.org/pros-and-cons-of-selenium-as-an-automation-testing-tool/>. (Dernier accès : Mai 2025).
- [12] **Playwright.** <https://playwright.dev/docs/intro>. (Dernier accès : Juin 2025).

- [13] **TestComplete**. <https://testautomationtools.dev/pros-and-cons-of-smartbear-testcomplete-reviews-general-overview/>. (Dernier accès : Mai 2025).
- [14] **Test manuel vs Test automatisé**. <https://www.geeksforgeeks.org/software-engineering-differences-between-manual-and-automation-testing/>. (Dernier accès : Juin 2025).
- [15] **Méthode agile**. Robert Cecil Martin. Agile Software Development. Alan Apt, 2002.
- [16] **Processus Scrum**. <https://www.planzone.fr/blog/quest-ce-que-la-methodologie-scrum>. (Dernier accès : Avril 2025).
- [17] **UML**. <http://projet.eu.org/pedago/sin/term/3-UML.pdf>. (Dernier accès : Mai 2025).
- [18] **Architecture microservices**. <https://learn.microsoft.com/fr-fr/dotnet/architecture/microservices/architect-microservice-container-applications/microservices-architecture>. (Dernier accès : Mai 2025).
- [19] **NLP**. <https://datascientest.com/introduction-au-nlp-natural-language-processing> (Dernier accès : Juin 2025).
- [20] **LLMs**. <https://azure.microsoft.com/fr-ca/resources/cloud-computing-dictionary/what-are-large-language-models-llms>. (Dernier accès : Juin 2025).
- [21] **Prompt engineering**. <https://www.promptingguide.ai/>. (Dernier accès : Mai 2025).
- [22] **DeepSeek-R1-Distill-Llama-8B**. <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B>. (Dernier accès : Juin 2025).
- [23] **Ollama**. <https://ollama.com/>. (Dernier accès : Mai 2025).
- [24] **all-MiniLM-L6-v2**. <https://medium.com/@rahultiwari065/unlocking-the-power-of-sentence-embeddings-with-all-minilm-l6-v2-7d6589a5f0aa>. (Dernier accès : Mai 2025).
- [25] **Similarité cosinus**. <https://www.planzone.fr/blog/quest-ce-que-la-methodologie-scrum>. (Dernier accès : Avril 2025).
- [26] **Flying Saucer**. <https://github.com/flyingsaucerproject/flyingsaucer>. (Dernier accès : Mai 2025).

ANNEXES

A.1 Détails des fonctionnalités décrites dans le diagramme de cas d'utilisation

A.1.1 Fonctionnalité A : Configurer les projets

Cette fonctionnalité, réservée au Super Admin, couvre la configuration des projets, incluant leur création et modification. Elle permet également d'attribuer des rôles aux utilisateurs et d'ajouter des membres aux projets afin de gérer les droits d'accès, comme illustré par le diagramme de cas d'utilisation présenté dans la figure A.1.

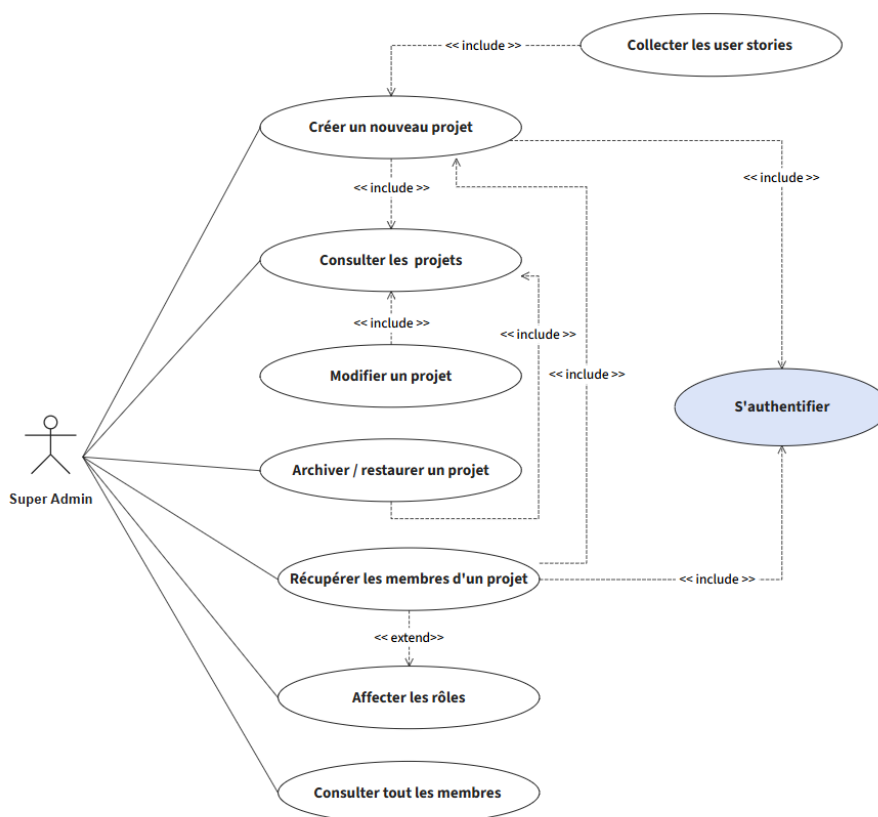


FIGURE A.1 – Diagramme des cas d'utilisation pour la configuration des projets

A.1.2 Fonctionnalité B : Générer des cas de test

Le QA est chargé de la génération intelligente des cas de test à partir des UserStories. Cette tâche repose sur l'utilisation de modèles de langage avancés pour créer des cas cohérents et contextualisés, comme indiqué dans la figure A.2.

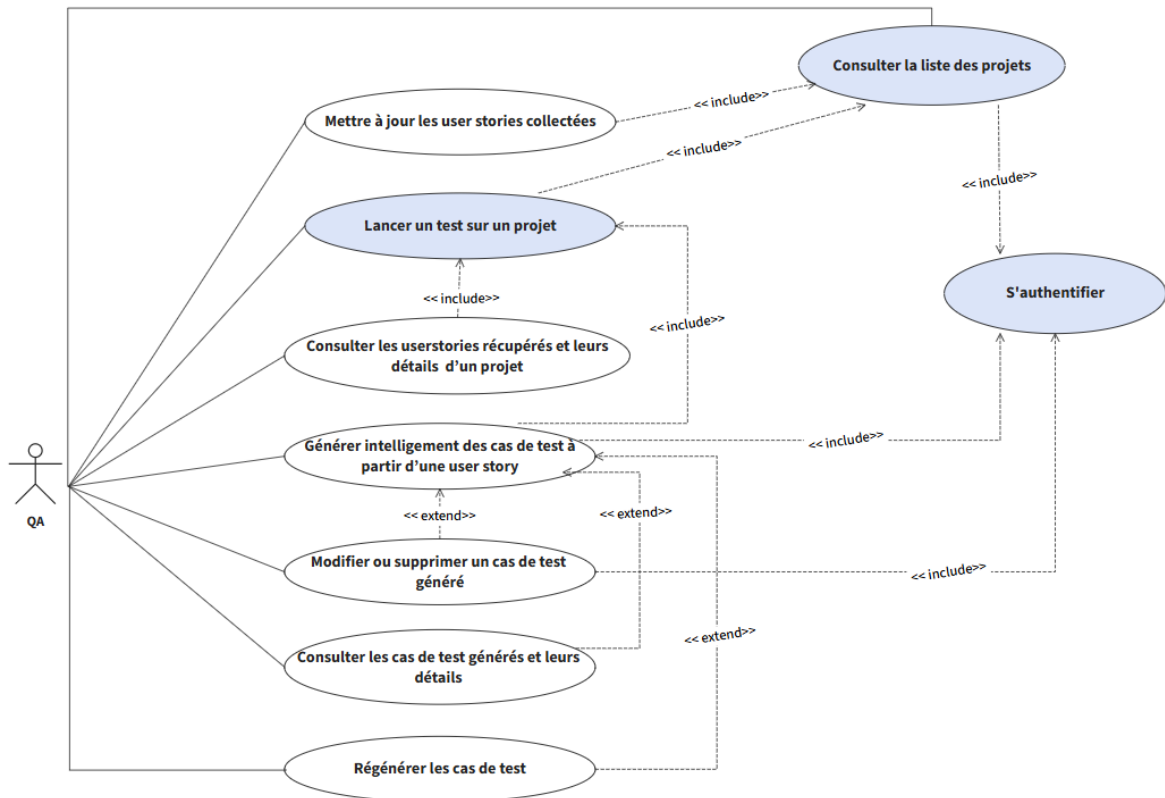


FIGURE A.2 – Diagramme des cas d'utilisation pour la génération des cas de test

A.1.3 Fonctionnalité C : Générer des scripts de test et exécuter les tests

Cette fonctionnalité permet au QA de générer de manière intelligente des scripts à partir des cas de test, puis de les exécuter afin de valider le comportement attendu du système, comme illustré dans la figure A.3.

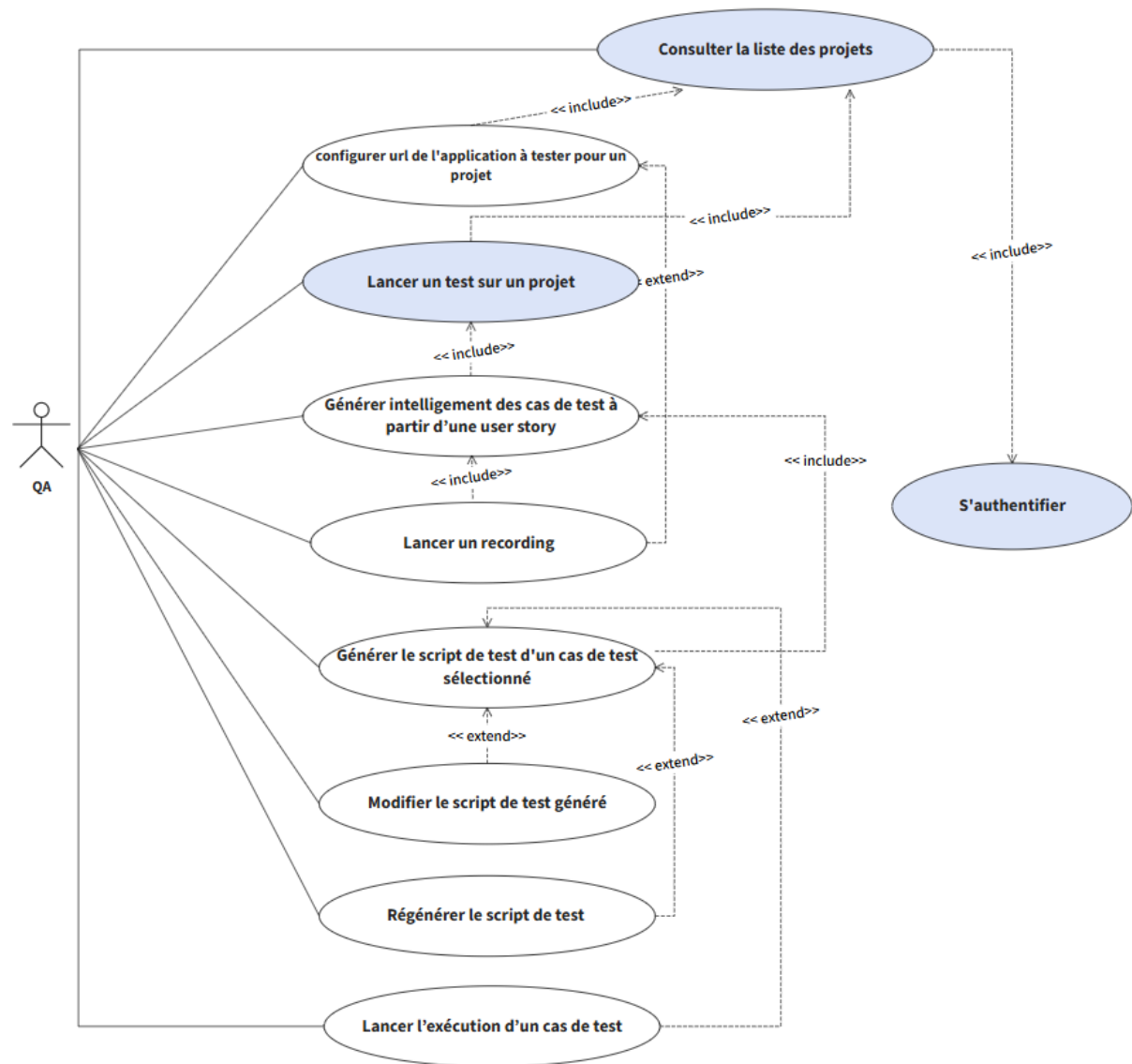


FIGURE A.3 – Diagramme des cas d'utilisation pour la génération de script et l'exécution des tests

A.2 Diagrammes de séquence de «Ajouter un projet»

Le diagramme de séquence du cas d'utilisation «Ajouter un projet» est représenté par la figure A.4

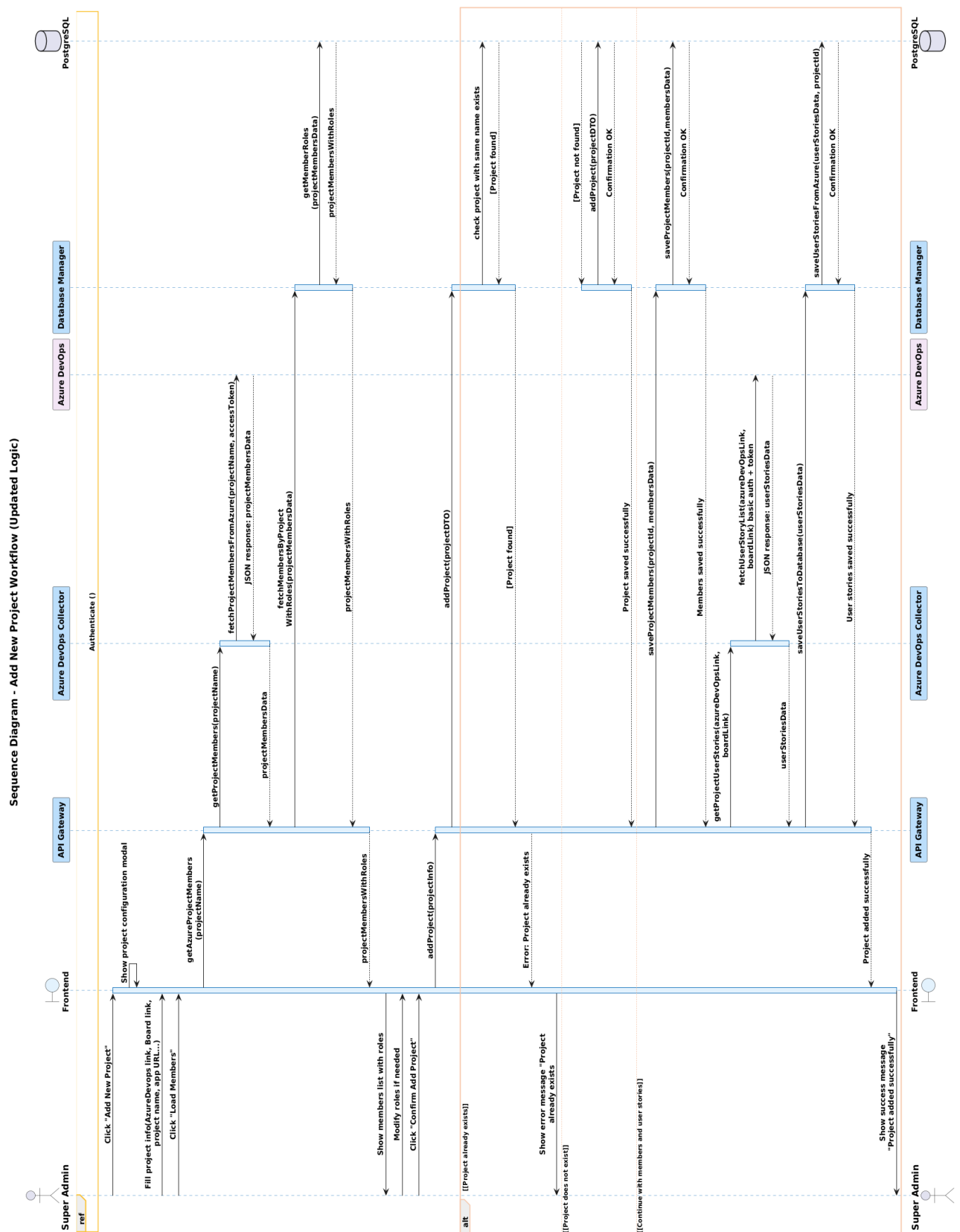


FIGURE A.4 – Diagramme de Séquence de «Ajouter un projet»

Développement d'une application intelligente pour la génération de cas de test et de scripts d'automatisation intitulée « Smart QA »

JMAL Amir

Résumé :

Ce projet de fin d'études s'inscrit dans le cadre de l'obtention du diplôme national d'ingénieur en ingénierie des données et systèmes décisionnels. Il porte sur la conception et le développement de Smart QA, une application web intelligente dédiée à l'optimisation et à l'automatisation de la phase de test logiciel pour les équipes QA de l'entreprise Freedom Of Dev Services (FOD). Ce produit vise à améliorer la qualité logicielle et à accélérer les cycles de livraison, en standardisant l'ensemble du processus de test : depuis la collecte des user stories, en passant par la génération intelligente des cas de test et de leurs scripts d'automatisation, l'exécution des tests, la centralisation des résultats, la génération de rapports, jusqu'au suivi des indicateurs clés de performance à travers un tableau de bord.

Mots Clés : Smart QA, tests logiciels, qualité logicielle, phase de test, automatisation, QA, génération intelligente, rapports, tableau de bord.

Abstract :

This work is developed as part of the graduation project to obtain the engineering degree in data engineering and decision-making systems. It focuses on the design and development of Smart QA, an intelligent web application dedicated to optimizing and automating the software testing phase for the QA teams at Freedom Of Dev Services (FOD). The product aims to improve software quality and accelerate delivery cycles by standardizing the entire testing process, from collecting user stories, through the intelligent generation of test cases and their automation scripts, test execution, results centralization, report generation, and the monitoring of key performance indicators via a dashboard.

Key Words : Smart QA, software testing, software quality, testing phase, automation, QA, intelligent generation, reports, dashboard.

تلخيص :

يندرج هذا العمل ضمن مشروع ختم الدروس للحصول على درجة الهندسة في هندسة البيانات وأنظمة صنع القرار. يركز المشروع على تصميم وتطوير Smart QA، وهو تطبيق ويب ذكي يهدف إلى تحسين وتشغيل آلي لمرحلة اختبار البرمجيات في فرق ضمان الجودة بشركة Freedom Of Dev Services (FOD). يهدف هذا المنتج إلى رفع جودة البرمجيات وتسريع دورات التسليم من خلال توحيد كامل عملية الاختبار، بدءًا من جمع قصص المستخدمين، مرورًا بالتوليد الذكي لحالات الاختبار ونصوص الأتمتة، تنفيذ الاختبارات، تجميع النتائج، إصدار التقارير، وحتى متابعة مؤشرات الأداء الرئيسية عبر لوحة معلومات متكاملة.

الكلمات المفتاحية : اختبارات البرمجيات، جودة البرمجيات، مرحلة الاختبار، التشغيل الآلي، فرق ضمان الجودة، التوليد الذكي، التقارير، لوحة المعلومات، مؤشرات الأداء.